

02 · 데이터 흐름: 갈라지는 지점

보강 안내 1/4 · 원시 기록, 분석 결과, 제조 알람을 분리

이 권의 핵심 질문

Kafka에 들어온 관측 한 건을 누가 읽고, 어떤 결과를 어느 토픽에 남기는가?

Kafka · sensor.telemetry.raw

Telegraf bridge가 만든 원시 관측. 태그·시각·품질을 보존한다.



갈래 A · 원시 이력

Telegraf sink가 raw를 InfluxDB process_raw에 쓴다. Flink를 거치지 않는다.



갈래 B · Flink

규칙·통계·시간 패턴·ONNX 모델이 계속 흐르는 값을 평가한다.



Flink 출력 ①

sensor.telemetry.clean: 중복·결측을 처리한 관측. 추정값은 quality로 표시.



Flink 출력 ② / ③

sensor.anomaly.score: 모델 점수 / sensor.alerts: 조건을 만족한 이상 이벤트.

흔한 오해

원시 이력이 반드시 Flink를 통과하는 것은 아니다. raw와 clean을 따로 보관해야 실측값과 보간값을 대조할 수 있다.

02 · 네 작업은 서로 다른 질문을 한다

보강 안내 2/4 · 규칙 / 통계 / 시간 순서 / 다변량 관계

Flink 작업	한 문장으로	교반기 사례에서
Threshold	허용 범위를 넘었나?	IT-102 전류-VT-101 진동의 정해진 경계
Z-Score	평소 분포에서 얼마나 벗어났나?	한 태그의 통계적 이탈
CEP	A 뒤 제한 시간 안에 B가 왔나?	전류 초과 뒤 진동 초과 순서
ONNX 모델	여러 센서의 관계가 평소와 다른가?	고정 규칙으로 설명하기 어려운 다변량 패턴

한 건의 알람이란?

Flink가 어떤 검출기·태그·심각도·시각으로 이상을 판정했는지 적은 메시지다. Kafka의 **sensor.alerts**에 기록한다. 점수 토픽은 계속 나가지만 알람은 기준을 만족할 때만 나올 수 있다.

모델과 언어 모델을 구분

여기서 ONNX는 센서 패턴의 이상 점수를 계산하는 모델 실행 형식이다. 문서에 답하는 언어 모델과 역할이 다르다.

02 · 알람은 한 곳에서 세 갈래로

보강 안내 3/4 · Kafka 토픽을 소비자마다 독립적으로 읽음

Flink → Kafka sensor.alerts

제조 이상 이벤트를 발행한다.



① Telegraf sink → InfluxDB

alerts 이력으로 저장한다. Grafana가 지난 알람을 조회한다.



② Telegraf alert-republish → EMQX

센서별 MQTT 원본과 scada/hmi/latest-alert 읽기용 메시지를 발행한다.



③ 독립 소비자

같은 Kafka 알람을 별도 소비자 그룹이 읽을 수 있다.

FUXA에 보이는 것은 무엇인가?

현재 HMI의 scada/hmi/latest-alert는 **마지막으로 도착한 분석 이벤트**다. 활성 알람 전체 목록이나 해제 판정은 아니다.

Kafka 소비자 그룹을 분리하면 한 소비자가 메시지를 읽었다고 다른 그룹의 읽기가 사라지지 않는다. 각 그룹의 offset과 중복 처리 기록은 별도로 관리해야 한다.

02 · 메시지 길을 확인하는 질문

보강 안내 4/4 · 실행 결과와 설명을 구분해서 읽기

질문	확인 위치	답을 잘못 읽는 예
원시값이 도착했나?	sensor.telemetry.raw 또는 process_raw	FUXA 숫자만 보고 전체 수집 성공이라 판단
Flink가 일하나?	잡 상태와 clean-score-alerts 출력	컨테이너 Up을 분석 성공으로 계산
알람이 발생했나?	sensor.alerts의 원본 이벤트	Grafana 차트 색상만 보고 탐지라 판단
화면까지 갔나?	MQTT latest-alert와 FUXA 표시	알람 이력 저장을 현재 HMI 표시로 오해

실제 코드와 대조한 근거

flink/sql · flink/onnx-job · telegraf/sink.conf · alert-republish.conf · kafka/create-topics.sh. 이 보강은 실행 재검증 보고가 아니다.

02

데이터 흐름·이상 탐지

기록된 흐름에서 규칙·순서·관계의 이상을 읽기

중복 설명을 통합한 3 권 읽기용 편집본

원문: 1_기술이해.pdf · 294 쪽

원문 기준일: 2026-09-17 | 편집일: 2026-09-21

이 권의 흐름

4 Kafka — 로그·파티션·복구·보존

5 스트림 처리 — 시간·상태·규격·Z-Score·보간

6 CEP — 패턴·부분 매치·시차 검증

7 머신러닝 이상 탐지 — 학습·ONNX 실행·판별력·한계

원리 → 실제 구현 → 예외·한계 → 대안의 순서를 유지합니다.

같은 표·설정·설명은 정본 위치에 모으고, 원문의 설명 차이는 편집 주석으로 구분했습니다.

읽기 전에

이 문서는 첨부된 교본의 내용을 읽기 편하게 재편집한 것입니다. 원문에 인용된 저장소나 시스템을 새로 실행·검증하지 않았습니다. 제품 버전, 설정값, 라이선스 언급, 성능 수치는 원문 작성 시점의 설명이며 현재 상태에 대한 별도 검증이 아닙니다.

본문의 “설계 보고서”는 교본이 인용한 15 쪽 자료를 뜻합니다. “원문” 또는 “원문 교본”은 첨부된 294 쪽 PDF 입니다. “원문의 별도 실습서”는 이 편집본의 2 권과 다른 자료입니다. 원문의 소절 번호를 가리키는 표현에는 “원문”을 붙였고, 장 번호 1~9는 그대로 유지했습니다.

원문에는 같은 주제가 다른 측정 구간이나 구현 단계에서 다시 등장합니다. 이런 수치·조건·예외는 단순 중복으로 지우지 않았습니다. 특히 설계 요구와 구현 완료, 상태의 정확히 한 번 처리와 외부 출력 중복, 데이터 감사 추적과 명령 감사 로그를 구분합니다.

이 권의 입력: `site·device·tag·ts·value·quality` 가 있는 관측 메시지입니다. 계측 태그와 단위, Modbus 주소·정규화·MQTT 세션의 전체 설명은 1 권에 있습니다. 여기서는 그 메시지의 기록·시간·상태·판정을 다룹니다.

표·코드·도해의 출처는 해당 장의 원문 범위입니다. 별도의 편집 주석은 원문 쪽수를 함께 표시합니다. 코드는 원리와 설정을 읽기 위한 발췌이며, 중간 생략이나 줄바꿈이 있는 블록을 그대로 실행하는 설치 매뉴얼은 아닙니다.

4 Kafka — 지우지 않는 장부

원문 교본 108~138 쪽의 설명을 통합

1 권에서 만든 공통 계약의 메시지는 Kafka 에 기록됩니다. 이 장은 메시지를 보관하는 구조, 파티션과 읽기 위치, 되감기와 복제, 그리고 실제 교육용 구성의 한계를 설명합니다.

왜 이런 물건이 생겼는가

Kafka 이전의 공장 데이터 시스템은 대개 직결 구조였습니다. 센서 수집 프로그램이 데이터베이스에 데이터를 직접 INSERT 하고, 대시보드가 그 데이터베이스를 조회했습니다.

INSERT 와 UPDATE 부터 풀겠습니다. 이 둘은 데이터베이스를 다루는 표준 언어인 SQL(에스큐엘)의 명령어입니다. 데이터베이스는 데이터를 표(테이블) 형태로 담습니다. 가로 한 줄이 행(row)이고 세로 칸이 열(column)입니다. INSERT 는 "표에 새 행을 한 줄 넣어라"이고, UPDATE 는 "이미 있는 행의 어떤 칸의 값을 다른 값으로 바꿔치기해라"입니다. 이 장 내내 중요한 것은 두 번째입니다. 데이터베이스 는 값을 덮어쓸 수 있고, 덮어쓰면 옛 값은 사라집니다.

직결 구조에는 세 가지 병이 있습니다.

첫째, 생산자와 소비자를 직접 이어 붙이면 소비자가 늘 때 기존 수집 프로그램도 고쳐야 합니다. 1 권의 발행/구독 설명에서 본 $N \times M$ 문제입니다.

둘째, 느린 쪽이 빠른 쪽을 멈춰 세웁니다. 데이터베이스가 디스크 정리를 하느라 5 분 동안 느려지면, INSERT 가 밀리고, 수집 프로그램의 버퍼(buffer, 완충 공간 — 보낼 것을 잠시 담아 두는 메모리)가 차고, 버퍼가 꽉 차면 그 다음 들어온 계측값은 버려집니다. 이 지체는 거슬러 올라가 현장 게이트웨이 까지 전파됩니다. 이렇게 하류의 지체가 상류로 거슬러 올라가는 현상을 역압(backpressure, 백프레서)이라고 합니다. 저장소가 잠깐 느려졌다는 이유로 센서 계측이 소실되는 것은 산업 현장에서 용납되지 않습니다.

셋째, 지나간 것을 다시 볼 수 없습니다. 전통적인 메시지 큐(RabbitMQ 등)는 소비자가 메시지를 가져가면 큐에서 지웁니다. 그래서 분석 프로그램에 버그가 있어 지난 6 시간 치를 잘못 처리했다면, 그 6 시간은 영영 사라집니다. 새 이상탐지 모델을 만들어 "지난주 데이터로 다시 돌려 보자"는 것도 불가능합니다.

Kafka 는 이 세 가지를 한 가지 발상으로 한꺼번에 풀니다. 메시지를 "배달하고 지우는 큐"로 다루지 않고, 끝에만 덧붙이는 기록 파일(append-only log, 덧붙이기 전용 로그)로 다루는 것입니다. 생산자는 기록부 끝에 한 줄 씩입니다. 소비자는 기록부를 읽되, 읽었다고 해서 그 줄을 지우지 않습니다. 소비자가 몇 명이든, 얼마나 느리든, 기록부는 영향을 받지 않습니다. 그래서 Kafka 를 분산 이벤트 스트리밍 플랫폼(distributed event streaming platform)이라고 부르며, 설계 보고서는 이 자리를 엔터프라이즈 백본(enterprise backbone, 전사 중추)이라고 부릅니다.

브로커, 클러스터, 로그, 오프셋

앞으로 계속 쓸 말부터 정하겠습니다.

브로커(broker)는 Kafka 서버 프로그램이 돌고 있는 한 대를 가리킵니다. 브로커가 데이터를 실제로 파일에 쓰고, 읽어 달라는 요청에 답합니다. 브로커 여러 대를 묶어 하나처럼 쓰는 것을 클러스터 (cluster, 묶음)라고 합니다. 이 장에서 "브로커가 ~한다"는 말은 전부 "Kafka 서버가 ~한다"는 뜻입니다.

여기서 말하는 로그(log)는 프로그램이 뱉는 오류 메시지가 아닙니다. 끝에만 덧붙일 수 있고, 중간을 고치거나 지울 수 없는 기록 파일을 가리키는 컴퓨터공학 용어입니다. 회계 장부와 같습니다. 잘못 적었다고 지우개로 지우지 않고, 아래에 정정 기록을 한 줄 더 적습니다.

그 로그에 줄이 하나 추가될 때마다 번호가 붙습니다. 이 번호가 오프셋(offset)입니다. 0 부터 시작하는 정수이고, 1 씩 증가하며, 한 번 붙으면 절대 바뀌지 않습니다. 오프셋을 붙이는 주체는 브로커입니다. 오프셋은 메시지의 주소입니다. "오프셋 184213"이라고 하면 그 로그에서 정확히 한 건을 가리킵니다.

Kafka 의 핵심은 읽은 위치를 브로커가 관리하지 않고 읽는 쪽이 기억한다는 점입니다. 소비자는 "나는 오프셋 184213 까지 처리했다"는 사실만 저장해 둡니다(이걸 커밋(commit, 기록 확정)이라고 합니다). 브로커는 누가 어디까지 읽었는지에 관계없이 데이터를 그대로 둡니다. 데이터가 지워지는 기준은 "누가 읽었느냐"가 아니라 "얼마나 오래됐느냐"입니다.

초보가 반드시 걸려 넘어지는 지점을 먼저 짚겠습니다.

- "읽으면 없어지나요?" — 아닙니다. 읽어도 그대로 있습니다.
- "커밋하면 지워지나요?" — 아닙니다. 커밋되는 것은 데이터가 아니라 읽은 위치입니다.
- "그럼 데이터베이스인가요?" — 아닙니다. 데이터베이스는 같은 행을 UPDATE 로 덮어씹습니다. Kafka 는 덮어쓰기가 없습니다. TT-101 온도가 72.4 → 72.6 으로 바뀌면, 72.4 를 고치는 게 아니라 72.6 이 라는 새 줄이 하나 더 생깁니다. "지금 값이 뭐냐"를 물어보는 용도가 아니라 "무슨 일이 어떤 순서로 있었느냐"를 남기는 용도입니다.
- "그럼 조회는 어떻게 하나요?" — Kafka 에는 내용으로 찾는 조회가 없습니다. "TT-101 이 70 도를 넘은 건을 찾아 줘"라고 물을 수 없고, 그런 질문에 답하려면 처음부터 순서대로 읽으면서 직접 걸러야 합니다. 그런 조회는 뒤에 붙는 InfluxDB(6 장)의 일입니다. 다만 위치로 찾는 조회는 있습니다. "이 시각 이후의 첫 메시지가 몇 번 오프셋이냐"는 브로커에게 물어볼 수 있고, 뒤에서 볼 --to-datetime 되감기가 바로 그 기능을 씁니다.

파일에 쓰는데 왜 느리지 않은가

여기서 초보가 거의 예외 없이 품는 의심이 있습니다. "디스크 파일에 쓴다면서 어떻게 초당 수만 건 을 감당하나?" 답은 세 겹입니다.

- ① 끝에만 덧붙이기 때문입니다. "파일은 느리다"는 통념은 임의 접근(random access, 아무 데나 골라 읽고 쓰기)을 전제로 한 말입니다. 파일 여기저기를 옮겨 다니며 고치면 저장장치가 매번 위치를 다시 찾아야 해서 느립니다. Kafka 는 중간을 고치는 일이 아예 없습니다. 항상 파일 끝에만 붙이고, 읽을 때도 대개 앞에서 뒤로 순서대로 훑습니다. 이런 순차 접근(sequential access)은 같은 저장장치에서도 임의 접근보다 훨씬 빠릅니다.
- ② 대부분 메모리에서 읽히기 때문입니다. 운영체제는 파일에 쓴 내용을 곧바로 디스크에 밀어 넣지 않고, 우선 페이지 캐시(page cache, 운영체제가 파일 내용을 담아 두는 메모리 영역)에 둡니다. 방금 기록된 메시지는 아직 그 메모리에 있고, 실시간으로 따라 읽는 소비자는 사실상 메모리에서 읽어 갑니다. 저장장치를 정말로 건드리는 것은 소비자가 한참 뒤쳐져 오래된 구간을 되짚어 읽을 때뿐입니다.
- ③ 묶어서 압축하기 때문입니다. 생산자는 메시지를 한 건씩 보내지 않고 짧은 시간 동안 여러 건을 모아 묶음으로 보냅니다. 그 묶음에 압축(compression)을 걸 수 있습니다. 토픽 설정 compression.type 이 그것이고, AR-100 은 lz4 를 씁니다. lz4 는 압축률을 조금 포기하는 대신 압축·해 제 속도가 매우 빠른 알고리즘이라, 초당 수만 건이 오가는 이런 자리에 잘 맞습니다. 압축된 상태 그대로 브로커의 파일에 저장되고 소비자가 받아서 풀기 때문에, 네트워크로 오가는 바이트와 디스크 사용량이 함께 줄어듭니다.

여기서 미리 짚을 것. ② 가 뜻하는 바는, "브로커가 기록했다"가 곧 "디스크에 안전하게 박혔다"는 뜻이 아니라는 것입니다. 대개는 페이지 캐시까지입니다. 뒤에 나오는 acks 설명이 이 사실 위 에 서 있습니다.

컨슈머는 받는 게 아니라 가져간다(폴링)

이것이 3 장의 MQTT 와 이번 장의 Kafka 를 가르는 가장 중요한 축인데, 겉으로는 잘 드러나지 않습니다.

MQTT 브로커는 밀어 줍니다(push). 구독자가 연결되어 있으면, 메시지가 도착하는 대로 브로커가 그 연결에 밀어 넣습니다. 받는 쪽이 느리면 브로커 안에 밀린 것이 쌓이고, 그 클라이언트용 세션 큐의 한도를 넘는 순간부터 버려집니다. 속도의 주도권이 보내는 쪽에 있습니다.

Kafka 는 정반대입니다. 컨슈머가 가져갑니다(pull). 브로커는 먼저 보내지 않습니다. 컨슈머가 "이 토픽 이 파티션의 오프셋 184213 부터, 최대 몇 바이트만큼 주세요"라고 요청(fetch)하고, 브로커가 그만큼 읽어서 답합니다. 컨슈머는 이 요청을 끊임없이 반복합니다. 이렇게 반복해서 물어보는 방식을 폴링(polling)이라고 하며, 실제 프로그램 코드에서도 poll() 이라는 이름의 함수를 반복 호출합니다.

이 한 가지 차이에서 이 장의 결론 대부분이 따라 나옵니다.

- 느린 소비자가 브로커를 압박하지 않습니다. 느린 컨슈머는 그저 덜 요청할 뿐이고, 브로커는 더 밀어 넣지 않습니다. 브로커의 메모리가 차오르지 않습니다.

- 밀린 것은 사라지지 않습니다. 아직 안 가져간 메시지는 브로커의 메모리에 대기하는 게 아니라 이미 로그 파일에 들어 있습니다. 그러므로 "밀렸다"가 곧 "버려졌다"가 아닙니다. 밀린 정도를 숫자로 나타낸 것이 뒤에 나올 랙입니다.

- 여러 소비자가 서로 다른 위치를 읽어도 됩니다. 브로커 입장에서는 요청받은 구간을 파일에서 읽어 주는 일이 전부이므로, 누가 어디를 읽든 상관이 없습니다.

초보가 헷갈리는 지점. "새 메시지가 없으면 빈 응답을 계속 받으며 CPU 를 태우는 것 아닌가?" 아닙니다. 컨슈머는 요청할 때 "새 데이터가 없으면 최대 이만큼은 기다렸다가 답하라"는 시간을 함께 보냅니다. 브로커는 그동안 응답을 붙들고 있다가 데이터가 생기면 즉시 돌려줍니다. 이 방식을 롱 폴링 (long polling, 긴 기다림 폴링)이라고 합니다.

Kafka 는 MQTT 를 말하지 못한다

MQTT 와 Kafka 는 "메시지를 주고받는다"는 점이 닮았지만, 서로 말이 통하지 않습니다. Kafka 클라이언트와 브로커는 Kafka 가 자체적으로 정의한 바이너리 TCP 프로토콜로 대화합니다. 설계 보고서의 기술 스택 표도 엔터프라이즈 백본 행의 통신 방식을 "Kafka 바이너리 TCP 프로토콜"이라고 명시적으로 적습니다.

용어를 풀면 이렇습니다. TCP 는 인터넷에서 두 프로그램이 순서와 도착을 보장하며 데이터를 주고받는 기본 규약입니다. 바이너리(binary)는 사람이 읽을 수 있는 글자가 아니라 기계가 읽는 바이트 배열로 주고받는다 뜻입니다. 즉 "메시지 보내기 요청은 몇 번, 그 뒤에 토픽 이름 길이가 몇 바이트, 그 다음이..." 하는 식으로 자리와 순서가 정해져 있습니다. MQTT 도 TCP 위의 바이너리 프로토콜이지만, 형식이 전혀 다른 별개의 규약입니다.

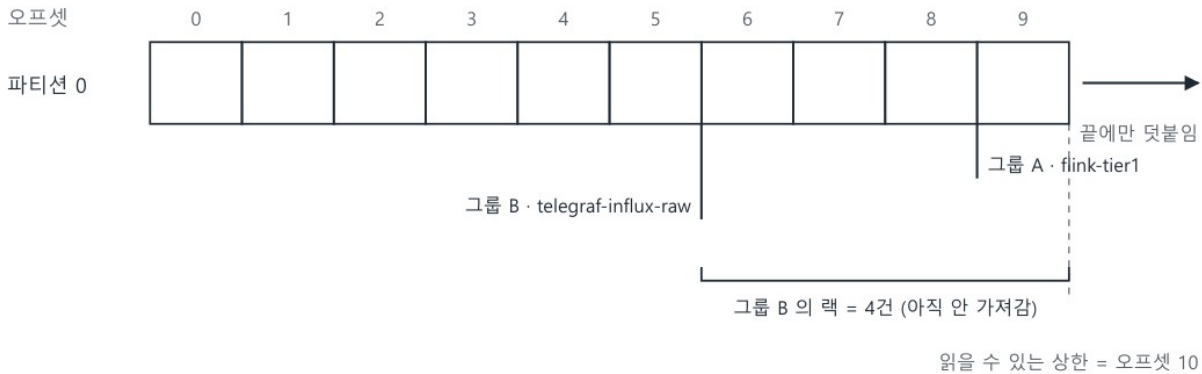
여기서 두 가지 결론이 나옵니다.

- ① 그래서 "브리지"라는 물건이 필요합니다. MQTT 로 들어온 메시지를 Kafka 에 넣으려면, 두 프로토콜을 모두 말할 줄 아는 프로그램이 중간에 서야 합니다. 이 프로그램을 브리지(bridge, 다리)라고 합니다. 브리지는 MQTT 브로커에 구독자로 붙어 메시지를 받고, 내용을 원하는 형태로 바꾼 뒤, Kafka 생산자로서 토픽에 씁니다. AR-100 에서 이 자리에 선 것이 Telegraf 입니다(아래에서 자세히 다룹니다).

- ② 연결의 성격이 다릅니다. MQTT 는 센서 한 대 한 대가 각자 연결을 갖는 것을 전제로 설계되어, 수 만~수십만 개의 가느다란 연결을 감당합니다. 반면 Kafka 클라이언트는 연결을 오래 유지하면서 메시지를 묶어 주고받는 무거운 물건이라, 현장 센서 수만 대가 각자 Kafka 에 직접 붙는 구성은 하지 않습니다. 뒤에 나올 비교표의 "수십 개의 굵은 연결"이 이 뜻입니다.

두 프로토콜은 서로 말이 통하지 않으므로 사이에 브리지가 필요합니다. AR-100 에서 그 자리는 telegraf-bridge 컨테이너입니다.

토픽 sensor.telemetry.raw



원문 도해 · 오프셋과 소비자 그룹별 읽은 위치 (원문 114 쪽)

토픽, 파티션, 그리고 순서가 보장되는 범위

토픽(topic)은 이름표가 붙은 장부입니다. sensor.telemetry.raw 처럼 이름을 붙여 용도별로 나눕니다. 데이터베이스의 테이블에 해당하는 위치라고 생각하면 처음 이해에 도움이 됩니다.

그런데 장부 한 권을 한 사람이 쓰고 한 사람이 읽으면, 그 한 사람의 속도가 시스템 전체의 상한이 됩니다. 그래서 Kafka 는 토픽을 여러 조각으로 쪼갭니다. 이 조각 하나하나가 파티션(partition, 칸막이로 나눈 구획)이고, 실제 로그 파일은 토픽이 아니라 파티션마다 하나씩 존재합니다. 오프셋도 토픽 단위가 아니라 파티션 단위로 매겨집니다. 파티션 0의 오프셋 100 과 파티션 1의 오프셋 100은 전혀 다른 메시지입니다.

여기서 Kafka 전체를 통틀어 가장 많이 오해받는 사실이 나옵니다.

순서는 파티션 안에서만 보장됩니다. 토픽 전체의 순서는 보장되지 않습니다.

파티션 0에 10시 00분 01초 메시지가 들어가고 파티션 1에 10시 00분 00초 메시지가 들어갔다면, 읽는 쪽에서 둘 중 무엇을 먼저 보게 될지는 정해져 있지 않습니다. 이것은 Kafka의 결함이 아니라 확장 의 대가입니다. 전 세계의 순서를 지키려면 줄을 하나만 세워야 하고, 줄이 하나면 여러 일꾼이 동시에 처리할 수 없습니다.

수평 확장(horizontal scaling, 스케일 아웃)이라는 말이 여기서 나옵니다. 한 대를 더 비싼 기계로 바꿔 성능을 올리는 것을 수직 확장이라 하고, 같은 일을 나눠 할 대수(또는 일꾼 수)를 늘려 성능을 올리는 것을 수평 확장이라고 합니다. Kafka에서 그 "나눌 수 있는 단위"가 바로 파티션입니다. 파티션을 늘리면 쓰는 쪽도 읽는 쪽도 동시에 여럿이 달라붙을 수 있습니다. 그래서 설계 보고서가 Kafka의 역할을 "파티셔닝 기반 선형 확장, 다중 컨슈머 격리"라고 적은 것입니다.

그렇다면 공정 데이터는 어떻게 순서를 지킬까요. 여기서 키(key)가 등장합니다.

해시, 그리고 키에 따른 파티션 배정

먼저 해시(hash)를 풀겠습니다. 해시는 아무 길이의 데이터를 정해진 크기의 숫자 하나로 바꾸는 계산입니다. 그 결과 숫자를 해시값이라고 합니다. 성질이 두 가지입니다.

- 같은 입력은 언제나 같은 숫자를 냅니다. TT-101을 몇 번을 넣어도 결과는 같습니다. 어느 컴퓨터에 서 계산해도 같습니다.

- 다른 입력은 대체로 제각각 흩어진 숫자를 냅니다. TT-101과 TT-102처럼 한 글자만 달라도 결과 숫자는 전혀 다른 값이 나옵니다. 비유하자면 이름을 넣으면 사물함 번호가 나오는 기계입니다. 같은 사람은 늘 같은 사물함을 받고, 사람이 다르다면 대체로 서로 다른 사물함으로 흩어집니다.

생산자는 메시지를 보낼 때 키를 함께 붙일 수 있습니다. 이때 파티션을 고르는 일은 전적으로 생산자 쪽 프로그램(정확히는 생산자 클라이언트 라이브러리)이 합니다. 브로커는 관여하지 않습니다. 앞에서 "오프셋은 브로커가 붙인다"고 했으니 헷갈리기 쉬운데, 두 일은 주체가 다릅니다.

일	하는 주체	시점
어느 파티션에 넣을지 고르기	생산자 클라이언트	보내기 직전

일	하는 주체	시점
그 파티션에서 몇 번 오프셋을 줄지	브로커	로그에 덧붙이는 순간

생산자 클라이언트가 파티션을 고르는 규칙은 키의 해시값을 파티션 개수로 나눈 나머지만큼입니다. 수식으로는 $\text{partition} = \text{hash}(\text{key}) \% \text{partition_count}$ 입니다. % 는 나눗셈의 나머지를 구하는 기호입니다. 예를 들어 해시값이 10007 이고 파티션이 6 개면 $10007 \% 6 = 5$, 즉 파티션 5 로 갑니다. 나머지는 항상 0 부터 5 사이이므로 어떤 해시값이 나와도 유효한 파티션 번호가 됩니다.

이 규칙에서 따라 나오는 결론이 결정적입니다. 같은 키를 가진 메시지는 언제나 같은 파티션으로 갑니다. 그러므로 같은 키끼리는 순서가 지켜집니다. 키를 TT-101 (반응기 온도 태그)로 잡으면, TT-101 의 계측값들끼리는 절대 뒤섞이지 않습니다. 반면 TT-101 과 PT-101 사이의 상대적 순서는 보장되지 않습니다. 그리고 공정 데이터에서 필요한 순서는 대개 후자가 아니라 전자입니다. 온도 이력은 온도끼리 순서대로면 충분합니다.

키를 붙이지 않으면 어떻게 될까요. 생산자는 파티션을 돌아가며(또는 묶음 단위로) 아무 데나 넣습니다. 부하는 고르게 퍼지지만 순서 보장은 완전히 사라집니다. 그래서 "키를 뭇로 잡느냐"는 Kafka 설계 에서 가장 먼저 결정해야 하는 항목입니다.

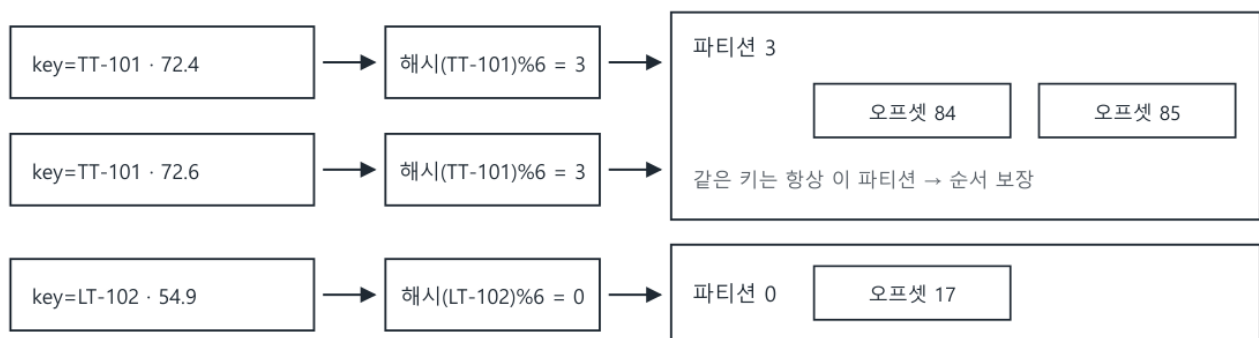
초보가 반드시 헷갈리는 지점. 해시는 고르게 나눠 주지 않습니다. 키 12 종을 파티션 6 개에 넣으면 "2 개씩 들어가겠지"라고 읽기 쉬운데, 그렇지 않습니다. 해시값이 어디로 떨어질지는 키마다 제각각이므로, 어떤 파티션에는 3 개가 들어가고 어떤 파티션에는 하나도 안 들어갈 수 있습니다. 해시가 보장하는 것은 "같은 키는 같은 곳"이지 "고르게 흩어짐"이 아닙니다. 키 종류가 수천·수만이 되면 통계적으로 고르게 퍼지지만, 열두 개 정도로는 그렇지 않습니다.

파티션 개수를 정하는 기준도 여기서 나옵니다. 파티션 수는 한 소비자 집단이 동시에 굴릴 수 있는 읽기 일꾼 수의 상한입니다. 파티션이 6 개면 일꾼을 7 명 붙여도 한 명은 놀입니다. 그래서 파티션 수는 "지금 필요한 병렬도"가 아니라 "앞으로 감당할 병렬도"를 보고 잡습니다.

생산자 telegraf-bridge

파티션 배정 (생산자 쪽)

브로커의 로그



원문 도해 · 키에 따른 파티션 배치 (원문 116 쪽)

주의. 파티션 개수는 나중에 늘릴 수는 있지만, 늘리는 것이 안전하지 않습니다. 파티션이 6 개에서 12 개로 바뀌면 $\text{hash}(\text{key}) \% 6$ 과 $\text{hash}(\text{key}) \% 12$ 의 답이 달라져서, 같은 TT-101 이 어제는 파티션 3, 오늘은 파티션 9 로 갑니다. 그 순간부터 "같은 키는 같은 파티션"이라는 약속이 과거 데이터에 대해 깨지고, 태그별 상태를 들고 처리하던 하류 프로그램의 계산이 틀어집니다. 줄이는 것은 아예 불가능합니다. 그래서 파티션 수는 처음에 여유 있게 잡습니다.

Kafka 에 들어가는 '한 줄'의 생김새

Kafka 의 메시지 한 건(정식 명칭은 레코드, record)은 네 부분으로 되어 있습니다.

부분	내용	누가 정하나
키(key)	파티션을 고르는 데 쓰는 값. 없어도 됨	생산자
값(value)	실제 내용. Kafka 입장에서는 그냥 바이트 덩어리	생산자
타임스탬프	생성 시각(또는 브로커 도착 시각)	생산자 또는 브로커
헤더(header)	이름-값 쌍으로 붙이는 부가 정보. 선택	생산자

여기서 놓치면 안 되는 사실은 Kafka 가 값의 내용을 전혀 해석하지 않는다는 점입니다. JSON 이든 사진이든 압축 파일이든 Kafka 는 바이트 배열로만 취급합니다. 그래서 "값을 어떤 형식으로 담을 것인가"는 Kafka 가 정해 주지 않고

시스템을 만드는 쪽이 약속으로 정해야 합니다. 이 약속을 데이터 계약 (data contract)이라고 합니다. 계약이 없으면 생산자마다 필드 이름을 제멋대로 쓰게 되고, 소비자는 들어오는 메시지 종류마다 다른 해석 코드를 두어야 합니다.

초보가 헛갈리는 지점. 레코드의 키와, 값 안에 들어 있는 필드는 다른 것입니다. AR-100 처럼 키가 태그명이고 값 JSON 안에도 tag 필드가 있으면 같은 정보가 두 군데 있는 셈인데, 쓰임이 다릅니다. 키 는 파티션 배정에만 쓰이고 대개 소비자가 다시 보지 않습니다. 값 안의 필드는 소비자가 실제로 꺼내 쓰는 내용입니다.

컨슈머와 컨슈머 그룹

컨슈머(consumer, 소비자)는 토픽을 읽는 프로그램입니다. 그런데 Kafka 에서 읽기의 단위는 컨슈머 하나가 아니라 컨슈머 그룹(consumer group)입니다. 같은 group.id 문자열을 설정한 컨슈머들은 자동으로 한 그룹으로 묶입니다.

규칙은 두 줄로 요약됩니다.

- 그룹 안에서는 나눠 갖습니다. 한 파티션은 그룹 내에서 오직 한 컨슈머에게만 배정됩니다. 파티션 6 개에 컨슈머 3 명이면 각자 2 개씩 맡습니다. 즉 그룹 내부는 분업입니다. 같은 메시지를 그룹 내 두 명이 중복해서 처리하는 일은 없습니다.

- 그룹이 다르면 전부 각자 읽습니다. 그룹 A와 그룹 B는 같은 토픽의 같은 메시지를 각각 전부 읽습니다. 서로의 진행 상황을 모르고, 서로에게 영향도 주지 않습니다. 그룹 B가 하루 종일 멈춰 있어도 그룹 A는 아무 일 없이 흘러갑니다.

읽은 위치는 그룹별로 따로 저장됩니다. 저장되는 곳은 놀랍게도 Kafka 자신입니다. `__consumer_offsets` 라는 내부 토픽에 "이 그룹은 이 토픽의 이 파티션을 오프셋 몇까지 읽었다"가 기록됩니다. Kafka 는 자기 자신의 관리 정보조차 로그로 남깁니다.

컨슈머가 죽거나 새로 들어오면 파티션 배정을 다시 짜는데, 이것을 리밸런스(rebalance, 재배정)라고 합니다. 리밸런스가 도는 동안에는 잠깐 소비가 멈춥니다. 컨슈머가 자주 죽었다 살아나는 시스템에서 처리량이 들쭉날쭉한 원인이 대개 이것입니다.

초보가 헛갈리는 지점. "같은 데이터를 여러 곳에 보내려면 토픽을 여러 개 만들어야 하나요?" — 아닙니다. 토픽은 하나 두고 컨슈머 그룹을 여러 개 두면 됩니다. 이것이 앞의 $N \times M$ 문제가 사라지는 지점입니다. 생산자는 소비자가 몇 명인지 알 필요가 없고, 소비자가 새로 붙어도 생산자 설정을 건드릴 일이 없습니다.

모든 컨슈머가 `__consumer_offsets` 를 쓰는 것은 아닙니다. Flink 처럼 자기 상태를 스스로 관리하는 처리 엔진은 읽은 위치를 자기 체크포인트에 저장하고, `__consumer_offsets` 에는 참고용으로 가끔만 적습니다. 이 차이가 실습 화면에 그대로 나타나는데, AR-100 절에서 다시 다룹니다.

컨슈머 랙(lag), 그리고 왜 이것이 핵심 지표인가

랙(lag, 밀린 정도)은 한 컨슈머 그룹이 얼마나 뒤쳐졌는지를 건수로 나타낸 값입니다. 뽀렘 한 줄입니다.

랙 = 소비자가 읽을 수 있는 마지막 지점 - 그 그룹이 커밋한 오프셋 "읽을 수 있는 마지막 지점"을 정확히 말하면 하이 워터마크(high watermark, 안전 수위선)입니다. 이것은 모든 정상 사본이 받아 간 지점이며, 그보다 뒤에 있는 메시지는 아직 사본에 다 퍼지지 않아 소비자에게 보이지 않습니다. 파일의 진짜 마지막 오프셋은 LEO(Log End Offset, 로그 끝 오프셋)라고 따로 부르며, 사본이 여럿이면 하이 워터마크보다 앞서 있을 수 있습니다. 사본이 하나뿐인 구성에서는 두 값이 같습니다(AR-100 이 그렇습니다). 복제 이야기는 바로 아래 절에서 이어집니다.

랙이 4 면 "아직 안 가져간 메시지가 4 건 남았다"는 뜻입니다. 파티션마다 계산되고, 보통 토픽·그룹 단위로 합산해서 봅니다.

이 값이 파이프라인 감시의 1 번 지표인 이유는 세 가지입니다.

첫째, 유입 속도와 처리 속도의 차이를 누적한 값이기 때문입니다. 순간 지연시간은 튀었다 말았다 하지만, 랙은 "따라잡고 있는가, 밀리고 있는가"를 통합해서 보여 줍니다. 랙이 200 근처에서 오르내리면 균형 잡힌 상태이고, 랙이 계속 우상향하면 소비자가 영영 못 따라잡고 있다는 뜻입니다.

둘째, 손실의 예고편이기 때문입니다. 랙 자체는 손실이 아닙니다. 데이터는 로그에 그대로 있습니다. 그러나 랙이 계속 커져서 읽어야 할 메시지가 보존 기간을 넘겨 삭제되는 순간 진짜 손실이 발생합니다. 랙 경보는 그 전에 울리는 사이렌입니다.

셋째, 한 숫자로 전 구간을 본다는 점입니다. Flink 가 느려졌든, InfluxDB 디스크가 찼든, 네트워크가 막혔든, 결과는 모두 "그 그룹의 랙 증가"로 나타납니다. 그래서 어느 계층이 병목인지 그룹 이름만으로 즉시 좁혀집니다.

PDF 도 이 지표를 명시적으로 지목합니다. 인프라 감시 계층인 Prometheus 가 감시해야 할 대상으로 "Kafka 클러스터의 토픽별 컨슈머 랙(Consumer Lag)"을 Flink 체크포인트·호스트 리소스와 나란히 열거합니다. 여기서

체크포인트(checkpoint)란, Flink 같은 스트림 처리 엔진이 계산 도중의 상태와 어디까지 읽었는지를 통째로 저장해 두는 안전 지점입니다. 프로그램이 죽어도 마지막 체크포인트로 되돌아가 거기서부터 다시 하면 되므로, 체크포인트가 자주 실패한다는 것은 "죽으면 복구할 수 없는 상태"라는 뜻입니다.

주의 — 랙 그래프를 잘못 읽는 세 가지

- ① 랙이 0 인데 건강하지 않은 경우. 센서가 아예 들어오지 않으면 처리할 것도 없으므로 랙은 0 입니다. 랙만 보면 최악의 상황이 가장 건강해 보입니다. 그래서 "유입이 멈췄는가"를 재는 별도 지표가 반드시 함께 있어야 합니다.
- ② 랙이 갑자기 0 으로 떨어진 경우. 처리량이 좋아진 게 아니라, 누군가 오프셋을 최신으로 리셋해서 밀린 데이터를 통째로 건너뛴 것일 수 있습니다. 랙은 반드시 "처리 건수"와 함께 봐야 합니다.
- ③ 랙 그래프가 사라진 경우. 이것이 가장 위험합니다. 컨슈머가 완전히 죽으면 커밋이 멈추고 랙은 계속 커집니다. 그런데 그 상태가 오래 지속되면, 브로커가 그룹의 커밋 기록 자체를 보관 기간 (offsets.retention.minutes, 기본 7 일)이 지났다고 판단해 지워 버립니다. 그러면 그 그룹은 목록에서 사라지고, 랙을 수집하는 도구가 내보낼 값 자체가 없어져 그래프가 끊깁니다. 이때 kafka_consumer_group_lag > 5000 같은 규칙은 비교할 값이 없으므로 아예 울리지 않습니다. "경보가 조용하다"가 "정상이다"를 뜻하지 않는 대표적인 경우이며, 그래서 스크랩 대상이 살아 있는지를 보는 별도 경보 (up == 0)가 따로 필요합니다.

되감기(오프셋 리셋)

데이터는 로그에 그대로 있고, 읽은 위치는 따로 기록되어 있습니다. 그렇다면 위치만 뒤로 옮기면 과거를 다시 읽을 수 있습니다. 이것이 되감기, 정확히는 오프셋 리셋(offset reset)입니다. Kafka가 데이터베이스도 큐도 아닌 "장부"이기 때문에 공짜로 얻어지는 능력입니다.

```
kafka-consumer-groups.sh --bootstrap-server kafka:9092 \
  --group telegraf-influx-clean --topic sensor.telemetry.clean \
  --reset-offsets --to-earliest --execute
```

--bootstrap-server가 무엇인지부터 풀겠습니다. 이것은 클러스터에 처음 말을 걸 주소입니다. 클라이언트는 이 주소로 한 번 연결해서 "이 클러스터에 브로커가 어디 어디 있고, 내가 쓰려는 파티션을 담당하는 브로커가 누구인지"를 받아 옵니다. 그 뒤의 실제 읽기·쓰기는 담당 브로커에게 직접 합니다. 그래서 브로커가 열 대여도 이 자리에 열 개를 다 적을 필요가 없습니다. 첫 인사만 받아 줄 한두 개면 됩니다. AR-100은 브로커가 하나라 컨테이너 안에서는 kafka:9092, 노트북에서 붙을 때는 localhost:27092입니다.

기준은 골라서 지정합니다. --to-earliest (남아 있는 가장 오래된 것부터), --to-latest (끝으로 건너뛰기), --to-datetime (특정 시각부터 — 브로커에게 "그 시각 이후 첫 메시지가 몇 번이나"를 물어서 정합니다), --shift-by -5000 (5000 건 뒤로).

이 기능의 실전 가치는 큼니다. 이상탐지 모델을 새로 학습했을 때 지난 하루치를 그대로 다시 흘려서 새 모델이 무엇을 잡아내는지 볼 수 있고, 적재 프로그램의 버그를 고친 뒤 잘못 저장된 구간만 다시 채울 수 있습니다. 소비자 쪽 사고가 복구 가능한 사고가 됩니다.

초보가 반드시 헛갈리는 지점 — auto.offset.reset. 설정 파일에 자주 보이는 이 항목 (earliest / latest / none)은 "매번 어디서부터 읽을지"가 아닙니다. 읽기 시작할 위치를 알 수 없을 때에만 쓰이는 비상용 기준값이며, 그런 경우는 두 가지뿐입니다.

- 그 그룹에 커밋된 오프셋이 하나도 없을 때 — 처음 뜨는 그룹, 또는 group.id를 새 이름으로 바꾼 경우.
- 커밋해 둔 오프셋이 이미 사라진 구간을 가리킬 때 — 랙이 커진 채로 방치되어, 읽으려던 자리가 보존 기간을 넘겨 삭제된 경우입니다. 이때 컨슈머는 "그런 오프셋은 없다"는 응답을 받고 이 설정값에 따라 움직입니다. earliest면 남아 있는 가장 오래된 지점으로 뛰어 밀린 것을 최대한 따라잡지만 삭제된 구간은 영영 못 읽고, latest면 지금 이 순간으로 뛰어 밀린 것을 전부 포기하며(랙이 순식간에 0이 됩니다), none이면 오류를 내고 멈춥니다. 앞 절에서 "랙이 커져 보존 기간을 넘기면 손실"이라고 한 그 순간, 컨슈머가 어디로 튈지를 정하는 것이 바로 이 값입니다.

한 번이라도 정상 커밋한 그룹은 이 설정을 바꿔도 아무 일이 일어나지 않습니다. 정말로 되감으려면 위의 --reset-offsets 명령을 써야 하고, 그 명령은 그룹의 컨슈머가 모두 멈춘 상태에서만 실행됩니다.

또 하나. 되감아서 다시 읽으면 하류에 같은 데이터가 두 번 들어갑니다. 그래서 되감기를 쓰려면 소비자가 멍등(idempotent, 같은 것을 두 번 처리해도 결과가 같은 성질)하게 만들어져 있어야 합니다.

리텐션(보존 정책)

지우지 않는다고 했지만 디스크는 무한하지 않습니다. Kafka 는 소비 여부와 무관하게 나이와 크기로 만 지웁니다. 이것이 리텐션(retention, 보존)입니다.

- retention.ms — 이 시간이 지난 데이터부터 삭제 대상. 예: 86400000 은 24 시간.
- retention.bytes — 파티션 하나가 이 용량을 넘으면 오래된 것부터 삭제.

삭제는 메시지 한 건씩이 아니라 세그먼트(segment, 토막) 단위로 이루어집니다. 파티션의 로그는 하나의 거대한 파일이 아니라 일정 크기·시간마다 잘린 여러 개의 파일로 저장되며, 삭제도 그 파일 단위 로 통째로 일어납니다. 그래서 "정확히 24 시간 지나면 없어진다"가 아니라 "24 시간 지난 세그먼트가 닫히면 지워진다" 쪽에 가깝습니다. 리텐션은 최소 보관 보장으로 읽는 것이 안전합니다.

정책은 두 가지가 있습니다.

- 삭제(delete) — 위에서 설명한 기본 동작. 시계열 이벤트용.
- 압축(compact, 로그 압축) — 키별로 마지막 값만 남기고 중간 이력을 정리합니다. "지금 상태가 뭐냐"를 담은 용도(설비 설정값 스냅샷 등)에 씁니다. 이력이 필요한 센서 계측에는 쓰면 안 됩니다. 이름 이 비슷하지만 앞에서 본 compression.type=lz4 (바이트를 줄이는 압축)와는 완전히 다른 이야기입니다.

리텐션 값을 정하는 기준은 "디스크가 얼마나 남았나"가 아니라 "장애가 났을 때 얼마나 뒤에서부터 복구하고 싶은가"입니다. 24 시간이면 하루 안에 문제를 발견해 되감을 수 있다는 뜻이고, 그보다 늦게 발견하면 그 데이터는 없습니다.

복제(replication)와 acks

Kafka 클러스터는 여러 대의 브로커로 이루어집니다. 파티션은 여러 브로커에 사본을 둡니다. 사본 개 수가 복제 계수(replication factor)입니다. 복제 계수 3 이면 같은 파티션의 내용이 서로 다른 세 대에 각각 있습니다.

사본 중 하나가 리더(leader)이고 나머지가 팔로워(follower)입니다. 읽기와 쓰기는 모두 리더가 처리 하고, 팔로워는 리더를 따라 복사만 합니다. 리더가 죽으면 팔로워 중 하나가 새 리더가 됩니다. 이때 "제대로 따라오고 있던 팔로워"의 명단을 ISR(In-Sync Replicas, 동기화된 복제본)이라고 합니다. 너무 뒤쳐진 팔로워는 ISR 에서 빠지고, 빠진 팔로워는 새 리더가 될 자격을 잃습니다. 오래된 데이터만 가진 사본이 리더가 되면 그 사이의 데이터가 통째로 사라지기 때문입니다.

앞 절의 하이 워터마크가 바로 여기서 정해집니다. ISR 전원이 받아 간 지점까지만 소비자에게 보입니다. 리더만 갖고 있고 팔로워에 아직 안 퍼진 부분은, 리더가 지금 죽으면 없어질 수도 있으므로 보여 주지 않습니다.

생산자는 "어디까지 저장됐다고 확인받고 다음으로 넘어갈지"를 acks(acknowledgements, 수신 확인)로 고릅니다.

acks 값	생산자가 기다리는 것	속도	잃을 수 있는 것
0	아무것도 안 기다림. 보내고 잊음	가장 빠름	브로커가 못 받아도 생산자는 모름
1	리더가 자기 로그에 기록했다는 응답 (대개 페이지 캐시까지이며 디스크 확정은 아님)	중간	팔로워로 복사되기 전에 리더가 죽으면 그 구간 소실
all (= -1)	ISR 명단에 있는 사본 전원이 받았다는 응답	가장 느림	ISR 이 리더 혼자뿐이면 1 과 같다. 복제 계수 3 + min.insync.replicas=2 여야 한 대 고장을 견딜

초보가 반드시 헷갈리는 지점. acks=all 은 그 자체로 무손실을 뜻하지 않습니다. 표의 마지막 행이 말하는 그대로입니다. 복제 계수가 1 이면 ISR 명단에 리더 혼자 있으므로, "전원이 받았다"는 확인은 "리더 한 대가 받았다"와 똑같습니다. 무손실을 말하려면 복제 계수 3 이상과 min.insync.replicas=2 (살아 있는 사본이 최소 2 대여야 쓰기를 받아들임)를 함께 걸어야 합니다.

acks 는 각오의 표현이고, 복제가 실제 안전장치입니다.

KRaft 가 ZooKeeper 를 대체한 것

Kafka 클러스터는 "어떤 토픽이 있는지, 파티션이 몇 개인지, 각 파티션의 리더가 어느 브로커인지, 누가 ISR 인지" 같은 메타데이터(metadata, 관리 정보)를 보관해야 합니다. 그것도 여러 대가 똑같이 알고 있어야 합니다. 어떤 브로커는 리더가 A 라고 알고 다른 브로커는 B 라고 알면 클러스터가 망가지기 때문입니다.

합의(consensus)가 필요한 지점이 여기입니다. 합의란 여러 대의 컴퓨터가, 일부가 죽거나 연락이 끊기는 상황에서도, 같은 값을 같은 순서로 갖도록 맞추는 절차입니다. 사람으로 치면 회의에서 안건을 하나씩 순서대로 의결하고 회의록을 똑같이 나눠 갖는 일과 비슷합니다. 이 절차를 정해 놓은 규칙을 합의 알고리즘(consensus algorithm)이라고 하고, 그중 널리 쓰이는 것이 Raft(래프트)입니다. Raft의 방식은 단순합니다. 참가자 중 한 대를 투표로 지도자로 뽑고, 모든 변경은 지도자를 거쳐 번호가 매겨진 기록으로 전파하며, 과반이 받아 적으면 확정합니다. 과반이라는 이 최소 인원을 쿼럼(quorum, 정족수)이라고 합니다.

Kafka는 오랫동안 이 일을 ZooKeeper(주키퍼)라는 별도 제품에 맡겼습니다. 결과적으로 운영자는 Kafka와 ZooKeeper 두 개의 분산 시스템을 동시에 돌보고, 두 벌의 설정·모니터링·보안을 관리해야 했습니다. 브로커 장애 시 메타데이터를 다시 읽어 오는 데 시간이 걸려 파티션이 수만 개를 넘으면 복구가 느려지는 문제도 있었습니다.

KRaft(Kafka Raft, 카프카 래프트)는 이 외부 의존을 없앤 방식입니다. Kafka가 Raft 합의를 스스로 구현해서, 메타데이터를 `__cluster_metadata`라는 내부 토픽 로그에 기록합니다. 즉 Kafka가 자기 철학("모든 것은 순서 있는 로그다")을 자기 자신의 관리 정보에 적용한 것입니다.

KRaft에서는 브로커가 역할을 가집니다. `process.roles`에 `broker`(데이터 처리), `controller`(메타데이터 합의), 또는 둘 다를 지정합니다. 컨트롤러들이 쿼럼을 이뤄 투표하며, 그 투표자 명단이 `controller.quorum.voters`입니다.

시점을 정확히 적으면 이렇습니다. KRaft는 2021년 Kafka 2.8에서 시험용 미리보기로 처음 나왔고, 3.3에서 상용 사용 가능(production ready) 판정을 받았으며, 3.5에서 ZooKeeper 모드가 비권장(deprecated)이 되었고, 4.0에서 ZooKeeper 모드가 제거되었습니다. 즉 2021년에 바로 바뀐 것이 아니라 몇 해에 걸쳐 넘어갔고, 그 사이에는 ZooKeeper 모드가 계속 기본이었습니다. 지금 새로 만드는 시스템에 ZooKeeper를 세울 이유는 없습니다. 설계 보고서의 기술 스택 표가 백본을 그냥 "Apache Kafka"가 아니라 "Apache Kafka (KRaft 클러스터)"로 못 박은 것은 이 흐름을 반영한 것입니다.

브로커(MQTT)와 장부(Kafka)를 왜 둘 다 두는가 가장 자주 나오는 질문입니다. "MQTT 브로커도 발행·구독을 하고 메시지를 나눠 주는데, Kafka를 또 두는 건 중복 아닌가요?" 중복이 아닙니다. 두 물건은 푸는 문제가 다릅니다. 3장에서 본 MQTT의 QoS(Quality of Service, 전달 보장 등급 — 0은 한 번 던지고 끝, 1은 최소 한 번, 2는 정확히 한 번)와 유언 메시지(Last Will and Testament — 클라이언트가 예고 없이 끊기면 브로커가 대신 발행해 주는 "나 죽었다" 알림)는 불안정한 회선에 매달린 기기를 다루기 위한 장치입니다. Kafka에는 그런 장치가 없고, 필요하지도 않습니다. Kafka의 상대는 센서가 아니라 서버이기 때문입니다.

비교 항목	MQTT 브로커 (EMQX)	Kafka
본질	지금 연결된 구독자에게 배달하는 방송	순서대로 쌓아 두는 장부
전달 방식	밀어 줌(push) — 브로커가 구독자 연결로 밀어 넣음	가져감(pull) — 컨슈머가 오프셋을 지정해 요청(폴링)
속도의 주도권	보내는 쪽. 받는 쪽이 느리면 브로커에 부담	받는 쪽. 느리면 덜 요청할 뿐 브로커는 무영향
통신 규약	MQTT (v3.1.1 / v5.0)	Kafka 자체 바이너리 TCP 프로토콜 — MQTT를 말하지 못함
연결 수	수만~수십만 개의 가느다란 연결	수십 개의 굵고 오래 유지되는 연결
보관	원칙적으로 배달하면 끝. 세션 큐는 특정 클라이언트용 임시 보관	디스크에 순서대로 보관. 소비와 무관하게 리텐션까지 유지
과거 재생	개념 자체가 없음	오프셋만 옮기면 언제든 다시 읽음
느린 소비자	세션 큐가 차면 유실	랙만 늘어날 뿐 데이터는 로그에 남음
강점	불안정한 무선·저대역 회선, QoS, 유언 메시지, 양방향 제어 경로	대용량 보존, 다수 소비자 독립 재생, 파티션 단위 수평 확장
말는 문제	현장의 연결·프로토콜 문제	전사의 보존·분배 문제

PDF가 말한 "전사 차원 완충 큐"와 "물리적 의존성 분리"

설계 보고서는 Tier 2를 설명하며 이렇게 씁니다. "EMQX의 데이터 통합 브리지를 통해 수신된 센서 텔레메트리는 지체 없이 Apache Kafka 클러스터의 `sensor.telemetry.raw` 토픽으로 적재된다. 이로써 전사 차원의 완충 큐가 확보되고 하위 시스템과의 물리적 의존성이 완전히 분리된다."

텔레메트리(telemetry)는 멀리 떨어진 장비에서 측정된 값을 중앙으로 자동 전송하는 것, 또는 그렇게 전송된 측정값 자체를 가리킵니다. "원격 계측 데이터"로 읽으면 됩니다. 이 교본에서 "센서 텔레메트리"는 온도·압력·유량 같은 계측값의 흐름을 뜻합니다.

① 전사 차원 완충 큐(buffer queue). 완충 큐는 들어오는 속도와 나가는 속도가 달라도 되게 만드는 저수지입니다. 상류(센서)는 초당 일정하게 밀어 넣고, 하류(저장·분석)는 자기 사정대로 퍼 갑니다. InfluxDB 를 10 분 재시작해도 센서 수집은 1 초도 멈추지 않고, InfluxDB 가 살아나면 자기 오프셋부터 읽어 그 사이 밀린 것을 따라잡습니다. 이렇게 쌓인 미처리 물량을 백로그(backlog, 밀린 일감)라고 합니다. "전사 차원"이라는 말은 이 저수지가 한 부서용이 아니라 회사 전체의 공용 저수지라는 뜻입니다. 나중에 품질팀·설비보전팀·연구소가 불을 때 현장을 다시 건드릴 필요가 없습니다.

② 하위 시스템과의 물리적 의존성 분리. 생산자가 소비자의 주소·개수·가용성을 아예 모르게 만든다는 뜻입니다. Telegraf 브리지는 Flink 가 몇 대인지, 살아 있는지, IP 가 뭔지 모릅니다. 그저 sensor.telemetry.raw 에 쓸 뿐입니다. 그래서 Flink 전체가 죽어도 수집은 계속되고, 반대로 현장 네트워크가 끊겨도 분석 계층은 이미 들어온 데이터로 계속 돕니다. 설계 보고서의 다른 문장이 이 효과를 더 강하게 표현합니다. "전사 단위의 데이터 내결함성 보장과 고가용성 버퍼링은 Kafka 가 전담함으로써 물리적 계측 계층의 불안정성이 상위 분석 및 스토리지 시스템으로 전파되는 현상을 원천적으로 차단한다."

왜 MQTT 만으로는 이게 안 되는가. MQTT 의 세션 큐는 "이 클라이언트가 잠깐 끊겼을 때 쓸 임시 사물함"입니다. 클라이언트 단위이고, 용량 상한이 곧 유실 지점이며, 무엇보다 새 소비자를 붙여 과거를 다시 읽는다는 개념이 없습니다. 완충과 재생은 장부의 성질이지 방송의 성질이 아닙니다.

AR-100 에서는

AR-100 · Kafka 구성. Kafka 3.9.0 을 KRaft 단일 노드로 띄웁니다. docker-compose.yml 의 kafka 서비스에 KAFKA_PROCESS_ROLES: broker,controller 가 설정되어 한 프로세스가 데이터 처리 와 메타데이터 합의를 겸하고, KAFKA_CONTROLLER_QUORUM_VOTERS: 1@kafka:9093 로 자기 자신만이 투표자입니다. 클러스터 식별자는 CLUSTER_ID: iiot-scada-kraft-cluster 입니다. ZooKeeper 컨테이너는 스택에 존재하지 않습니다. 실수로 토픽이 자동 생성되지 않도록 KAFKA_AUTO_CREATE_TOPICS_ENABLE: "false" 로 잠가 두었습니다(오타 난 토픽 이름이 새 토픽으로 조용히 만들어지는 사고를 막습니다).

리스너와 광고 주소

위 설정에 나오는 다음 세 줄이 초보가 가장 자주 막히는 자리입니다.

```
KAFKA_LISTENERS: PLAINTEXT://:9092,CONTROLLER://:9093,EXTERNAL://:19092
KAFKA_ADVERTISED_LISTENERS: PLAINTEXT://kafka:9092,EXTERNAL://localhost:27092
KAFKA_LISTENER_SECURITY_PROTOCOL_MAP: PLAINTEXT:PLAINTEXT,CONTROLLER:PLAINTEXT,EXTERNAL:PLAINTEXT
```

리스너(listener)는 브로커가 접속을 받는 창구입니다. 한 브로커가 창구를 여러 개 열 수 있고, 각 창구는 <이름>://<주소>:<포트> 형태로 적습니다. 여기서 PLAINTEXT · CONTROLLER · EXTERNAL 은 Kafka 가 미리 정한 낱말이 아니라 관리자가 창구에 붙인 별명입니다. 이 점을 모르면 "PLAINTEXT 라고 적혀 있으니 암호화가 안 되는 거구나" 같은 잘못된 짐작을 하게 됩니다. 실제 보안 방식(평문 / TLS 암호화 / 인증)은 세 번째 줄의 ..._SECURITY_PROTOCOL_MAP 이 창구 별명마다 따로 지정합니다. AR-100 은 교육용이라 세 창구 모두 평문입니다.

AR-100 의 세 창구는 이렇습니다.

- PLAINTEXT://:9092 — 도커 네트워크 안의 다른 컨테이너(Telegraf, Flink)가 붙는 창구.
- CONTROLLER://:9093 — 컨트롤러끼리 메타데이터 합의에 쓰는 전용 창구. KAFKA_CONTROLLER_LISTENER_NAMES: CONTROLLER 로 "이 별명이 컨트롤러용"임을 알려 줍니다.
- EXTERNAL://:19092 — 컨테이너 바깥(노트북)에서 들어오는 접속. .env 의 PORT_KAFKA=27092 가 호스트의 27092 포트를 여기로 연결합니다.

광고 주소(advertised listeners)는 성격이 다릅니다. 앞에서 --bootstrap-server 로 첫 인사를 하면 브로커가 "실제 통신은 이 주소로 해라"고 답한다고 했는데, 그때 알려 줄 주소가 바로 이 값입니다. 컨테이너 안에서 붙은 클라이언트에게는 kafka:9092 를, 밖에서 붙은 클라이언트에게는 localhost:27092 를 알려 줍니다. 이 값을 잘못 적으면 접속은 되는데 그 다음부터 통신이 안 되는 이 상한 증상이 납니다. 클라이언트가 브로커가 알려 준 주소로 다시 연결하려다 실패하기 때문입니다.

AR-100 · 토픽 네 개. `kafka/create-topics.sh` 가 기동 시 한 번 실행되어 토픽을 만듭니다 (이미 있으면 건너뛰는 멍등 스크립트입니다).

토픽	파티션	보존	쓰는 쪽(생산자)	읽는 쪽(컨슈머 그룹)
sensor.telemetry.raw	6	24 시간	telegraf-bridge (MQTT→Kafka 브리지)	flink-tier1 (Flink SQL) · flink-tier2-onnx (Flink ONNX 잡) · telegraf-influx-raw
sensor.telemetry.clean	6	24 시간	Flink ONNX 잡의 Interpolator	telegraf-influx-clean
sensor.anomaly.score	3	24 시간	Flink ONNX 잡의 OnnxScorer	telegraf-influx-score
sensor.alerts	3	7 일	Flink SQL(Tier-1) + Flink ONNX(Tier-2)	telegraf-influx-alerts · telegraf-alert-republish

네 토픽 모두 `--replication-factor 1`, `compression.type=lz4` 입니다. 알람만 보존이 7 일 (604800000 ms)인 이유는 분명합니다. 계측값은 InfluxDB 에 영구 적재되지만 알람은 사람이 나중에 꺼내 보는 물건이라 장부 쪽에 오래 남겨 둡니다.

AR-100 · 키와 파티션. 파티션 키는 태그명입니다. Telegraf 브리지 설정 `telegraf/bridge-edgex.conf` 에 `routing_tag = "tag"` 한 줄로 지정되어 있습니다. 그래서 TT-101 의 계측값들은 언제나 같은 파티션에 순서대로 쌓이고, Flink 쪽에서 `.keyBy(r -> r.tag)` 로 태그별 상태(직전 값, 결측 구간)를 유지하는 것과 배정 기준이 일치합니다.

파티션을 6 으로 잡은 것은 태그가 12 종이므로 "한 파티션당 둘 정도면 넉넉하다"는 규모 감각에서 나온 값입니다. 태그가 파티션마다 정확히 2 개씩 들어간다는 뜻이 아닙니다. 앞에서 본 대로 해시는 고르게 나눠 주지 않으므로, 실제로는 태그 3 개가 몰린 파티션도 있고 하나도 없는 파티션도 있습니다. 균등 분배가 아니라 순서 보장과 병렬도 상한이 이 값을 고른 이유입니다.

생산자 쪽은 `required_acks = -1` (= `acks=all`), `max_retry = 10` (실패 시 최대 10 회 재시도), `compression_codec = 3` (lz4) 로 설정되어 있습니다.

계약과 메시지 키

Kafka 는 JSON 내용을 해석하지 않습니다. 값에는 `ts·site·device·tag·value·quality` 가 실리고, Telegraf 의 `routing_tag="tag"`가 별도의 Kafka 메시지 키를 만듭니다. 같은 태그의 순서는 해당 파티션 안에서 지켜집니다. JSON 의 tag 필드가 존재하는 것과 Kafka 키로 지정하는 것은 별개 설정입니다.

원본(raw)의 범위도 구분합니다. 이 시스템의 raw 는 센서 전기신호나 필터 이전 원시값이 아니라, 공통 계약으로 변환하고 비계측 리소스와 센티널을 제외한 뒤의 유효 관측입니다. 원문 130 쪽의 측정에서는 raw 2,266 건 중 dropout 대상 TT-101 이 176 건, 대조 LT-102 가 190 건으로 기록되었습니다. 센티널 제거로 만든 공백이며 이후 보간은 raw 를 덮어쓰지 않습니다.

AR-100 · 같은 데이터를 세 그룹이 각자 읽습니다. `sensor.telemetry.raw` 하나를 세 그룹 이 동시에, 서로 모른 채, 각자 속도로 소비합니다.

· `flink-tier1` — `flink/sql/01_sources.sql` 의 `'properties.group.id' = 'flink-tier1'` . 규격 초과 임계치 검사, Z-Score(지금 값이 최근 평균에서 표준편차의 몇 배만큼 떨어져 있는지를 재는 통계 값. 3 을 넘으면 "평소와 다르다"고 본다), CEP(Complex Event Processing, 복합 이벤트 처리 — "A 가 일어나고 10 초 안에 B 가 일어나면" 같이 여러 사건의 순서와 시간 간격을 조건으로 거는 방식) 를 돌립니다.

· `flink-tier2-onnx` — `flink/onnx-job/job.properties` 의 `kafka.group.id` . 결측 보간과 오토인코더 추론을 합니다. 오토인코더(autoencoder)는 입력을 잘게 압축했다가 다시 펼쳐 원래대로 복원하도록 학습시킨 신경망입니다. 평소 보던 패턴은 잘 복원하지만 처음 보는 패턴은 복원에 실패하므로, 그 복원 오차의 크기가 곧 "얼마나 낯선가"의 척도가 됩니다. ONNX(Open Neural Network Exchange)는 학습해 둔 신경망을 특정 프로그래밍 언어나 학습 도구에 매이지 않는 공통 파일 형식으로 저장하는 규격이라, 파이썬으로 학습한 모델을 자바로 돌아가는 Flink 안에서 그대로 실행할 수 있습니다.

· `telegraf-influx-raw` — 보간 전 원본을 InfluxDB 의 `process_raw` 계측으로 적재합니다.

어느 하나가 멈춰도 나머지 둘은 영향을 받지 않습니다. 이것이 앞에서 말한 "여러 소비자가 각자 속도로"의 실물입니다.

AR-100 · raw 와 clean 을 토픽으로 갈라 놓았습니다. 설계 보고서는 "원천 데이터 레벨에서 인위적으로 계산된 가공치를 덮어쓰게 되면 실제 물리 계측치와 추정치 간의 경계가 모호해져 향후 사고 발생 시 감사 추적(Audit Trail)의 법적·기술적 신뢰성을 훼손할 수 있다"고 경고합니다.

이 원칙을 규칙이 아니라 토폴로지로 강제한 것이 토픽 분리입니다. 토폴로지(topology)는 구성 요소들이 어떻게 연결되어 있는지의 짜임새를 뜻합니다. "보간값을 원본에 쓰지 말자"를 사람이 지키는 규칙으로 두면 언젠가 누군가 어깁니다. 아예 원본 토픽에 쓸 수 있는 프로그램을 브리지 하나로 못 박고, 보간 결과는 다른 토픽으로만 나가게 배선해 두면, 어기고 싶어도 어길 경로가 없습니다.

raw 는 무손실 원본이며 위에서 본 방식으로 결측 구간이 공백으로 남습니다. 보간값은 clean 에만 들어가고 quality 필드가 그것이 추정치임을 밝힙니다. 코드에 실제로 존재하는 값은 셋입니다.

quality 값	뜻	붙이는 곳
GOOD	실제로 센서가 측정한 값	Reading.java:14 의 기본 값 / 브리지 starlark 필터
INTERPOLATED_LINEAR	앞뒤 두 실측값을 잇는 직선 위의 값으로 빈칸을 채운 추정치	Interpolator.java:75
INTERPOLATED_LOCF	LOCF(Last Observation Carried Forward, 직전 관측치 유지) — 빈칸을 마지막으로 본 값 그대로 채운 추정치. 공백이 너무 길어 (interpolation.max-gap-ms = 20000 초과) 직선 보간이 무리일 때 대신 쓴다	Interpolator.java:75

실측 결과 InfluxDB 에는 GOOD 15,858 건(99.74%), INTERPOLATED_LINEAR 42 건(0.26%)이 구분된 채로 저장되었습니다. Kafka 는 덮어쓰기가 없는 구조이므로, 원본 토픽에 남은 것은 그대로 남습니다.

주의 — 문서에만 있고 코드에는 없는 값. 이 프로젝트의 ARCHITECTURE.md 177 번째 줄은 quality 의 값으로 네 가지를 적어 두었고, 그중 STALE (직역하면 "김빠진, 오래된" — 값이 갱신되지 않고 낡았다는 표시)은 어느 코드에도 구현되어 있지 않습니다. 프로젝트 전체를 검색하면 그 한 줄에만 나옵니다. 실측표(GOOD 15,858 / INTERPOLATED_LINEAR 42)에도 당연히 없습니다. 이것을 굳이 적는 이유는, 설계 문서와 실제 코드가 어긋나 있는 것을 발견했을 때 코드를 정보으로 삼아야 한다는 점이 이 장의 실무 교훈이기 때문입니다. 문서에 적힌 값이 동작하리라 믿고 하류에서 STALE 을 걸러 내는 코드를 짜면, 그 조건은 영원히 참이 되지 않습니다.

AR-100 · Flink 는 읽은 위치를 다른 곳에 둡니다. 앞에서 "읽은 위치는 __consumer_offsets 에 저장된다"고 했는데, Flink 는 예외입니다. Flink 는 어디까지 읽었는지를 자기 체크포인트 안에 계산 상태와 함께 통째로 저장합니다. 그래야 "상태와 읽은 위치가 한 시점에 서 맞아떨어지는" 복구가 가능하기 때문입니다. __consumer_offsets 에는 사람이 랙을 볼 수 있도록 참고용으로만, 그것도 체크포인트가 완료될 때에만 적습니다.

AR-100 의 체크포인트 주기는 flink/sql/01_sources.sql 의 SET 'execution.checkpointing.interval' = '10s'; 와 flink/conf/config.yaml 의 interval: 10s 로 10 초입니다.

그래서 실습 화면에서 이런 모습을 보게 됩니다. flink-tier1 과 flink-tier2-onnx 그룹의 랙 그래프는 매끄럽게 흐르지 않고 톱니 또는 계단 모양으로 됩니다. 10 초 동안 커밋이 없으니 랙이 계속 올라가다가, 체크포인트가 끝나는 순간 커밋이 한꺼번에 반영되어 똑 떨어지고, 다시 올라갑니다. 12 건/초 유입이면 톱니의 높이는 대략 120 건 안팎입니다. 이것은 고장이 아니라 정상입니다. 반면 telegraf-* 그룹들은 읽는 대로 자주 커밋하므로 그래프가 훨씬 평평합니다. 같은 화면 안에서 두 모양이 나란히 나타나는 것이 정상 상태입니다.

AR-100 · 랙 감시. Kafka 자체는 랙을 Prometheus 형식으로 내보내지 않으므로 kafka-exporter 컨테이너가 :9308 에서 대신 노출하고, prometheus/prometheus.yml 의 job_name: kafka 가 이를 수집합니다(주석도 "Kafka 컨슈머 랙 — 파이프라인 지연의 1 차 지표"라고 적혀 있습니다). 경보 규칙은 prometheus/rules.yml 에 있습니다.

```
- alert: KafkaConsumerLagHigh
  expr: sum by (consumergroup, topic) (kafka_consumergroup_lag) > 5000
  for: 2m
- alert: TelemetryIngestStalled
  expr: sum(rate(kafka_topic_partition_current_offset{topic="sensor.telemetry.raw"}[2m])) == 0
  for: 2m
```

이 두 줄은 PromQL(Prometheus Query Language, 프로메테우스 질의 언어)로 쓰여 있습니다. 읽는 법은 다음과 같습니다.

조각	읽는 법
kafka_consumergroup_lag	지표 이름. 그룹·토픽·파티션 조합마다 값이 하나씩 나온다
sum by (consumergroup, topic) (...)	괄호 안의 값들을 그룹과 토픽이 같은 것끼리 더해라. 파티션별로 흩어진 값이 그룹·토픽 단위 숫자 하나로 합쳐진다
> 5000	그 합이 5000을 넘는 조합만 남긴다. 남는 게 있으면 경보 후보
for: 2m	조건이 2분 내내 참이어야 실제로 올린다. 순간적으로 튀는 값은 무시
{topic="sensor.telemetry.raw"}	중괄호는 라벨 걸러내기. 토픽 이름이 이것인 시계열만 고른다
rate(...[2m])	최근 2분 구간에서 그 값이 초당 얼마나 늘었는지. 오프셋은 계속 커지기만 하므로, 그 증가 속도가 곧 유입 속도다
== 0	초당 증가가 0, 즉 아무것도 들어오지 않고 있다

앞의 규칙은 소비가 못 따라가는 상태를, 뒤의 규칙은 생산이 멈춘 상태를 잡습니다. 랙만 보면 "센서가 아예 안 들어와서 랙이 0"인 최악의 경우를 놓치기 때문에 두 개를 함께 걸었습니다. 같은 파일의 PipelineServiceDown (up == 0)은 세 번째 축으로, 앞의 주의 상자에서 말한 "랙 그래프가 통째로 사라진" 상황을 잡아 줍니다. 이 경보들은 모두 파이프라인 자체의 장애만 다루며, 공정 이상 (센서 값)은 Flink → sensor.alerts 경로가 담당합니다 — Prometheus와 InfluxDB의 역할 분리는 7장에서 다룹니다.

AR-100 · 오프셋으로 처리량을 잽니다. 실측치는 다음과 같습니다.

지표	실측	측정 방법
센서 유입	12 건/초 (252 건 / 20 초)	오프셋 차분
raw → clean	528 → 528 (40 초), 손실 0	양쪽 오프셋 차분
anomaly.score	1.1 건/초 (44 건 / 40 초)	오프셋 차분
정상 운전 중 알람	0 건 / 90 초	sensor.alerts 오프셋 차분

측정 명령은 컨슈머가 아니라 kafka-get-offsets.sh 입니다. 두 시점의 오프셋을 읽어 그 차이를 구하는, 단순한 뽀샘입니다.

```
off(){ docker exec kafka /opt/kafka/bin/kafka-get-offsets.sh \
  --bootstrap-server localhost:9092 --topic "$1" | awk -F: '{s+=$3} END{print s+0}'; }
a=$(off sensor.telemetry.raw); sleep 20; b=$(off sensor.telemetry.raw)
echo "$((b-a))건/20초"
```

이렇게 한 데에는 사연이 있습니다. 처음에는 kafka-console-consumer.sh (터미널에 메시지를 찍어 주는 시험용 컨슈머)로 건수를 켜는데 기대치의 약 4.7 배(12 건/초여야 하는데 56 건/초)가 나와 "파이프라인이 중복 발행한다"고 오판했습니다.

원인은 두 가지가 겹친 것이었습니다. 첫째, 콘솔 컨슈머는 JVM(Java Virtual Machine, 자바 가상 머신 — 자바 프로그램이 실제로 그 위에서 돌아가는 실행 환경) 기동에만 10여 초가 걸립니다. 둘째, 그 10여 초 동안 토픽에는 계속 메시지가 쌓이고, 컨슈머는 뜨자마자 그 백로그(밀려 있던 미처리 물량)를 한꺼번에 읽어 들입니다. 그러니 "20초 동안 몇 건 읽었나"를 세면 실제 유입 속도보다 훨씬 큰 숫자가 나옵니다.

컨슈머는 자기 자신이 관측 대상에 영향을 줍니다. 반면 오프셋은 브로커가 이미 들고 있는 숫자이므로, 읽어도 시스템을 건드리지 않고 컨슈머 그룹을 만들지도 않습니다. 참고로 --timeout-ms 는 "그 시간 동안 메시지가 없으면 종료"라는 뜻이라, 계속 생산되는 토픽에서는 영원히 끝나지 않습니다(실제로 12 초를 줬는데 87 초가 지나도 안 끝났습니다).

입력 중복 제거와 재시작 후 중복은 다른 문제

Interpolator는 태그별 마지막 통과 시각을 기억하고 in.ts <= lastTs 이면 버립니다. 따라서 동일 관측의 재전송뿐 아니라, 12:00:03 뒤에 처음 도착한 12:00:02 같은 늦은 정상 관측도 버립니다. 최근 (tag, ts) 집합을 보관하는 대안은 상태 크기를 더 요구합니다. 이 구현은 태그당 마지막 시각 하나를 택했습니다. [원문 36, 99, 136 쪽]

원문 136 쪽의 Java Kafka 싱크는 AT_LEAST_ONCE 입니다. 장애 후 체크포인트에서 다시 읽으면 이미 쓴 결과가 다시 나갈 수 있고, 싱크보다 앞에 있는 입력 중복 제거가 뒤쪽에서 생기는 재출력을 막지는 못합니다. Flink 상태의 EXACTLY_ONCE와 최종 외부 출력의 중복 없음은 같은 보장이 아닙니다. [원문 136, 150, 151 쪽]

AR-100 · 되감기를 일부를 쓰지 않습니다. Flink SQL 쪽은 'scan.startup.mode' = 'latest-offset' , Java 잡 쪽은 OffsetsInitializer.latest() , Telegraf 컨슈머들은 offset = "newest" 로 모두 지금부터 읽도록 되어 있습니다. 실습에서는 "방금 주입한 고장이 화면에 나타나는가"를 보는 것이 목적이라, 기동할 때마다 과거를 다시 훑으면 오히려 방해가 되기 때문입니다. 과거를 다시 훑려 보고 싶을 때만 컨슈머 그룹을 멈추고 --reset-offsets 를 실행합니다.

되감기와 재시도를 안전하게 쓰려면 소비자가 먹등해야 한다고 했는데, 이 시스템에서 그 원칙이 코드로 나타난 자리가 Interpolator 의 첫 네 줄입니다.

```
// — 먹등 처리: MQTT QoS1 재전송으로 인한 중복을 (tag, ts) 기준으로 제거 —
// (PDF p.4 - QoS2 대신 QoS1 + 스트림 계층 먹등성)
Long lastTs = lastEmittedTsNs.value();
if (lastTs != null && in.ts <= lastTs) {
    return;
}
```

이 코드가 막는 것이 무엇인지 정확히 봐야 합니다. 주석이 밝히듯 대상은 입력단의 중복입니다. MQTT 의 QoS 1 은 "최소한 한 번" 전달이라 확인 응답이 유실되면 같은 메시지가 다시 옵니다. 그러면 같은 (tag, ts) 조합이 두 번 들어오는데, 이 코드는 태그별로 마지막에 내보낸 타임스탬프를 상태에 들고 있다가 그보다 오래되거나 같은 것이 오면 그냥 버립니다. 설계 보고서가 권고한 "중복 수신 문제는 스트림 처리 계층의 먹등성(Idempotency) 로직으로 해소한다"가 이것입니다.

반대로, 이 코드가 막지 못하는 중복도 분명히 있습니다. AnomalyJob.java 의 Kafka 싱크는 DeliveryGuarantee. AT_LEAST_ONCE (최소 한 번)로 설정되어 있어, 잡이 재시작되면 이미 내보낸 메시지가 한 번 더 나갈 수 있습니다. 그런데 잡의 흐름은 raw 소스 → keyBy(tag) → Interpolator → clean/score/alerts 싱크 이므로 Interpolator 는 싱크보다 앞(상류)에 있습니다. 뒤에서 생긴 중복 을 앞에 있는 코드가 걸러 낼 수는 없습니다. 게다가 재시작하면 Interpolator 가 들고 있던 상태도 같은 체크포인트 시점으로 함께 되돌아가므로, 다시 읽은 레코드는 "처음 보는 것"으로 판정되어 그대로 통과합니다. 싱크 재시도로 생기는 중복 제거는 이 예제의 범위 밖이며, 실제로 필요하다면 Kafka 트랜잭션을 쓰는 EXACTLY_ONCE 모드로 올려야 합니다.

주의. 이 구성은 단일 브로커 · 복제 계수 1 입니다. 따라서 required_acks = -1 이라도 확인해 주는 사본은 리더 하나뿐이고, 브로커가 죽으면 그 순간의 데이터는 복구되지 않습니다. 랙 절에서 말한 하이 워터마크와 LEO 가 같은 값인 것도 이 때문입니다. 이것은 "한 대 노트북에서 docker compose up 한 번으로 전 구간을 보여 주는" 교육용 구성의 명시적 한계이며, 실제 공장에 올릴 때는 브로커 3 대 이상 · 복제 계수 3 · min.insync.replicas=2 가 출발점입니다. 프로젝트 문서도고가용성(Kafka 다중 브로커, Flink HA)을 이 예제의 범위 밖으로 명시하고 있습니다.

주의 — 제어 명령은 Kafka 를 타지 않습니다. AR-100 에서 운전원이 펌프를 켜거나 밸브 개도를 바꾸면, 그 명령은 Kafka 를 거치지 않고 FUXA 가 시뮬레이터의 Modbus/TCP 레지스터에 직접 씁니다(EdgeX 경로에서는 core-command 의 REST 호출). 이것은 빠뜨린 것이 아니라 의도한 설계입니다. 설계 보고서는 "센서나 PLC 레지스터에 특정 제어 값을 주입하는 사우스바운드(Southbound) 양방향 제어 경로를 수립하고 유지하는 주체는 Kafka 가 아닌 IoT 미들웨어다"라고 못 박습니다.

사우스바운드는 중앙에서 현장 장비 쪽으로 내려가는 방향을 뜻합니다(그림에서 현장을 아래에 그리는 관례에서 나온 말입니다). 반대로 현장에서 중앙으로 올라오는 방향은 노스바운드 (Northbound)라고 합니다. 장부는 일어난 일을 남기는 곳이지, 명령을 하달하는 곳이 아닙니다. 명령은 "지금 이 장비에 즉시 도달했는가"가 중요하지 "영원히 보관되는가"가 중요하지 않기 때문입니다.

다만 올라가는 방향은 Kafka 를 탑니다 — sensor.alerts 를 telegraf-alert-republish 그룹이 읽어 MQTT scada/alerts/<태그명> 로 되돌려 주고, FUXA 가 그것을 구독해 P&ID(Piping and Instrumentation Diagram, 배관 계장도 — 탱크·펌프·배관·계기를 기호로 그린 공정 도면) 화면의 설비 심볼을 점멸시킵니다.

대안과 비교

같은 자리의 다른 도구들 RabbitMQ · ActiveMQ (전통 메시지 브로커) — 소비자가 가져가면 큐에서 지우는 방식입니다. 메시지를 조건에 따라 이 큐 저 큐로 정교하게 라우팅하는 능력은 Kafka 보다 뛰어나고, 작업 분배(job queue)에는 더 알맞습니다. 그러나 과거 재생이 불가능하고, 소비자가 늘어날수록 큐를 늘려 복제 배달해야 합니다. "지우지 않는 장부"라는 이번 장의 성질 자체가 없습니다. 참고로 이 이름은 설계 보고서의 참고자료 61 번 (ThingsBoard 저장소의 "Durable message queues with RabbitMQ" 이슈)에 한 번 등장할 뿐이며, AR-100 프로젝트의 의사결정 기록에서 정식으로 검토된 후보는 아닙니다. Apache Pulsar — Kafka 와 같은 자리를 노리는 가장 직접적인 경쟁자입니다. 메시지를 받아 주는 브로커 와 실제로 저장하는 계층(BookKeeper — 로그 조각을 여러 대에 나눠 안전하게 보관하는 별도의 분산 저장 시스템)을 분리한 구조라, 저장 용량과 처리 능력을 각각 따로 늘릴 수 있습니다. 테넌트 격리도 강점입니다. 테넌트(tenant)는 같은 시스템을 함께 쓰는 서로 다른 조직·부서를 뜻하고, 테넌트 격리는 한 부서

꼭주해도 다른 부서의 성능·권한·용량이 침범당하지 않게 칸을 막아 주는 기능입니다. 대신 구성 요소가 하나 더 늘어 운영 난도가 높고 주변 생태계·자료가 Kafka 보다 얇습니다. PDF 표에서 Apache StreamPipes 항목이 "내장 Kafka/Pulsar 엔진 기반"으로 적힌 것이 이 대체 관계를 보여 줍니다. Redis Streams — Redis 안에 로그형 자료구조를 넣은 것으로, 오프셋과 컨슈머 그룹 개념도 있습니다. 매우 가볍고 빠르지만 기본이 메모리이고, 장기 보존·복제·대용량 재생에서는 Kafka 와 급이 다릅니다. 짧은 완충에는 좋고 전사 백본에는 맞지 않습니다. NATS JetStream — 초경량·저지연 메시징에 보존 기능을 더한 것입니다. 에지 구간의 가벼운 스트리밍에 는 매력적이거나, 수년 치 이력과 대규모 재처리를 전제한 히스토리안 앞단으로는 검증 사례가 적습니다. AWS Kinesis · Google Pub/Sub · Azure Event Hubs (관리형) — 서버를 직접 세우지 않고 클라우드 사업자가 운영해 주는 형태입니다. Kinesis 의 샤드(shard, 조각)는 Kafka 의 파티션에, Pub/Sub 의 구독(subscription)은 컨슈머 그룹에 각각 대응합니다. 운영 부담이 사라지는 것이 최대 장점입니다. 대신 보존 기간 상한과 과금이 사업자의 정책으로 정해지고 클라우드 사업자에 묶입니다. 설계 보고서가 사용자 제안 스택을 평가하며 "특정 상용 프레임워크에 종속되지 않으면서도 시스템 처리 용량을 무제한으로 수평 확장"할 수 있다고 적은 것이 바로 이 대비입니다. ThingsBoard 내장 큐 — 장치 관리·룰 엔진·대시보드를 한 제품으로 묶은 IoT 플랫폼으로, 내부에 자체 메시지 큐를 갖고 있고 룰 엔진의 Kafka 노드로 외부 Kafka 에 내보낼 수도 있습니다. 시작은 빠르지만 설계 보고서는 "플랫폼의 모놀리식/마이크로서비스 구조 자체가 무겁고, 초당 수십만 건 이상의 초고빈도 스트림이 유입 될 경우 플랫폼 내부 메시지 큐의 오버헤드가 발생할 수 있다"고 평가합니다. AR-100 의 의사결정 기록(ADR-003)이 ThingsBoard 를 기각한 사유는 성능이 아니라 한 줄입니다 — "EdgeX(디바이스 관리) + FUXA(HMI) 와 역할 100% 중복". EdgeX 내부의 eKuiper / ZeroMQ, 또는 MQTT 직결 — 에지 한 대 안에서만 데이터를 돌린다면 이것으로 충분합니다. PDF 표도 EdgeX 의 수집 백본을 "내부 eKuiper/ZeroMQ 또는 MQTT"로, 확장성을 "단일 에지 게이트웨이 리소스에 종속"으로 적습니다. 공장 전체·여러 라인·여러 소비 부서가 등장하는 순간 이 자리는 Kafka 가 말합니다. 데이터베이스 테이블을 큐처럼 쓰기 — 현장에서 여전히 많이 보이는 방식입니다. INSERT 로 쌓고 "처리 함" 표시 칸을 UPDATE 로 바꿔 가며 진행 상황을 관리합니다. 소규모에서는 동작하지만 세 가지가 발목을 잡습니다. 첫째, 소비자가 늘면 잠금 경합(lock contention)이 생깁니다. 데이터베이스는 같은 행을 두 프로그램이 동시에 고치지 못하도록 잠금(lock)을 걸어 한쪽을 기다리게 하는데, 여러 소비자가 "아직 처리 안 된 다음 건"을 서로 먼저 집으려 달려들면 대부분의 시간을 서로 기다리며 보내게 됩니다. 둘째, 순서 보장과 병렬 처리를 동시에 얻기 어렵습니다. 셋째, 무엇보다 장부가 곧 저장소라서 하류 장애가 상류를 세웁니다. 이번 장이 처음에 열거한 세 가지 병이 그대로 남습니다.

본질의 차이는 한 줄로 갈립니다. 위 도구들 대부분은 "메시지를 배달하는" 물건이고, Kafka 는 "일어난 일을 보관하는" 물건입니다. 배달이 필요하면 MQTT 나 RabbitMQ 가 낫고, 같은 사실을 여러 부서가 각자 속도로 몇 번이고 다시 읽어야 하면 Kafka 가 유일한 답에 가깝습니다. AR-100 이 두 가지를 모두 쓰고 그 사이에 브리지 를 세운 이유가 여기 있습니다.

5 스트림 처리 — 멈추지 않는 질의

원문 교본 139-174 쪽의 설명을 통합

이 장은 시간·상태·복구를 먼저 이해한 뒤, 규격 검사와 Z-Score, 결측 보간이 그 위에서 어떻게 동작하는지 연결합니다. 같은 Flink 안에서도 SQL 잡과 Java ONNX 잡의 시간 처리 방식은 다릅니다.

배치와 스트림 · 창 · 워터마크 · 상태 · 정확히 한 번 · 탐지와 보간

한 번 묻고 끝나는 질의와, 커 두는 질의

먼저 질의(query)라는 말부터 풀겠습니다. 질의는 데이터에 던지는 물음입니다. "지난달 불량률은 몇 퍼센트인가", "지금 반응기 압력이 6 barg를 넘었는가" 같은 것이 전부 질의입니다.

barg도 여기서 풀고 갑니다. bar는 압력의 단위이고 뒤에 붙은 g는 gauge, 게이지 압력이라는 뜻입니다. 우리가 숨 쉬는 대기압을 0으로 놓고 그보다 얼마나 더 높은지를 잴 값입니다. 반대로 완전한 진공을 0으로 놓고 재면 절대압이고 bara로 씁니다. 대기압이 약 1 bar이므로 1 barg는 2 bara와 같은 압력입니다. 공장의 압력계는 거의 전부 게이지 압력을 표시합니다. 탱크가 터질지를 정하는 것은 안과 밖의 압력 차이이지 절대값이 아니기 때문입니다. 이 교본에서 6 barg가 반응기의 상한 규격이라는 말은 "대기압보다 6 bar 높은 데까지가 설계 한계"라는 뜻입니다.

이제 본론입니다. 데이터베이스를 배운 사람이 아는 질의는 전부 배치 질의(batch query)입니다. 데이터가 이미 어딘가에 다 쌓여 있고, 그 고정된 더미를 향해 물음을 한 번 던지면, 엔진이 처음부터 끝까지 훑어서 답 하나를 내놓고 끝납니다. 물음을 다시 하려면 질의를 다시 실행해야 합니다.

이 방식이 왜 공장에서 문제가 되는지는 순서를 그려 보면 바로 보입니다. 센서값이 발생한다 → 브로커를 거쳐 데이터베이스에 적재된다 → 누군가(사람이든 스케줄러든) 질의를 실행한다 → 답이 나온다. 이 순서에서는 값이 저장된 다음에야 판단이 시작됩니다. 설계 보고서는 이 지점을 분명히 지적합니다. 시계열 데이터베이스에 일단 적재한 뒤에 보간이나 탐지를 수행하는 방식은 "실시간 이상 탐지가 요구하는 수십 밀리초 단위의 종단 간 지연시간(End-to-End Latency) 요건을 충족할 수 없다"는 것입니다. 반응기 압력이 규격을 넘은 순간 인터록이 걸려야 하는데 5분 뒤에 배치 질의가 돌아서 알려 주면 이미 사고가 끝난 뒤입니다.

인터록(interlock)도 처음 나왔으니 풀겠습니다. 정해진 조건이 성립하면 사람의 판단을 기다리지 않고 설비가 스스로 거는 잠금 또는 강제 정지입니다. "압력이 6.5 barg에 닿으면 공급펌프를 무조건 세운다" 같은 것입니다. 운전원이 승인 버튼을 누를 필요도 없고, 누르지 않아도 걸립니다. 사람의 반응 속도로는 늦기 때문에 기계에게 미리 맡겨 두는 조치입니다.

그러면 배치 질의를 아주 자주 돌리면 되지 않나. 5초마다 돌리면 되지 않느냐는 생각이 자연스럽게 듭니다. 그렇게 해도 근본 문제는 그대로입니다. 문제는 속도가 아니라 판단이 적재 뒤에 시작된다는 순서 자체이기 때문입니다. 덧붙여 비용도 계속 듭니다. 배치 질의는 아무것도 기억하지 않으므로 5초 전에 계산해 둔 결과를 이어서 쓸 수 없고, "최근 60초 평균"처럼 범위가 실행 간격보다 긴 질문이면 겹치는 55초를 매번 다시 읽습니다. 뒤에서 볼 스트림 질의가 이것을 상태(state)로 해결합니다.

그래서 나온 것이 스트림 질의(streaming query)입니다. 발상을 뒤집습니다. 데이터를 고정해 두고 질의를 지나가게 하는 대신, 질의를 고정해 두고 데이터를 지나가게 합니다. 질의문을 한 번 등록하면 그 질의는 종료되지 않고 계속 살아 있으면서, 새 레코드가 하나 도착할 때마다 자기 결과를 조금씩 갱신합니다. 개념적으로는 "끝이 없는 표(unbounded table)"에 붙어 있는 뷰라고 생각하면 정확합니다. 표에 행이 추가될 때마다 뷰가 갱신되는데, 그 표에 마지막 행이 영원히 오지 않는 것입니다.

구조는 이렇게 생겼습니다. 스트림 처리 엔진의 프로그램 단위를 잡(job)이라고 부릅니다. 잡 하나는 연산자(operator)들이 줄줄이 이어진 그래프입니다. 맨 앞은 데이터를 읽어 들이는 소스(source), 가운데는 거르고 계산하고 합치는 변환 연산자들, 맨 끝은 결과를 내보내는 싱크(sink)입니다. 이 그래프가 일단 배치되면 데이터는 소스에서 싱크로 한 방향으로만 흐릅니다. 서로 이어진 연산자 몇 개를 한 덩어리로 묶어 같은 스레드에서 돌리는 것을 연산자 체인(operator chain)이라고 합니다. 연산자 사이에 데이터를 주고받는 비용을 없애려는 최적화입니다.

이 잡을 실제로 돌리는 쪽은 두 종류의 프로세스입니다. 잡매니저(JobManager)는 감독입니다. 잡을 받아 실행 계획을 세우고, 어느 기계에 어느 연산자를 놓을지 정하고, 뒤에서 설명할 체크포인트를 지시하고, 뭔가 죽으면 다시 살립니다. 태스크매니저(TaskManager)는 작업자입니다. 실제 연산을 수행 하며, 자기 자원을 슬롯(slot)이라는 칸으로 나눠

깁니다. 둘 다 JVM 프로세스입니다 — JVM은 Java Virtual Machine, 자바로 만든 프로그램이 그 위에서 도는 가상 기계입니다. Flink가 자바로 만들어졌기 때문에 앞으로 나올 메모리 이야기가 전부 이 JVM 이야기입니다.

슬롯이 무엇을 담는지가 초보에게 가장 헷갈리는 대목입니다. 슬롯 하나에 연산자 하나가 들어가는 것이 아닙니다. Flink는 기본적으로 슬롯 공유(slot sharing)를 하므로, 슬롯 하나가 잡 파이프라인 한 줄을 통째로 담습니다. 소스 하나, 보간 연산자 하나, 추론 연산자 하나, 싱크 하나가 한 슬롯 안에 같이 앉아 있는 모양입니다. 그래서 필요한 슬롯 수는 "연산자 개수"가 아니라 "잡에서 가장 병렬도가 높은 연산자의 병렬도"가 됩니다.

병렬도(parallelism)는 같은 연산자를 몇 벌 복제해 데이터를 나눠 처리할지를 정하는 숫자입니다. 병렬도가 2면 그 잡은 슬롯 2칸을 씁니다. 병렬도를 올리면 처리량이 늘어나는 것은 맞지만, 어디까지나 나눌 것이 남아 있을 때까지입니다. 뒤에서 보겠지만 이 시스템에서는 소스가 읽는 토픽의 파티션이 6개이고 태그가 12개이므로, 그 숫자를 넘겨 병렬도를 올리면 남는 인스턴스는 그냥 놓입니다. 그리고 이 교본이 인용하는 실측 기록 어디에도 병렬도를 바꿔 가며 처리량을 쟀 시험은 없습니다. "슬롯을 늘리면 선형으로 빨라진다"는 말은 이 시스템에서 확인되지 않았습니다. 그래도 Esper 같은 단일 JVM 엔진과 갈리는 결정적 차이는 여전히 여기입니다 — 그쪽은 늘릴 수 있는 축이 애초에 없습니다.

초보가 반드시 헷갈리는 지점 세 가지를 미리 짚겠습니다. 첫째, 스트림 SQL은 실행해도 끝나지 않습니다. INSERT INTO alerts SELECT ... FROM telemetry_raw를 실행하면 프롬프트가 돌아오지 않고, 대신 "잡이 제출되었습니다"라는 메시지와 잡 ID가 나옵니다. 이 문장은 답을 내고 죽는 것이 아니라 계속 살아 있는 프로그램이 됩니다. 그래서 "SQL이 왜 안 끝나지?"가 아니라 "SQL이 잘 켜졌구나"가 맞는 반응입니다. 둘째, 결과는 한 번 확정되고 마는 값이 아닙니다. 배치 질의의 결과는 스냅샷이지만 스트림 질의의 결과는 살아 움직이는 값입니다. "현재 평균"은 1초 뒤에 다른 값입니다. 셋째, 실시간은 "빠르다"가 아니라 "지연에 상한이 있다"는 뜻입니다. 스트림 처리가 배치보다 무조건 빠른 것이 아니라, 결과가 나오기까지 기다려야 하는 시간의 최대치를 설계로 묶어 둘 수 있다는 것이 본질입니다.

구분	배치 질의	스트림 질의
데이터	고정된 유한한 더미	끝이 없는 무한한 흐름
실행	한 번 실행하고 종료	등록 후 종료하지 않음
결과	그 시점의 답 하나	계속 갱신되는 값
범위 지정	WHERE ts BETWEEN ... 로 명시	창(window)·워터마크로 지정
직전 계산 재사용	불가 — 매번 처음부터	상태로 이어서 씀
"완료"의 의미	전부 읽었다	이 시각까지는 다 왔다고 본다
실패했을 때	다시 실행하면 그만	중간 상태까지 복구해야 함
전형적 지연	분~시간	밀리초~초

AR-100에서는, 스트림 계층은 Apache Flink 1.20.1 한 벌이고(flink/Dockerfile 의 FROM flink:1.20.1-scala_2.12-java17), 그 위에서 잡이 정확히 4개 돕니다.

규격 위반 탐지·롤링 Z-Score·CEP 패턴 매칭 세 개는 flink/sql/ 아래의 순수 SQL 파일 (02_tier1_rules.sql , 03_tier1_zscore.sql , 04_tier1_cep.sql)로 돌고, 결측 보간과 ONNX 추론 한 개는 자바 잡(AnomalyJob.java)으로 돕니다. 여기서 약어 두 개를 미리 풀어 돕니다. CEP는 Complex Event Processing, 복합 이벤트 처리입니다. 값 하나가 선을 넘었는지를 보는 대신 여러 이벤트가 어떤 순서로, 얼마 안에 일어났는지를 보는 탐지입니다. 이 장에서는 이름과 자리만 잡고, 문법과 실제 패턴은 6장에서 따로 다룹니다. ONNX는 Open Neural Network Exchange, 학습을 끝낸 신경망 모델을 특정 프레임워크에 매이지 않는 공통 파일 형식으로 저장하는 규격입니다. 파이썬에서 학습한 모델을 .onnx 파일 하나로 내 보내면 자바로 돌아가는 Flink 태스크매니저 안에서 그대로 실행할 수 있습니다. 모델 자체는 6장의 주제이고, 이 장에서는 "그 모델에 먹일 입력을 어떻게 만드는가"만 다룹니다.

설정은 flink/conf/config.yaml 한 곳에 모여 있고 taskmanager.numberOfTaskSlots: 8 , parallelism.default: 2 입니다. 이 두 숫자와 잡 4개가 정확히 맞물립니다 — 잡 4개 × 병렬도 2 = 슬롯 8칸으로, 남거나 모자라지 않습니다. 앞에서 본 슬롯 공유 덕분에, 소스·보간·추론·싱크가 이어진 자바 잡도 슬롯을 2칸만 씁니다.

잡이 실제로 살아 있는지는 텍스트 로그가 아니라 상태로 확인합니다 — curl http://flink-jobmanager:8081/jobs/overview 의 "state":"RUNNING" 개수를 세어 4개 미만이면 기동을 실패로 판 정합니다(flink/submit-jobs.sh). 사람이 볼 화면은 http://localhost:27081 입니다. 실측 유입량은 12 태그 × 1 Hz = 12 건/초(Kafka 오프셋 차분으로 20 초에 252 건), 이때 태스크매니저가 2.19 GiB, 잡매니저가 1.16 GiB를 씁니다.

태스크매니저 1대 · 슬롯 8칸

numberOfTaskSlots: 8 · parallelism.default: 2 · 잡 4개 → $4 \times 2 = 8$



슬롯 한 칸에는 연산자 하나가 아니라 잡 파이프라인 한 줄이 통째로 들어갑니다.

보라색 칸 하나에 소스 · 보간(Interpolator) · 추론(OnnxScorer) · 싱크 3개가 모두 앉아 있습니다.

그래서 필요한 슬롯 수는 연산자 개수가 아니라 가장 높은 병렬도로 정해집니다.

원문 도해 · 태스크매니저의 슬롯과 네 개 잡 (원문 142 쪽)

5.1 창(window) — 무한한 것을 유한하게 자르는 법

끝나지 않는 표에서 평균을 내는 방법

"지난 1분간 평균 온도"는 배치에서는 아무 문제가 없는 질문입니다. 지난 1분이 이미 지나갔고 그 구간의 데이터가 전부 있으니까요. 그런데 스트림에서 "평균 온도"를 그냥 물으면 답이 없습니다. 데이터가 무한하니 평균도 영원히 확정되지 않습니다. 합계·평균·개수·최댓값처럼 여러 건을 모아야 나오는 계산(집계, aggregation)은 무한 스트림에서 그대로 쓸 수 없습니다. 그래서 무한한 흐름을 유한한 토막으로 잘라야 하고, 그 토막이 창(window)입니다. 창은 스트림 처리에서 멧을 부리려고 만든 개념이 아니라, 무한을 유한으로 바꾸는 유일한 방법이라서 반드시 등장하는 개념입니다.

창은 세 가지가 표준입니다.

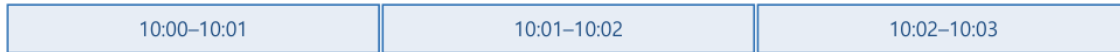
① 텀블링 창(tumbling window) — 겹치지 않게 똑똑 자릅니다. 길이만 정합니다. 1분 텀블링이면 10:00:00–10:01:00, 10:01:00–10:02:00 처럼 붙어 있으면서 겹치지 않는 토막이 만들어집니다. 레코드 하나는 반드시 창 하나에만 들어갑니다. 회계적으로 말이 되는 집계, 즉 합산해서 보고해야 하는 값에 씁니다. 시간당 생산량, 교대조별 불량 개수, 1분 평균 소비전력 같은 것입니다. 중복 계산이 없다는 것이 이 창의 존재 이유입니다.

② 슬라이딩 창(sliding window) — 길이와 간격을 따로 정합니다. "길이 60 초, 10 초마다"라고 하면 60 초짜리 창이 10 초씩 앞으로 미끄러집니다. 창끼리 50 초씩 겹치므로 레코드 하나가 창 6 개에 동시에 들어갑니다. 왜 이렇게 하느냐면, 텀블링만 쓰면 이상 신호가 창 경계에 걸쳐서 반 토막 나기 때문입니다. 59 초와 61 초에 걸쳐 일어난 급상승은 1분 텀블링에서는 두 창에 나뉘어 둘 다 맛있게 보입니다. 감시와 이상 탐지는 거의 항상 슬라이딩입니다. 대신 계산량이 겹치는 배수만큼 늘고, 같은 이상에 대 해 알람이 여러 번 울릴 수 있습니다.

슬라이딩 창이 보장하는 것을 정확히 적어 둡니다. 흔히 "슬라이딩을 쓰면 최근 60 초는 항상 온전히 한 창 안에 있다"고 말하는데 이것은 사실이 아닙니다. 길이 L , 간격 s 인 슬라이딩 창에서 어떤 창 하나에 통째로 담기는 것이 보장되는 구간의 길이는 $L - s$ 까지입니다. 길이 60 초·간격 10 초라면 50 초입니다. 임의의 시점에 뒤를 돌아보면 최근 60 초 중 최대 10 초는 어느 창에도 온전히 들어가 있지 않을 수 있습니다. 그러니 "몇 초짜리 사건까지 반드시 한 창 안에서 보고 싶은가"를 먼저 정하고, $L - s$ 가 그 길이보다 크도록 간격을 잡아야 합니다. 간격을 좁힐수록 보장 범위는 L 에 가까워지고 계산량은 그만큼 늘어납니다.

③ 세션 창(session window) — 길이를 정하지 않고 "조용해지면 닫습니다". 정하는 것은 갭(gap), 즉 "이만큼 아무 일도 없으면 한 덩어리가 끝난 것으로 본다"는 간격뿐입니다. 갭을 5 분으로 잡으면, 이벤트가 계속 들어오는 동안 창은 계속 자라다가 5 분간 비면 그 자리에서 닫히고 다음 이벤트부터 새 창 이 시작됩니다. 길이가 데이터에 따라 제각각입니다. 활동이 몰렸다 끊겼다 하는 대상에 씁니다 — 배치 한 번의 생산 구간, 작업자 한 명의 조작 세션, 정비 작업 한 건, 설비 한 번의 기동·정지 사이클 같은 것입니다. 사람이나 공정의 "한 건"이 몇 분짜리인지 미리 알 수 없을 때 유일하게 맞는 창입니다.

① 텀블링 창 — 겹침 없음, 빈틈 없음



레코드 하나는 창 하나에만 · 1분 생산량, 교대조 불량수

② 슬라이딩 창 — 길이 60초, 30초마다 새로 시작



같은 값이 여러 창에 들어간다 · 한 창에 통째로 담기는 것이 보장되는 길이 = $60 - 30 = 30$ 초

③ 세션 창 — 조용해지면(갭) 그 자리에서 닫힌다



창 길이가 데이터에 따라 제각각 · 배치 한 번, 정비 한 건, 기동-정지 한 사이클

시간 →

원문 도해 · 텀블링·슬라이딩·세션 창 (원문 144 쪽)

창	지정하는 것	겹침	한 레코드가 속하는 창 수	온전히 담기는 구간	공장에서 쓰는 자리
텀블링	길이 L	없음	정확히 1 개	경계에 걸리면 잘림	생산량·불량수 집계, 교대조 보고
슬라이딩	길이 L + 간격 s	있음	$L \div s$ 개	L - s 까지	이상 탐지, 이동평균, 추세 감시
세션	갭	없음	정확히 1 개	한 덩어리 전체	배치 단위, 정비 작업, 기동-정지 사이클

AR-100에서는, 세 창을 다 쓰지는 않습니다. 이 시스템이 실제로 쓰는 것은 슬라이딩 계열 두 가지입니다.

첫째, SQL의 행 기준 슬라이딩. `flink/sql/03_tier1_zscore.sql`이 이렇게 선언합니다.

```
WINDOW w AS (PARTITION BY tag ORDER BY event_time
ROWS BETWEEN 59 PRECEDING AND CURRENT ROW)
```

"시간 60 초"가 아니라 "직전 59 행 + 현재 행 = 60 행"입니다. 태그마다 1 초에 정확히 한 건이 오므로 결과적으로 60 초이지만, 결측이 생기면 시간이 아니라 개수가 기준이 됩니다. 값이 한 건 들어올 때마다 창이 한 칸 미끄러지므로 간격이 1 행짜리 슬라이딩 창이고, 앞의 공식에 넣으면 $L - s = 59$ 행이 온전히 담기는 범위입니다. 거의 L에 가까운 대신 매 행마다 60 개를 다시 집계하는 비용을 치릅니다.

둘째, 자바 잡의 텐서 슬라이딩 버퍼. 여기서 텐서(tensor)를 풀고 갑니다. 숫자를 여러 축으로 늘어 놓은 덩어리입니다. 숫자 하나는 0 차원, 한 줄로 늘어놓으면 1 차원, 가로세로 표현 2 차원입니다. 신경망의 입력은 이 모양과 크기가 미리 고정되어 있어서, 정해진 칸을 하나도 빠짐없이 채워 줘야 계산이 돌아갑니다. `flink/onnx-job/job.properties`의 `window.steps = 10`, `inference.interval.ms = 1000`에 따라 12 개 태그의 최신값 벡터를 1 초마다 한 줄씩 쌓아 10 줄을 채우고, 다 차면 맨 앞 줄을 버리고 뒤에 한 줄을 붙입니다(`OnnxScorer.java`의 `MapState<Integer, double[]> window`). 10 줄 × 12 태그 = 120 칸짜리 텐서가 1 초마다 하나씩 나옵니다.

여기에 더해 CEP 패턴에는 `WITHIN INTERVAL '10' SECOND`라는 시간 제한이 붙는데(`04_tier1_cep.sql`), 이것도 "패턴 전체가 성립해야 하는 창"입니다.

설정 파일을 바꿀 때 주의할 것이 있습니다. `job.properties` 머리말에는 "재빌드 없이 이 파일만 바꿔 동작을 조정합니다"라고 적혀 있는데, 그 주석이 사실과 다릅니다. `flink/Dockerfile`이 `COPY onnx-job/job.properties /opt/flink/job/job.properties`로 이 파일을 이미지 안에 구워 넣고, `docker-compose.yml`이 Flink 컨테이너에 걸어 주는 볼륨은 `flink/conf/config.yaml`과 `flink/sql/` 둘뿐입니다. 즉 이 파일은 컨테이너 밖에서 바뀌어도 반영되지 않습니다. 저널의 재시작 범위표도 같은 말을 합니다 — `job.properties`를 바꾸면 Flink 이미지 재빌드 + `make jobs`가 필요합니다(`10-config-reference.md § 10.4`). 반대로 `flink/sql/*.sql`은 볼륨으로 마운트되어 있으므로 `make jobs`만으로 반영됩니다.

주의. 창을 고를 때 초보가 가장 자주 하는 실수는 탐지에 텀블링을 쓰는 것입니다. "1 분마다 평균 을 내서 이상하면 알람"은 구현이 쉬워서 자꾸 손이 갑니다. 그러나 이상 신호가 창 경계에 걸치면 두 창 모두 정상으로 보입니다. 반대로 슬라이딩을 쓰면 같은 이상 하나에 알람이 겹친 창 수만큼 울립니다. 이 중복은 창을 바꿔서가 아니라 뒤에서 볼 지속성 조건과 알람

억제로 다릅니다. 세션 창에도 함정이 있습니다. 갭이 끝나지 않으면 창이 영원히 닫히지 않고 그만큼 상태가 계속 커집니다. 세션 창을 쓸 때는 최대 길이를 같이 걸어야 합니다.

5.2 이벤트 시각과 처리 시각 — 워터마크와 지각 데이터

시각이 두 개라는 사실, 그리고 언제 창을 닫을지 정하는 법

레코드 하나에는 서로 다른 시각이 최소 두 개 붙습니다. 이벤트 시각(event time)은 그 일이 실제로 일어난 시각입니다. 센서가 72.43 °C 를 읽은 바로 그 순간 이고, 레코드 안에 ts 필드로 실려 다닙니다. 처리 시각(processing time)은 그 레코드가 스트림 엔진에 도착해서 처리되는 순간의 벽시계 시각입니다. 레코드 안에 들어 있지 않고, 연산자가 자기 시계를 보고 알아냅니다.

둘은 반드시 어긋납니다. 이유가 여러 겹입니다. 전송 지연이 있고, 네트워크가 잠깐 끊겼다가 살아나면 에지 게이트웨이가 쌓아 뒀던 것을 한꺼번에 밀어 넣습니다(3 장의 Store-and-Forward). Kafka 파티션이 여러 개면 파티션마다 속도가 달라서 순서가 섞입니다. 심지어 도착 순서가 발생 순서와 뒤집히기도 합니다. 10 초 전에 발생한 값이 지금 막 도착하는 일이 정상적으로 벌어집니다.

그래서 어느 시각으로 계산해야 하는가. 원칙은 분명합니다. 이벤트 시각으로 계산해야 합니다. 처리 시각으로 창을 자르면 같은 데이터를 다시 흘려보냈을 때 결과가 달라집니다. 어제 일어난 사고를 재현하려고 어제 데이터를 다시 넣었는데 창 경계가 달라져서 알람이 안 뜨면, 그 시스템으로는 사고 원인을 규명할 수 없습니다. 이벤트 시각 기준이면 몇 번을 다시 돌려도 같은 답이 나옵니다. 이것을 결정론적(deterministic) 재처리라고 부릅니다.

그런데 여기서 진짜 문제가 생깁니다. 이벤트 시각으로 "10:00:00-10:01:00 창"을 만들었다고 합시다. 이 창을 언제 닫고 평균을 계산해야 합니까? 10:01:00 에 해당하는 레코드가 들어왔다고 닫아 버리면, 아직 길에서 오고 있는 10:00:58 짜리 레코드를 빠뜨립니다. 그렇다고 영원히 기다리면 결과가 영원히 나오지 않습니다. 무한 스트림에는 "다 왔다"는 신호가 존재하지 않습니다. 이 근본적인 문제를 푸는 장치가 워터마크입니다.

워터마크(watermark)는 스트림에 섞여 흐르는 특별한 표식입니다. 값이 하나 들어 있는데, 그 값의 뜻은 이렇습니다 — "이 시각보다 이른 이벤트는 이제 더 오지 않는다고 본다." 마지막 문장이 중요합니다. 실제로 안 온다는 보장이 아니라, 안 온다고 치고 진행하겠다는 선언입니다. 가장 흔한 생성 규칙은 "지금까지 본 이벤트 시각의 최댓값에서 여유 시간 T 를 뺀 값"입니다. T 가 허용 지연(out-of-orderness)입니다. 워터마크가 어떤 창의 끝 시각을 넘어서면 그 창이 닫히고 결과가 나옵니다.

T 를 정하는 것은 순수한 맞교환입니다. T 를 크게 잡으면 늦게 오는 데이터를 잘 담지만 모든 결과가 그만큼 늦게 나옵니다. T 를 작게 잡으면 결과는 빨리 나오지만 늦게 온 데이터가 버려집니다. 워터마크보다 이른 이벤트 시각을 달고 뒤늦게 도착한 레코드를 지각 데이터(late data)라고 하고, 기본 동작 은 조용히 버리는 것입니다. 버리기 싫으면 세 가지 선택지가 있습니다 — 허용 지각(allowed lateness)을 걸어 창을 잠시 더 살려 두고 결과를 갱신하거나, 별도 출력(side output)으로 빼서 따로 처리하거나, 처음부터 T 를 넉넉히 잡거나.

그리고 T 는 지연 예산에 그대로 더해집니다. 이벤트 시각으로 정렬해서 내보내는 연산은 워터마크가 그 행의 시각을 지나야 결과를 낼 수 있으므로, 어떤 판정이든 최소 T 만큼은 늦게 나옵니다. 5.6 에서 알람이 실제로 몇 초 늦는지 계산할 때 이 항이 다시 나옵니다.

워터마크 = 3s 최대 이벤트 시각(8s) - 허용 지연(5s) 이 선을 넘긴 창은 닫고 결과를 내보내다 닫혔다고 보는 구간원 안 숫자 = 도착 순서 순서가 뒤집혀 도착 1 2 3 5 4 6 1s 2s 3s 4s 5s 7s 8s 이벤트 시각 →

⑤는 ④보다 이른 시각인데 늦게 왔다 — 그래도 워터마크(3s) 오른쪽이므로 정상 처리된다. 만약 2s 짜리가 지금 도착한다면 워터마크 왼쪽이므로 지각 데이터가 되어 기본값으로는 버려진다. 워터마크는 "다 왔다"는 사실이 아니라 "다 왔다고 치겠다"는 약속입니다. 허용 지연을 얼마로 잡느냐가 곧 지연과 누락의 맞교환입니다.

AR-100에서는, `flink/sql/01_sources.sql` 에 이벤트 시각과 워터마크가 함께 선언되어 있습니다.

```
ts          BIGINT,
event_time  AS TO_TIMESTAMP_LTZ(ts / 1000000, 3),
-- 센서 클럭 지터와 EdgeX 스캔 편차를 흡수
WATERMARK FOR event_time AS event_time - INTERVAL '5' SECOND
```

세 줄에 세 가지 결정이 들어 있습니다. 첫째, 데이터 계약의 ts 는 나노초이므로 1,000,000 으로 나눠 밀리초로 바꾸고 정밀도 3 으로 타임스탬프를 만듭니다(`TO_TIMESTAMP_LTZ` 는 숫자를 시간대가 붙은 타임스탬프로 바꾸는 Flink SQL 함수이고, 둘째 인자 3 이 "소수점 아래 세 자리 = 밀리초"라는 뜻입니다). 둘째, `event_time` 은 저장된 컬럼이 아니라 계산 컬럼입니다 — ts 에서 그때그때 유도됩니다. 셋째, 허용 지연은 5 초입니다.

5 초라는 값의 근거로 주석에 적힌 지터(jitter)도 풀어 둡니다. 일정한 간격으로 와야 할 신호가 조금씩 앞뒤로 흔들리는 것입니다. 1 초마다 읽기로 한 센서가 실제로는 0.98 초, 1.03 초, 0.99 초 간격으로 읽히는 식입니다. 여기에 EdgeX 가 장비를 돌아가며 읽는 순서 때문에 생기는 편차가 겹칩니다. 둘 다 고장이 아니라 정상 동작의 일부이고, 그 흔들림을 덮을 만큼 워터마크를 뒤로 물린 값이 5 초입니다.

같은 파일에 SET 'table.exec.source.idle-timeout' = '5s'; 도 함께 있습니다. 왜 필요한지는 바로 아래 주의 상자에서 다룹니다.

반면 자바 잡은 일부러 처리 시각을 씁니다. AnomalyJob.java 는 소스를 WatermarkStrategy.noWatermarks() 로 붙이고, OnnxScorer.java 는 ctx.timerService().registerProcessingTimeTimer(fire) 로 1 초마다 타이머를 겁니다. 코드 주석에 이유가 그대로 적혀 있습니다 — "센서마다 도착 시각이 미세하게 다르므로 이벤트 단위로 추론하면 같은 시점이 중복 평가된다." 추론은 일정한 박자로 12 개 태그의 현재 스냅샷을 찍어야 의미가 있기 때문에, 이 잡에서는 이벤트 시각이 아니라 처리 시각이 맞는 기준입니다. 같은 파이프라인 안 에서도 연산의 목적에 따라 시각 기준이 갈립니다.

주의 — 이 시스템에서는 유틸 파티션이 반드시 생깁니다. sensor.telemetry.raw 토픽은 파티션이 6 개입니다(kafka/create-topics.sh). 그리고 Kafka 로 보내는 쪽인 Telegraf 설정에 이 줄이 있습니다.

```
routing_tag = "tag"          # 파티션 키 = 태그 → 태그 단위 순서 보장
```

파티션 키를 태그 이름으로 잡았다는 뜻입니다. 같은 태그는 항상 같은 파티션 하나에만 들어갑니다. 태그가 12 개, 파티션이 6 개이므로 파티션 하나가 태그 두 개씩을 맡습니다. 이것은 우연이 아니라 설계상 항상 참입니다 — Flink 의 키 단위 상태와 순서를 맞추려고 일부러 그렇게 한 것입니다 (3 장).

그런데 워터마크는 모든 입력 파티션 중 가장 뒤쳐진 것을 따라갑니다. 어떤 파티션이 맡은 태그들이 잠시 조용해지면 그 파티션의 워터마크가 멈추고, 잡 전체의 시계가 같이 멈춥니다. 창이 닫히지 않고 결과가 나오지 않는데 예러는 하나도 없는, 진단하기 가장 까다로운 증상이 이것입니다.

table.exec.source.idle-timeout = 5s 는 바로 이것을 막는 설정입니다. 5 초 동안 아무것도 안 오는 파티션은 "놓고 있음"으로 표시해 워터마크 계산에서 빼 버립니다. 파티션 키가 태그로 고정되어 있는 이 구조에서는 이 설정이 선택이 아니라 필수입니다. 값을 너무 짧게 잡으면 잠깐 쉬던 파티션 이 다시 보내는 데이터를 지각 처리하게 되므로, 스캔 주기(1 초)의 몇 배 수준으로 잡는 것이 안전 합니다.

5.3 상태(state) — 스트림이 기억하는 것

기억이 없으면 못 하는 일들, 그리고 그 기억을 지키는 법

연산은 두 종류입니다. 무상태(stateless) 연산은 지금 들어온 한 건만 보고 답을 냅니다. "값이 6.0 보다 크다" 같은 필터가 그렇습니다. 앞에 뭐가 있었는지 몰라도 되고, 그래서 아무 데서나 병렬로 돌릴 수 있고, 죽었다 살아나도 앓을 게 없습니다.

상태 연산(stateful operation)은 다릅니다. "최근 60 건의 평균"을 내려면 앞의 59 건을 기억하고 있어야 합니다. "직전 유효값"으로 결측을 메우려면 직전 유효값을 들고 있어야 합니다. "전류가 초과한 뒤 10 초 안에 진동이 초과했는가"를 판정하려면 전류가 초과했다는 사실과 그 시각을 기억하고 있어야 합니다. 이렇게 연산자가 레코드 사이에 들고 다니는 기억이 상태(state)입니다.

상태는 아무렇게나 저장되지 않고 키 단위(keyed state)로 관리됩니다. 스트림을 키로 쪼개면(keyBy 또는 SQL 의 PARTITION BY) 같은 키의 레코드는 항상 같은 병렬 인스턴스로 갑니다. 그 인스턴스는 자기가 맡은 키들의 상태만 들고 있으면 됩니다. 이래야 병렬 처리와 기억이 양립합니다. 상태의 모양은 몇 가지로 정해져 있습니다 — 값 하나를 담은 ValueState, 키-값 여러 쌍을 담은 MapState, 목록을 담은 ListState 등입니다.

상태를 어디에 둘 것인가 — 상태 백엔드(state backend). 선택지는 사실상 둘입니다.

힙(heap) 백엔드는 상태를 자바 객체 그대로 메모리에 둡니다. 접근이 아주 빠르지만 상태가 커지면 JVM 힙(자바 프로그램이 객체를 담아 두는 메모리 영역)이 터지고, 가비지 컬렉션이 길어져 처리가 툭툭 끊깁니다. 가비지 컬렉션은 JVM 이 더는 쓰이지 않는 객체를 찾아 메모리를 회수하는 청소 작업인데, 청소하는 동안 프로그램이 잠깐씩 멈춥니다. 치울 것이 많을수록 그 멈춤이 길어집니다.

RocksDB 백엔드는 상태를 RocksDB 라는 임베디드 키-값 저장소에 둡니다. RocksDB 는 페이스북이 만든 라이브러리로, 별도 서버 프로세스가 아니라 태스크매니저 프로세스 안에 라이브러리로 박혀서 로컬 디스크에 데이터를 씁니다. LSM 트리(Log-Structured Merge Tree) 구조를 쓰는데, 이름이 길지만 뜻은 간단합니다 — 고칠 자리를 찾아가서 제자리에

덮어쓰는 대신, 변경을 순차 로그처럼 뒤에 계속 이어 붙였다가 나중에 한꺼번에 병합하는 방식입니다. 디스크는 여기저기 찾아가며 쓰는 것보다 한 줄로 이어 쓰는 것이 훨씬 빠르므로, 쓰기가 많은 작업에 유리합니다. RocksDB는 자주 쓰는 부분만 메모리에 캐시하고 나머지는 디스크에 둡니다. 그래서 메모리보다 큰 상태를 다룰 수 있습니다. 대가는 접근할 때마다 직렬화·역직렬화가 일어난다는 것입니다. 직렬화는 메모리 안의 객체를 바이트 줄로 쪼개는 것이고 역직렬화는 그 반대입니다. 디스크에는 객체를 그대로 담을 수 없으니 넣을 때 쪼개고 꺼낼 때 다시 조립해야 하며, 이 왕복 비용 때문에 힙 백엔드보다 느립니다. 수천 개 센서의 슬라이딩 윈도우와 상관 행렬을 유지해야 하는 산업용 파이프라인에서는 RocksDB가 기본 선택이고, 설계 보고서도 Flink의 강점으로 "RocksDB 기반의 분산 상태 저장소"를 꼽습니다.

체크포인트(checkpoint) — 기억을 잃지 않는 법. 상태가 태스크매니저 안에만 있으면 그 기계가 죽는 순간 기억이 통째로 사라집니다. 그래서 주기적으로 모든 연산자의 상태를 한꺼번에 떼서 안전한 곳에 저장합니다. 이것이 체크포인트이고, 원리는 Chandy-Lamport 분산 스냅샷 알고리즘입니다. 이름이 어려워 보이지만 하는 일은 간단합니다. 잡매니저가 소스에 배리어(barrier)라는 표식을 하나 흘려 보냅니다. 배리어는 데이터와 같은 줄에 서서 그래프를 따라 내려가고, 각 연산자는 배리어가 자기에 게 도착하는 순간의 자기 상태를 잡아 둔 뒤 배리어를 다음으로 넘깁니다. 배리어가 마지막 싱크까지 통과하면 그 체크포인트가 완성됩니다.

여기서 두 가지를 구분해야 합니다. 스트림이 멈추지 않는 것은 배리어 방식 덕입니다. 모든 기계를 동시에 세우고 사진을 찍는 것이 아니라, 표식 하나가 데이터 줄에 끼어 흘러가면서 각자 자기 차례에 자기 것만 기록합니다. 그래서 전체를 정지시킬 필요가 없습니다. "비동기(asynchronous)"라는 이름은 다른 것을 가리킵니다. 배리어가 도착한 순간의 상태를 잡아 두기만 하고, 그것을 실제로 저장소에 써 넣는 작업은 뒤에서 따로 돌립니다. 연산자는 쓰기가 끝나기를 기다리지 않고 바로 다음 레코드를 처리합니다. 저장이 느려도 처리가 멈추지 않는다는 뜻이 "비동기"입니다. 두 장치가 합쳐져야 "돌면서 찍는다"가 성립합니다.

또 하나 중요한 것이 증분(incremental) 체크포인트입니다. 상태 전체를 매번 쓰는 대신 지난번 이후 바뀐 부분만 씁니다. RocksDB 백엔드에서만 가능하고, 상태가 클수록 차이가 큼니다.

세이프포인트(savepoint)는 체크포인트와 형제입니다. 체크포인트는 장애 복구용으로 엔진이 자동으로 뜨고 자동으로 지웁니다. 세이프포인트는 사람이 명령해서 뜨고, 잡을 정상적으로 세웠다가 코드를 고친 뒤 그 지점부터 다시 시작할 때 씁니다. 운영 중 무중단 배포가 이것으로 이루어집니다.

"정확히 한 번(exactly-once)"이 정확히 무슨 뜻인가. 여기가 이 절에서 가장 많이 오해받는 대목입니다. 스트림 처리에서 전달 보장은 세 단계로 이야기합니다. 최대 한 번(at-most-once): 유실될 수 있지만 중복은 없습니다. 아무 장치도 안 한 상태입니다. 최소 한 번(at-least-once): 유실은 없지만 같은 것이 두 번 나갈 수 있습니다. 장애 후 체크포인트 지점으로 되감으면 이미 내보낸 것을 다시 내보내기 때문입니다. 정확히 한 번(exactly-once): 유실도 중복도 없습니다.

그런데 Flink가 말하는 정확히 한 번은 정확히는 "상태에 대해 정확히 한 번"입니다. 즉 장애가 나서 되감아도, 복구된 뒤의 내부 상태(카운터, 평균, 패턴 진행 상황)는 마치 각 레코드가 딱 한 번만 처리된 것과 동일한 값이 된다는 뜻입니다. 외부로 내보내는 것까지 한 번임을 보장하려면 싱크가 따로 협조해야 합니다.

Kafka 싱크라면 2단계 커밋(two-phase commit)이 필요합니다. 이름 그대로 확정을 두 단계로 나눕니다 — 1단계에서는 "보낼 준비가 됐다"고 예약만 해 두고 아직 아무도 그 데이터를 읽을 수 없게 두었다가, 체크포인트가 완성되어 모든 참여자가 준비됐음이 확인되면 2단계에서 한꺼번에 확정합니다. 중간에 죽으면 예약을 통째로 취소하면 되므로 절반만 반영되는 일이 없습니다. 대가는 지연입니다. 확정이 체크포인트 주기에 묶이므로, 10초마다 체크포인트를 뜨는 설정이라면 결과도 최대 10초 뒤에야 하류에서 보입니다.

그래서 현실의 많은 파이프라인이 싱크는 최소 한 번으로 두고, 중복은 멱등성(idempotency)으로 흡수합니다. 멱등이란 같은 것을 두 번 넣어도 결과가 한 번 넣은 것과 같은 성질입니다. 고유 키를 달아 두고 이미 본 키는 무시하면 됩니다. 설계 보고서가 MQTT에서 QoS 2 대신 QoS 1을 쓰고 "중복 수신 문제는 스트림 처리 계층의 멱등성 로직으로 해소하는 것이 표준"이라고 한 것이 정확히 이 구도입니다.

항목	힙 상태 백엔드	RocksDB 상태 백엔드
저장 위치	JVM 힙 안, 객체 그대로	태스크매니저 프로세스 내 임베디드 DB, 로컬 디스크
상태 크기 한계	힙 크기까지	디스크 크기까지
접근 속도	매우 빠름	직렬화·역직렬화 비용만큼 느림
가비지 컬렉션 영향	상태가 클수록 심각	거의 없음
증분 체크포인트	불가	가능

항목	힙 상태 백엔드	RocksDB 상태 백엔드
null 값	넣어도 통과	거부 — 예외 발생
맞는 자리	상태가 작고 지연이 극단적으로 중요한 잡	센서 수천 개의 윈도우·상관 상태를 드는 산업용 잡

AR-100에서는, 상태와 복구 설정은 전부 `flink/conf/config.yaml`에 있습니다.

```
execution:
  checkpointing:
    interval: 10s
    mode: EXACTLY_ONCE
    timeout: 60s
    min-pause: 5s
    externalized-checkpoint-retention: RETAIN_ON_CANCELLATION
  state:
    backend:
      type: rocksdb
      incremental: true
      checkpoints:
        dir: file:///opt/flink/checkpoints
      savepoints:
        dir: file:///opt/flink/savepoints
```

PDF p.6 — RocksDB 기반 분산 상태 저장소

10 초마다 체크포인트를 쓰고, 한 번 쓰는 데 60 초를 넘기면 실패로 보고, 끝난 뒤 최소 5 초는 쉬었다가 다음을 시작합니다(`min-pause` 가 없으면 체크포인트가 쉬지 않고 이어져 정상 처리를 밀어냅니다).

`RETAIN_ON_CANCELLATION` 은 잡을 취소해도 체크포인트를 지우지 않겠다는 뜻이라, 사람이 세웠다가 그 지점에서 되살릴 수 있습니다.

실제 상태가 어디에 잡혀 있는지는 자바 코드에서 볼 수 있습니다. `Interpolator.java` 는 태그별로 `ValueState<Reading> lastGood` (직전 유효 관측치)과 `ValueState<Long> lastEmittedTsNs` (중복 제거용 마지막 타임스탬프)를 들고, `OnnxScorer.java` 는 디바이스별로 `MapState<String, Double> latest` (태그별 최신값)와 `MapState<Integer, double[]> window` (10 스텝 슬라이딩 텐서 버퍼)를 듭니다.

먹등 처리도 코드에 그대로 있습니다. `Interpolator.processElement()` 첫머리가 이렇습니다.

```
Long lastTs = lastEmittedTsNs.value();
if (lastTs != null && in.ts <= lastTs) {
    return; // MQTT QoS1 재전송으로 인한 중복 제거
}
```

같은 태그에서 이미 본 것과 같거나 더 이른 타임스탬프가 또 오면 그냥 버립니다.

그리고 정직하게 밝혀 둘 것이 셋 있습니다. 첫째, 이 잡의 Kafka 싱크는 정확히 한 번이 아닙니다. `AnomalyJob.java` 의 `jsonSink()` 가 `.setDeliveryGuarantee(DeliveryGuarantee.AT_LEAST_ONCE)` 로 설정되어 있습니다. 즉 엔진 내부 상태는 정확히 한 번, 외부로 나가는 쓰기는 최소 한 번입니다. 2 단계 커밋이 붙이는 지연을 피한 선택입니다. 둘째, 위의 타임스탬프 검사가 막아 주는 것은 입력 쪽 중복뿐입니다. raw 토픽에 같은 (태그, 시각)이 두 번 실려 와도 두 번째는 버려집니다. 그러나 잡이 죽었다 되살아나면서 `clean · score · alerts` 에 이미 쓴 것을 다시 쓰는 출력 쪽 중복은 이 검사로 걸러지지 않습니다. 그 것을 없애려면 하류(InfluxDB 적재, 알람 처리)가 자기 쪽에서 다시 걸러야 합니다. 셋째, 문서에 적힌 seq 먹등 키는 실제로는 흐르지 않습니다. `ARCHITECTURE.md` 의 데이터 계약 예시에는 "seq": 184213 이 들어 있고 " (device,tag,seq) 기준 dedup" 이라고 적혀 있지만, 파이프라인에는 없습니다. Telegraf 가 Kafka 로 내보내는 JSON 은 `telegraf/bridge-edgex.conf` 의 `json_transformation` 에 적힌 대로 `ts · site · device · tag · value · quality` 여섯 필드뿐이고, `Reading.java` 에도 seq 필드가 없으며, 자바 잡 어디에도 그것을 쓰는 코드가 없습니다. 시뮬레이터가 Modbus 레지스터 200 번에 스캔 카운터를 싣기는 하지만(`plant.yaml` 의 `sequence`), Telegraf 의 Starlark 필터가 계속 태그 12 개만 통과시키므로 Kafka 까지 오지 않습니다. 문서에 남은 옛 설계 와 실제 코드가 어긋난 자리이고, 이 교본은 코드 쪽을 사실로 씁니다.

"정확히 한 번"이라는 문구를 파이프라인 전 구간에 무비판적으로 붙이지 않는 것, 그리고 문서가 아니라 코드를 근거로 삼는 것이 이 설계의 태도입니다.

주의 — 체크포인트만으로는 복구되지 않습니다. 이 시스템 구축 중에 실제로 겪은 사고입니다 (ADR-010). 태스크매니저 하트비트가 타임아웃되면서 ONNX 잡이 죽었는데, 체크포인트를 켜 뒀는데도 복구가 일어나지 않았습니다. 하트비트 (heartbeat)는 심장 박동이라는 이름 그대로, "나 살아 있다"고 주기적으로 보내는 짧은 신호입니다. 태스크매니저가

잡매니저에게 일정 간격으로 보내고, 정해진 시간 안에 안 오면 잡매니저는 그 태스크매니저가 죽었다고 판정합니다. 실제로 죽지 않았어도 신호가 늦으면 죽은 것으로 취급됩니다 — 이 사고가 그 경우였습니다.

Caused by: java.util.concurrent.TimeoutException:
The heartbeat of TaskManager with id flink-taskmanager:43157 timed out.
Recovery is suppressed by NoRestartBackoffTimeStrategy

마지막 줄이 원인입니다. Flink의 기본 재시작 전략은 NoRestartBackoffTimeStrategy, 즉 "한 번 실패하면 그냥 죽는다"입니다. 체크포인트는 어디서부터 다시 시작할지를 알려 줄 뿐, 다시 시작하라고 명령하지는 않습니다. 복구는 체크포인트와 재시작 전략이 둘 다 있어야 성립합니다. 조치는 세 가지였습니다.

```
restart-strategy:
  type: exponential-delay
  exponential-delay:
    initial-backoff: 5s
    max-backoff: 2min
    backoff-multiplier: 2.0
    reset-backoff-threshold: 10min
    jitter-factor: 0.1
heartbeat:
  timeout: 120000          # 50s → 120s 완화
taskmanager:
  memory:
    process:
      size: 4608m          # 3072m → 4608m 상향
```

재시작 간격을 5초에서 시작해 2배씩 늘려 최대 2분까지 벌리는 지수 백오프입니다. 무한정 같은 간격으로 재시작하면 원인이 안 풀린 채 재시작 폭주가 일어나기 때문입니다. 10분간 안정적이면 백오프가 초기화됩니다. 하트비트 타임아웃을 늘린 것과 태스크매니저 메모리를 올린 것은 재시작이 아니라 애초에 죽지 않게 하는 조치입니다. 이 셋을 넣은 뒤 잡 4개가 안정적으로 유지됩니다.

5.4 백프레셔(backpressure) — 파이프라인이 스스로 속도를 줄이는 법

뒷사람이 못 따라올 때 앞사람이 천천히 하는 것

스트림 잡은 소스에서 싱크까지 연산자가 줄지어 있는 구조입니다. 그런데 중간의 어떤 연산자가 느려지면 어떻게 될까요. 추론 연산이 무거워졌다거나, 싱크로 쓰는 데이터베이스가 디스크 입출력으로 버벅인다거나, 태스크매니저 하나가 다른 컨테이너와 CPU를 다투게 된 경우입니다. 상류는 계속 밀어 넣는데 하류가 못 받아 냅니다.

여기서 두 갈래 길이 있습니다. 하나는 못 받은 것을 버리는 길입니다. 유실이 생기고, 공장 데이터에서는 허용되지 않습니다. 다른 하나는 못 받겠다고 위로 알리는 길입니다. 이것이 백프레셔입니다. 역압(逆壓)이라고 옮기기도 합니다. 하류가 "내 버퍼가 찼다"고 알리면 상류가 보내는 속도를 줄이고, 그 상류도 자기 상류에게 같은 신호를 보냅니다. 신호가 소스까지 거슬러 올라가면 소스가 Kafka에서 읽는 속도를 줄입니다. Flink는 이것을 크레딧 기반 흐름 제어(credit-based flow control)로 구현합니다 — 하류가 "내가 지금 몇 칸 받을 수 있다"는 크레딧을 상류에 주고, 상류는 크레딧만큼만 보냅니다.

여기서 결정적으로 중요한 사실. 백프레셔는 고장이 아닙니다. 유실을 지연으로 바꾸는 안전장치입니다. 데이터는 Kafka에 그대로 남아 있고, 잡은 조금 늦게 읽을 뿐입니다. 3장에서 Kafka를 버퍼로 둔 이유가 여기서 회수됩니다. 문제는 백프레셔가 계속 걸려 있을 때입니다. 처리 속도가 유입 속도를 영구히 못 따라가면 밀린 양이 무한정 쌓이고, Kafka 보존 기간(이 시스템은 텔레메트리 토픽이 24시간)을 넘기면 그때는 진짜 유실이 됩니다.

백프레셔는 이렇게 드러납니다

- ① Kafka 컨슈머 락(consumer lag)이 계속 증가합니다. 락은 "토픽에 쌓인 최신 오프셋 - 내가 읽은 오프셋", 즉 밀린 건수입니다. 가장 먼저 봐야 할 지표입니다.
- ② 체크포인트 소요 시간이 길어집니다. 배리어가 데이터 줄에 서서 내려가므로, 버퍼가 꽉 차 있으면 배리어도 같이 밀립니다. 체크포인트가 느려지거나 타임아웃으로 실패하기 시작하면 백프레셔를 의심해야 합니다.
- ③ Flink UI에서 연산자가 바쁨/백프레셔 비율로 붉게 표시됩니다. 여기서 백프레셔를 받고 있지 않으면서 바쁜 첫 번째 연산자가 진짜 병목입니다. 상류가 붉은 것은 하류 때문에 눌린 결과일 뿐입니다.

Flink의 체크포인트 실패·재시작 등 운영 지표는 Prometheus가 감시합니다. 정확한 지표 경로와 경보식은 3권 9장의 관제·운영 감시 절에 모았습니다.

5 분 안에 체크포인트가 3 번 이상 실패하거나 10 분 안에 잡이 4 번 이상 재시작하면 경보가 울립니다. Grafana 인프라 대시보드(grafana/dashboards/04-infra.json)는 flink_jobmanager_job_lastCheckpointDuration 패널에 "증가 추세는 상태 크기 증가 또는 백프레셔 를 시사합니다"라는 설명을 달아 두었습니다.

Flink 의 체크포인트 실패·재시작 등 운영 지표는 Prometheus 가 감시합니다. 정확한 지표 경로와 경보식은 3 권 9 장의 관제·운영 감시 절에 모았습니다.

그리고 이 시스템에서 백프레셔는 관측되지 않았습니다. 12 건/초라는 부하는 슬롯 8 칸짜리 Flink 에 게 너무 작습니다. sensor.telemetry.raw → sensor.telemetry.clean 이 40 초간 528 건 → 528 건으로 손실 0 이었습니다. 부하를 올려 보려면 .env 의 SCAN_INTERVAL_MS 를 1000 에서 100 으로 낮추면 됩니다(10-config-reference.md § 10.3). 다만 그 시험을 실제로 돌린 기록은 이 프로젝트에 없으므로, 이 스택이 몇 건/초까지 견디는지는 미검증입니다.

5.5 탐지 1 겹 — 엔지니어링 규격(USL/LSL)

주기 변경의 전제: .env 의 SCAN_INTERVAL_MS 만 바꾸면 시뮬레이터만 빨라집니다. EdgeX 경로의 수집도 바꾸려면 plant.yaml 을 맞추고 make regen-edgex 로 정의를 다시 생성해야 합니다. 원문 27·68 쪽의 조건을 함께 적용합니다. [원문 27, 68, 155 쪽]

가장 단순하고, 가장 먼저 올려야 하는 탐지

먼저 "겹"과 "계층"이라는 말을 정리하고 들어갑니다. 설계 보고서는 이상 탐지를 두 계층으로 설계합니다. 1 계층은 "결정론적 규칙과 국소 통계치를 활용하여 수 밀리초 이내의 초저지연 알람"을 내는 층이고 그 안에 세 항목이 들어갑니다 — 엔지니어링 규격 감지, 동적 롤링 Z-Score 감지, 복합 상태 규칙 (CEP). 2 계층이 머신러닝·딥러닝 기반 탐지입니다. 이 교본이 "겹"이라고 부르는 것은 그 1 계층 안을 설명 순서대로 다시 나눈 것입니다. 1 겹이 규격, 2 겹 이 Z-Score 이고, 순서를 보는 CEP 는 같은 1 계층에 함께 있으면서 6 장에서 따로 다룹니다. 그래서 알람 로그에 찍히는 탐지기 이름이 TIER1_RULE · TIER1_ZSCORE · TIER1_CEP 로 전부 TIER1 이고, 6 장의 오토인코더만 TIER2_ML 입니다. 코드가 쓰는 숫자는 설계 보고서의 계층 번호이고, 이 장의 "겹"은 설명 순서입니다. 둘이 어긋나 보인다면 그 때문입니다.

USL 은 Upper Specification Limit, 상한 규격치입니다. LSL 은 Lower Specification Limit, 하한 규격치입니다. 설비 설계자나 공정 엔지니어가 "이 범위를 벗어나면 안 된다"고 문서로 정해 놓은 값입니다. 반응기 설계 압력이 6 barg 면 6 barg 가 USL 이고, 원료탱크 레벨이 10% 아래로 내려가면 펌프가 캐비테이션을 일으키므로 10%가 LSL 입니다.

캐비테이션(cavitation)도 풀고 갑니다. 펌프가 액체를 빨아들이는 쪽의 압력이 너무 낮아지면 액체 안에서 기포가 생겼다가, 압력이 회복되는 자리에서 그 기포가 터집니다. 터질 때 아주 좁은 지점에 충격이 집중되어 금속 표면을 조금씩 깎아 냅니다. 탱크 레벨이 낮아지면 펌프 입구 압력이 떨어지므로 이 현상이 생기고, 소리와 진동이 커지면서 임펠러(날개)가 곰보처럼 패입니다. 그래서 레벨 하한은 "값이 예뻐 보이라고" 정한 숫자가 아니라 설비가 망가지기 시작하는 선입니다.

이 점이 중요합니다. USL·LSL 은 통계에서 나온 값이 아니라 물리와 설계와 법규에서 나온 값입니다. 데이터가 어떻게 생겼든 바뀌지 않습니다.

왜 이게 첫 겹인가. 세 가지 이유입니다. 첫째, 판정에 과거가 필요 없습니다. 값 하나만 보면 됩니다. 상태도, 창도, 워터마크도 필요 없으므로 지연이 사실상 0 입니다. 둘째, 설명이 필요 없습니다. "7.493 barg 인데 규격은 6.0 입니다"는 누구에게나 통합니다. 통계 모델이나 신경망의 판정과 달리 현장에서 논쟁이 생기지 않습니다. 셋째, 안전 관련 설비에서는 반드시 결정론적인 층이 있어야 합니다. 학습한 모델이 무슨 이유로든 안 울렸을 때 최종적으로 걸리는 그물이 이 층입니다.

구현 구조는 스트림 조인입니다. 센서값은 무한히 흐르는 스트림이지만 규격표는 거의 변하지 않는 작은 표입니다. 이런 표를 차원 테이블(dimension table)이라고 부르고, 흐르는 스트림에 붙여 값을 채우는 것을 조인이라고 합니다. 태그 이름으로 조인해서 그 태그의 USL/LSL 을 가져온 뒤 비교하면 끝 입니다. 창도 정렬도 없으므로 행이 도착하는 즉시 판정이 나갑니다.

AR-100 에서는. 규격표의 단일 출처는 simulator/plant.yaml 이고, 거기서 flink/sql/tag_limits.csv 가 생성됩니다(ADR-009). 실제 파일 내용입니다.

```

LT-101,%,10.0,95.0
LT-102,%,15.0,90.0
TT-101,degC,40.0,95.0
TT-102,degC,,125.0
PT-101,barg,,6.0
FT-101,m3/h,,
FT-102,m3/h,,
IT-101,A,,14.0
IT-102,A,,9.6
VT-101,mm/s,,7.1
pH-101,pH,6.2,8.4
CT-101,mS/cm,,

```

열은 태그·단위·LSL·USL 순서이고, 빈칸은 그쪽 규격이 없다는 뜻입니다. 압력 PT-101은 상한만 있고 하한이 없습니다. 유량 FT-101/FT-102와 전도도 CT-101은 규격 자체가 없어서 이 층에서는 아예 판정 대상이 아닙니다. 이 표는 01_sources.sql에서 파일시스템 커넥터로 읽힙니다.

```

CREATE TABLE tag_limits (
  tag STRING, unit STRING, lsl DOUBLE, usl DOUBLE,
  PRIMARY KEY (tag) NOT ENFORCED
) WITH ('connector'='filesystem', 'path'='file:///opt/flink/sql/tag_limits.csv',
       'format'='csv', 'csv.ignore-parse-errors'='true');

```

판정은 02_tier1_rules.sql 한 문장입니다.

```

INSERT INTO alerts
SELECT t.ts, t.site, t.device, t.tag, t.value,
       CASE WHEN l.usl IS NOT NULL AND t.value > l.usl THEN 'THRESHOLD_USL'
            ELSE 'THRESHOLD_LSL' END AS alert_type,
       'CRITICAL' AS severity,
       'TIER1_RULE' AS detector,
       CONCAT(...) AS detail
FROM telemetry_raw AS t
JOIN tag_limits AS l ON t.tag = l.tag
WHERE (l.usl IS NOT NULL AND t.value > l.usl)
      OR (l.lsl IS NOT NULL AND t.value < l.lsl);

```

IS NOT NULL 검사가 두 번 나오는 이유가 위의 빈칸입니다. 규격이 없는 태그를 NULL과 비교하면 조용히 아무 일도 일어나지 않으므로, 명시적으로 걸러 줍니다. spike 고장을 주입했을 때 실제로 나온 알람은 이렇습니다.

```
✓ spike → TIER1_RULE - PT-101 = 7.493 barg / 규격 [-, 6.0]
```

잡 이름은 SET 'pipeline.name' = 'AR100-Tier1-Threshold';로 파일마다 따로 붙입니다. 전역 설정 파일에 한 번만 쓰면 잡 4개 이름이 전부 같아져서 UI에서 구분이 안 됩니다.

경보와 안전 정지는 서로 다른 층

규격 판정은 tag_limits.csv에 따라 Flink가 경보를 만들고, 안전 인터록은 네트워크 밖 현장 시뮬레이터가 수행합니다. 압력 규격 USL은 6.0 barg, 인터록 코드는 ≥ 6.5 barg에서 트립하고 ≤ 5.2 barg에서 해제합니다. 지시값을 참조하는 이유와 히스테리시스 1 권 1장에 통합했습니다. 경보·통계·안전 정지를 같은 검출기로 생각하면 안 됩니다.

"이 센서치고는 이상하다"를 재는 자

규격은 모든 태그에 하나씩 고정되어 있습니다. 그런데 같은 70 °C라도 반응이 안정된 구간에서의 70 °C와 방금 원료를 넣은 구간에서의 70 °C는 의미가 다릅니다. 절대값이 아니라 평소 대비 얼마나 튀었는가를 재야 하는 것입니다. 그 자가 Z-Score, 우리말로 표준점수입니다.

수식은 이것 하나입니다.

$$z = \frac{|x - \mu|}{\sigma}$$

x는 지금 들어온 값, μ (뮤)는 최근 구간의 평균, σ (시그마)는 최근 구간의 표준편차입니다. 표준편차는 "값들이 평균에서 평균적으로 얼마나 떨어져 있는가"를 나타내는 수치, 즉 그 센서의 평소 흔들림 폭입니다. 따라서 z는 "지금 값이 평소 흔들림 폭의 몇 배만큼 벗어났는가"를 뜻합니다. z가 3이면 평소 흔들림의 3배만큼 튼 것입니다.

왜 하필 3인가. 여기에 정규분포가 나옵니다. 정규분포는 평균 둘레에 값이 가장 많이 모이고, 평균에서 멀어질수록 급격히 드물어지는 종 모양의 분포입니다. 같은 조건에서 반복해 잰 계측값은 대체로 이 모양을 따릅니다. 정규분포를

가정하면 평균에서 σ 의 3배 넘게 떨어진 값이 나올 확률은 대략 0.3%, 즉 1000 번에 3 번쯤입니다. 그래서 "우연으로 보기에는 드물다"는 선으로 3.0 이 관습적으로 쓰입니다. 다만 이것은 가정입니다. 공정값이 정규분포가 아니면 이 확률도 맞지 않습니다.

"롤링(rolling)"이 붙은 이유. μ 와 σ 를 공장 전체 역사에서 구하면 안 됩니다. 공정은 계절, 원료 로트, 설정치 변경에 따라 정상 수준 자체가 움직이기 때문입니다. 그래서 최근 N 건만 보는 창을 두고, 값이 들어올 때마다 그 창을 한 칸 밀면서 μ 와 σ 를 다시 계산합니다. 이것이 5.1 에서 본 슬라이딩 창이고, 이 창이 상태 연산인 이유입니다. 기준선이 스스로 따라 움직이기 때문에 규격표가 없는 태그에도 쓸 수 있고, 규격 한참 안쪽에서 일어나는 급변도 잡아냅니다.

그런데 이 방법에는 구조적인 약점이 있습니다. 그리고 이 시스템을 만들면서 그 약점에 정면으로 부딪혔습니다.

1 차 구현은 교과서 그대로였습니다 — 60 샘플 창, $z > 3.0$ 이면 알람. 고장을 하나도 주입하지 않은 정상 운전 상태에서 2 분간 8 건의 알람이 울렸습니다.

```
[TIER1_ZSCORE] TT-101 = 71.87803 :: TT-101 z=3.09 ( $\mu=72.319$ ,  $\sigma=0.1431$ )
[TIER1_ZSCORE] TT-101 = 71.6813 :: TT-101 z=3.02 ( $\mu=72.267$ ,  $\sigma=0.1941$ )
```

원인은 램프(ramp) 구간이었습니다. 이 시뮬레이터는 실제 플랜트처럼 autopilot 이 설정치를 주기적으로 완만하게 바꿉니다. 설정치가 움직이면 공정값이 한 방향으로 단조롭게 이동합니다. 이때 창 안 의 60 개 값은 전부 "조금씩 내려가는 중"이고, 그 평균 μ 는 구조적으로 현재값보다 뒤쳐집니다. 이동 평균은 언제나 지나간 값들의 무게중심이기 때문입니다. 첫 줄을 그대로 계산해 보면 이렇습니다.

```
 $\mu = 72.319$ , 현재값 = 71.878 → 차이 0.441 ← 분자
 $\sigma = 0.1431$  ← 분모
 $z = 0.441 / 0.1431 = 3.081$  → 3.0 초과 → 알람
```

여기서 σ 의 역할을 정확히 짚고 가야 합니다. 램프 때문에 창 안의 변동이 커져서 σ 가 부풀려진 것은 사실입니다. TT-101의 순수한 계측 잡음은 simulator/plant.yaml 의 noise: 절에 TT-101: 0.06 (단위 °C)으로 적혀 있는데, 실제 창에서 나온 σ 는 0.1431로 두 배가 넘습니다. 그러나 σ 는 분모입니다. 부풀려질수록 z 는 작아집니다. 즉 부품 σ 는 알람을 막는 쪽으로 작용했고, 그런데도 z 가 3을 넘었다는 것은 μ 가 뒤쳐진 정도(0.441)가 그보다 더 컸다는 뜻입니다. 오탐을 만든 원인은 μ 의 지연 하나뿐입니다. σ 는 오탐의 원인이 아니라 오탐을 조금 눌러 준 항입니다.

인용한 로그와 손계산이 미세하게 다른 이유도 적어 둡니다. 로그에 찍힌 z 는 3.09 인데 위에서 계산한 값은 3.081 입니다. SQL 의 알람 문구가 μ 를 소수 셋째 자리, σ 를 넷째 자리로 ROUND 해서 찍기 때문입니다(03_tier1_zscore.sql). z 는 반올림 전의 원래 값으로 계산되고, 문구에 실린 $\mu \cdot \sigma$ 는 사람이 읽기 좋게 자른 값입니다. 따라 계산해서 소수 둘째 자리가 어긋나면 그것이 이유입니다.

통계적으로는 "3 σ 이탈"이 맞습니다. 그런데 물리적으로는 아무 일도 일어나지 않았습니니다. 온도가 설정치를 따라 정상적으로 내려가고 있을 뿐입니다. 이것이 오탐(false positive, 거짓 양성)입니다. 그리고 오탐은 단순한 불편이 아닙니다. 정상 운전 중에 알람이 계속 뜨면 현장 운전원은 알람 자체를 무시하게 됩니다. 그 순간 이 탐지 층은 있으나 없으나 같아지고, 진짜 알람도 함께 묻힙니다.

해법은 세 축입니다.

(1) 임계치 상향 — z 기준을 3.0 에서 3.5 나 4.0 으로 올립니다. 쉽지만 진짜 이상에 대한 민감도도 같이 떨어집니다.

(2) 지속성/디바운스 — "몇 번 연속으로 위반했는가"를 함께 봅니다. 램프 때문에 생기는 위반은 한두 샘플 튀고 사라지지만, 실제 이상은 여러 샘플에 걸쳐 이어집니다. 산업 현장의 표준 대응입니다.

(3) 추세 제거(detrending) — 값에서 추세 성분을 먼저 빼고 남은 잔차에 Z-Score 를 겁니다. 잔차(residual)는 실제 값에서 모델이 설명한 부분을 빼고 남은 나머집니다. "온도가 분당 0.2 도씩 내려가는 중"이라는 추세를 먼저 빼면, 남는 것은 그 추세 위의 흔들림뿐이고 μ 가 뒤쳐지는 문제가 사라집니다. 원리적으로 가장 옳지만 SQL 로 표현하면 복잡도가 급증합니다.

이 시스템은 (1)과 (2)를 함께 썼습니다.

AR-100에서는. 최종 판정식은 flink/sql/03_tier1_zscore.sql 에 있고, 안쪽부터 네 겹의 중첩 질의입니다.

```
-- ① 태그별 60행 슬라이딩 창에서  $\mu$ ,  $\sigma$ , 표본 수 n
SELECT ts, site, device, tag, value, event_time,
       AVG(value) OVER w AS mu,
       STDDEV_SAMP(value) OVER w AS sd,
       COUNT(*) OVER w AS n
FROM telemetry_raw
WINDOW w AS (PARTITION BY tag ORDER BY event_time
             ROWS BETWEEN 59 PRECEDING AND CURRENT ROW)
-- ② 창이 덜 찼을 때의 불안정한 통계는 판정에서 제외
WHERE n >= 30
-- ③ z 계산.  $\sigma$  가 0 에 가까우면 0 으로 나누게 되므로 가드
CASE WHEN sd > 1e-9 THEN ABS(value - mu) / sd ELSE 0.0 END AS z
CASE WHEN z > 3.5 THEN 1 ELSE 0 END AS viol
-- ④ 지속성: 최근 5 샘플(직전 4 + 현재) 중 위반 횟수
SUM(viol) OVER (PARTITION BY tag ORDER BY event_time
               ROWS BETWEEN 4 PRECEDING AND CURRENT ROW) AS viol_run
...
WHERE viol_run >= 3
```

1차 구현에서 바뀐 것은 두 줄뿐입니다 — 임계치 3.0 → 3.5, 그리고 최근 5 샘플 중 3 회 이상 위반 이라는 지속성 조건. 결과는 정상 운전 90 초간 알람 0 건이고, 분산이 실제로 커지는 noise 시나리오의 탐지는 그대로 유지됩니다. 알람 문구에도 근거를 다 실어 보냅니다 — TT-101 $z=3.72$ ($\mu=72.104$, $\sigma=0.2013$, 최근 5 중 4 회 위반) 형태입니다.

대가는 자연입니다. 그리고 그 자연이 두 향으로 이루어집니다

① 지속성 대기 — 최근 5 샘플 중 3 회를 채워야 하므로 최소 3 샘플, 최대 5 샘플이 필요합니다. 1 초 주기이므로 3~5 초입니다.

② 워터마크 대기 — 위 절의 OVER 창이 ORDER BY event_time 으로 정렬되어 있습니다. 5.2 에서 본 대로 이벤트 시각으로 정렬한 결과는 워터마크가 그 행의 시각을 지나야 나갈 수 있고, 워터마크 는 event_time - INTERVAL '5' SECOND 입니다. 따라서 여기에 약 5 초가 더 붙습니다. 둘을 합치면 통계 이상 알람은 10 초 안팎에 나온다는 계산이 나옵니다. 반면 5.5 의 규격 위반은 창 도 정렬도 없이 조인 한 번이므로 도착 즉시 나갑니다. 규격 위반은 즉시, 통계 이상은 10 초 안팎 — 이 층 나눔이 그대로 자연 예산입니다.

다만 이 10 초는 계산값이지 실측값이 아닙니다. 이 프로젝트의 실측 기록(09-metrics.md §9.4)에 남아 있는 자연은 ONNX 추론 0.126~0.180 ms, EdgeX 스캔에서 Kafka 도달까지 1 초 이내, FUXA 제어에서 물리 반응까지 1 스캔 이내입니다. 알람이 실제로 몇 초 만에 뚫는지를 켜 기록은 없습니다. 설정에서 유도한 값으로 읽으십시오.

검토한 대안	내용	채택 여부와 이유
임계치만 상황	$z > 4$ 또는 5	부분 채택(3.5). 더 올리면 실제 이상 민감도가 함께 떨어짐
지속성 요건	최근 5 샘플 중 3 회 이상	채택. 램프의 순간 위반과 실제 이상을 구분하는 현장 표준
추세 제거 후 잔차에 Z-Score	이동추세를 빼고 잔차로 판정	기각. SQL 복잡도가 급증해 교육 자료로 부적합
윈도우 확대 60 → 300	평균을 더 길게	기각. 램프가 길면 같은 문제가 반복되고 반응 지연만 커짐

주의 — 반드시 함께 넣어야 하는 가드 두 개. 하나는 σ 가 0 에 가까울 때입니다. 값이 완전히 정지 한 센서(고장으로 같은 값만 내보내는 경우 포함)는 σ 가 0 이 되어 z 가 무한대가 됩니다. 그래서 $sd > 1e-9$ 검사를 담니다. 다른 하나는 창이 덜 찼을 때입니다. 잡을 막 띄운 직후에는 창에 값이 몇 개 없어서 μ 와 σ 가 심하게 흔들리고, 이때 나오는 z 는 아무 의미가 없습니다. $n \geq 30$ 이 그 방어입니다. 이 두 줄이 없으면 잡을 재시작할 때마다 알람이 쏟아집니다.

주의 — 이 층이 못 보는 것. Z-Score 는 태그를 하나씩 따로 봅니다. 여기서 두 가지가 빠져나갑니다.

① 태그들 사이의 관계가 어긋나는 고장. 이 시뮬레이터의 drift 시나리오가 그것입니다. 정상 운전에서 pH-101 과 CT-101 은 서로 반대로 움직이는데(실측 상관계수 -0.934), 이 고장은 둘을 동시에 같은 방향으로 밀어 상관 구조를 뒤집습니다. 그러면서 각 태그는 규격 [6.2, 8.4] 안에 얹전히 머무릅니다. 규격 규칙은 여기서 원리적으로 침묵합니다.

다만 "태그를 하나씩 보는 어떤 자로도 잡히지 않는다"고까지 말하면 지나칩니다. 원문 작성자의 PC 에서 직접 주입해 보니 바로 이 Z-Score 가 4 초 만에 반응했습니다(pH $z=4.13$ · CT $z=4.83$). 위의 60 초 창보다 드리프트 경사(45 초)가 빠르기 때문입니다. Z-Score 가 말해 주는 것은 "두 태그가 각자 평소와 다르다"까지이고, "같이 움직이면 안 되는 둘이 같이 움직였다"와 어느 태그가 얼마나 기여했는지는 다변량 모델만 말합니다. 그 차이가 이 층을 따로 두는 이유입니다. 여러 태그를 한꺼번에 보는 층이 필요하고, 그것이 6 장의 오토인코더입니다. 오토인코더는 정상 데이터를 압축했다가 다시 펼쳐 복원하도록 학습한 신경망이고, 원래 입력과 복원된 것의 차이를 재구성 오차(reconstruction error) 라고

부릅니다. 정상 패턴은 잘 복원되어 오차가 작고, 학습한 적 없는 어긋난 패턴은 복원이 안 되어 오차가 큼니다. 실측에서 drift 는 TIER2_ML 이 전담해 잡았습니다 — 재구성오차 0.03386 > 임계 0.02913 · 기여 상위: CT-101(52%), pH-101(30%) .

② 사건들 사이의 순서. 이것이 이 장 첫머리에서 이름만 말해 둔 CEP(복합 이벤트 처리)가 맡는 자리입니다. 설계 보고서가 든 예가 이렇습니다 — "모터 전류가 정격의 120%를 초과한 후 10 초 이내에 베어링 진동 센서의 진폭이 기준치를 상회하는 경우". 전류가 높은 것 자체도, 진동이 큰 것 자체도 각각으로는 그저 두 개의 임계치 초과입니다. 전류가 먼저 오르고 진동이 뒤따랐다는 순서가 베어링 열화를 가리킵니다. 순서를 뒤집으면 다른 이야기가 되고, 간격이 10 분이면 서로 무관한 일입니다. 값 하나만 보는 자로는 이런 질문을 표현할 수 없습니다. 이 시스템은 그것을 flink/sql/04_tier1_cep.sql 한 파일로 구현했고, bearing_wear 고장을 주입했을 때 실제로 이렇게 울렸습니다.

✓ bearing_wear → TIER1_CEP
교반기 전류 9.69A (정격 120% 초과) 후 7 초 내 진동 7.21mm/s 상회 → 베어링 열화 의심

"7 초 내"라는 숫자가 이 알람의 핵심입니다. 그것이 0 초로 찍히면 순서를 본 것이 아니라 두 조건이 동시에 참인 것을 본 것이고, 그럴 거면 CEP 가 필요 없습니다. 문법과 함정은 6 장에서 다룹니다.

5.7 결측치 보간 — 메울 것인가, 비워 둘 것인가

구멍을 메우는 네 가지 방법, 그리고 메우면 안 되는 자리

결측치(missing value)는 있어야 할 자리에 값이 없는 것입니다. 1 초마다 오던 TT-101 이 15 초 동안 안 왔다면 그 15 칸이 결측입니다. 설계 보고서는 원인을 넷으로 정리합니다 — 패킷 드롭, 센서 하드웨어 결함, 급격한 전압 강하, 센서 헤드 캘리브레이션 지연. 즉 통신 문제만이 아니라 계측기 자체에서도 생깁니다.

결측을 왜 메워야 하는가. 사람이 그래프를 보는 용도라면 사실 안 메워도 됩니다. 선이 끊긴 자리를 보고 "여기 데이터가 없었구나" 하면 그만입니다. 문제는 기계에게 먹일 때입니다. 5.1 에서 본 대로 신경망의 입력은 고정된 차원의 완전한 텐서여야 합니다. 12 개 태그 × 10 스텝 = 120 칸짜리 행렬에 한 칸이라도 비어 있으면 두 가지 중 하나가 벌어집니다.

① NaN (Not a Number, 숫자가 아님)이 들어가면 부동소수점 연산이 전부 오염됩니다. 부동소수점은 컴퓨터가 소수를 다루는 방식인데, 규칙상 NaN 이 한 번 섞이면 그것을 더하거나 곱한 결과도 전부 NaN 이 됩니다. 한 칸에서 시작해 출력 120 칸이 통째로 NaN 이 됩니다.

② 0 같은 임의의 값으로 때우면 모델이 그것을 거대한 이상으로 읽습니다. 70 °C 를 오가던 자리에 0 이 들어오면 그보다 더 큰 이상 신호가 없습니다. 아무 고장도 없는데 알람이 터집니다. 그래서 설계 보고서는 "머신러닝 모델을 파이프라인에 통합할 경우, 결측치 보간은 반드시 거쳐야 하는 필수 전처리 공정"이라고 못 박습니다.

어디서 메워야 하는가. 설계 보고서의 답은 분명합니다 — Kafka 와 시계열 데이터베이스 사이의 스트림 처리 계층입니다. 이유를 두 개 듭니다. 에지 게이트웨이는 "수백 개 센서의 다변량 상관 회귀나 슬라이딩 윈도우 상태를 유지하기에 메모리와 연산 능력이 제한적"이고, 반대로 시계열 데이터베이스에 적재한 뒤에 메우는 방식은 "수십 밀리초 단위의 중단 간 지연시간 요건을 충족할 수 없다"는 것입니다. 보간 은 앞뒤 값을 들고 있어야 하는 상태 연산이므로, 상태를 제대로 다룰 수 있으면서 아직 하루로 내려 보내기 전인 자리가 유일한 답입니다. 5.3 에서 상태 백엔드를 길게 다룬 이유가 여기서 회수됩니다.

메우는 방법은 네 가지입니다.

① LOCF (Last Observation Carried Forward, 직전 관측치 대체). 가장 단순합니다. 결측 직전의 마지막 유효값을 그대로 씁니다.

$\hat{x}(t) = x(t-k)$ ← $x(t-k)$ 는 결측 직전의 최신 유효 관측값

연산 복잡도가 $O(1)$ 이고 메모리는 값 하나면 됩니다. 결정적인 장점은 지연이 0 이라는 것입니다. 뒤를 기다릴 필요가 없으니 결측을 인지한 즉시 채울 수 있습니다. 단점은 시계열의 추세가 완전히 단절된다는 것입니다. 온도가 올라가는 중이었다면 LOCF 로 채운 구간은 수평선이 되고, 공백이 끝나는 순간 계단처럼 됩니다. 그 계단 자체가 이상 신호로 오인될 수 있습니다.

② 선형 보간 (Linear interpolation, 시간차 비례 보간). 공백의 앞값과 뒷값을 직선으로 잇고 그 위의 점을 씁니다.

$$\hat{x}(t) = x(a) + \frac{t - t(a)}{t(b) - t(a)} \times (x(b) - x(a))$$

 $x(a)$ = 공백 직전 값, $x(b)$ = 공백 직후 값

온도, 레벨처럼 연속적으로 변하는 물리량에 대해 물리적 타당성이 높습니다. 실제로 그렇게 변했을 가능성이 크기 때문입니다. 대신 결정적인 대가가 있습니다 — $x(b)$ 가 도착할 때까지 계산할 수 없습니다. 공백이 닫힐 때까지 기다려야 하므로 그만큼 지연이 생깁니다. 15 초짜리 공백이면 15 초 늦습니다.

③ 다항 곡선 적합 회귀(Polynomial/Spline). 국소 창 안의 점들에 n 차 다항식 곡선을 맞추고 그 곡선 위의 값을 씁니다. 기계 진동이나 회전체 속도처럼 주기적이고 곡선형인 패턴을 직선보다 훨씬 정확하게 보존합니다. 대신 연산 오버헤드가 커지고, 룬게 현상(Runge phenomenon)이라는 고전적인 위험이 있습니다. 차수를 높인 다항식은 창의 가장자리에서 격렬하게 진동하며 발산합니다. 가운데는 잘 맞는데 양 끝이 터무니없는 값으로 튀는 것입니다. 보간해 넣은 값이 오히려 가짜 이상을 만들어 냅니다.

④ 다변량 공간 회귀(Spatial regression, 상관 센서 회귀). 발상이 다릅니다. 같은 센서의 과거를 보는 대신, 그 센서와 상관관계가 있는 다른 센서들을 보고 값을 복원합니다. 유량과 펌프 전류가 강하게 연동되어 있다면(이 시스템의 실측 상관계수는 FT-101 ↔ IT-101 이 +0.989 입니다), 유량계가 죽어도 전류에서 유량을 역산할 수 있습니다. 한 센서가 장시간 완전히 끊겨도 복원이 가능한 유일한 방법입니다. 대가는 센서 간 상관관계 행렬을 계속 유지해야 한다는 비용이고, 치명적 약점은 상관 센서가 같이 끊기면 모델이 통째로 붕괴한다는 것입니다. 같은 통신선에 물린 센서들은 대개 함께 끊깁니다.

기법	핵심 원리	장점	단점·제약	배치 위치
LOCF	직전 유효값 유지	연산 $O(1)$, 메모리 극소, 지연 0	추세 단절, 공백 끝에서 계단	에지 게이트웨이, 스트림 엔진
선형 보간	앞뒤 값을 시간차 비례로 잇기	연속 물리량에 물리적 타당성 높음	뒷값이 올 때까지 버퍼링 지연	스트림 엔진(슬라이딩 윈도우)
다항 회귀	국소 창에 n 차 곡선 적합	주기·곡선형 패턴 정밀 보존	연산 오버헤드, 경계면 발산(룬게 현상)	중앙 스트림 엔진(Flink)
다변량 공간 회귀	상관 센서로 역산	장시간 단절에도 복원 가능	상관 행렬 유지 비용, 동시 단절 시 붕괴	중앙 스트림 엔진(상태 연산)

"보여줄 거면 메우지 마라"

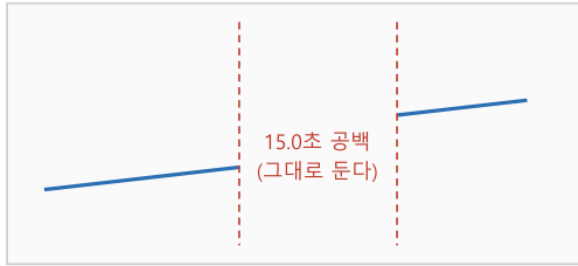
기법보다 중요한 것이 이 판단입니다. 설계 보고서의 문장을 그대로 옮기면 이렇습니다. 머신러닝 없이 단순 모니터링과 시각화만이 목적이라면 "데이터 파이프라인 내부에서 결측치를 인위적으로 보간할 당위성은 낮다." 이유는 두 가지입니다. 첫째, InfluxDB 나 TimescaleDB 같은 시계열 데이터베이스는 질의 시점에 LOCF 나 선형 보간으로 공백을 메우는 함수를 이미 제공합니다. 굳이 저장할 때 메울 필요가 없습니다. 둘째가 진짜 이유입니다 — "원천 데이터 레벨에서 인위적으로 계산된 가공치를 덮어쓰게 되면 실제 물리 계측치와 추정치 간의 경계가 모호해져 향후 사고 발생 시 감사 추적(Audit Trail)의 법적·기술적 신뢰성을 훼손할 수 있기 때문"입니다.

감사 추적(Audit Trail)은 "이 값이 어디서 어떻게 나왔는지를 나중에 되짚을 수 있게 남겨 둔 기록"입니다. 공장에서 사고가 나면 데이터는 증거가 됩니다. 조사관이 "사고 3분 전 압력이 얼마였습니까?"라고 물었을 때, 돌아온 값이 계측기가 실제로 읽은 값인지 파이프라인이 계산해 넣은 값인지 구분되지 않는다면 그 데이터는 증거로서 가치가 없습니다. 더 나쁜 경우도 있습니다. 통신이 끊긴 구간을 LOCF 로 메웠다면, 기록상으로는 그 시간 동안 압력이 아무 문제 없이 일정했던 것처럼 보입니다. 실제로는 아무것도 측정되지 않았는데 말입니다.

그래서 요구가 둘로 갈라집니다. 사람이 보고 사후 조사에 쓰는 기록은 실측과 추정이 구분되어야 하고, 모델에 먹이는 입력은 빈칸 없이 완전해야 합니다. 이 둘을 한 곳에 담으려 하면 반드시 하나가 망가집니다.

해법은 문서가 아니라 구조입니다. "보간값으로 원본을 덮어쓰지 말 것"이라고 규약 문서에 적어 두면 언젠가 누군가 어깁니다. 대신 토폴로지를 나눠 버리면 어길 수가 없습니다. 원본만 들어가는 토픽과 보간된 값이 들어가는 토픽을 물리적으로 분리하고, 보간값에는 quality 필드를 붙여 "이 값은 추정치다"를 값 자체가 지니고 다니게 합니다. 그러면 하류의 어떤 소비자도 실측과 추정을 혼동할 수 없고, 시계열 데이터베이스까지 그 구분이 그대로 살아서 사후에 quality 로 걸러 조회할 수 있습니다. 규약을 지키자고 부탁하는 대신, 어길 수 없는 구조를 만드는 것입니다.

sensor.telemetry.raw — 원본, 구멍 그대로



sensor.telemetry.clean — 메우고 표시



같은 15초 dropout 을 주입했을 때의 실측 결과. 두 토픽 모두 InfluxDB 에 적재되므로, 사후 조사에서 "이 값은 계측치인가 추정치인가"에 답할 수 있습니다.

원문 도해 · raw 공백과 clean 보간값의 구분 (원문 166 쪽)

품질 코드의 실제 구현

quality	의미·발생 위치
GOOD	Telegraf 를 통과한 실제 관측
INTERPOLATED_LINEAR	Flink Interpolator 가 앞뒤 관측으로 채운 기록용 값
INTERPOLATED_LOCF	직전 유효값을 유지한 추정값

STALE 는 원문 34 쪽이 “설계 문서에만 있고 생성 코드에는 없음”이라고 구분한 값입니다. 원문 259 쪽은 다시 네 번째 값으로 나열하지만, 이 편집본에서는 “설계 표기”와 “구현된 세 값”을 섞지 않습니다. 이는 원문 내부의 설명 차이이며 새로 코드를 검증한 결과는 아닙니다. [원문 34, 259 쪽]

AR-100 에서는. 먼저 quality 필드가 실제로 가지는 값은 세 가지입니다 — GOOD (실측), INTERPOLATED_LOCF , INTERPOLATED_LINEAR . Reading.java 의 주석에 적힌 것이 이 셋이고, 값을 만들어 내는 곳은 Interpolator.java 한 곳뿐입니다. ARCHITECTURE.md 177 행에는 여기에 STALE (오래된 값)이 하나 더 적혀 있지만 그 값을 쓰는 코드가 어디에도 없습니다. 설계 문서가 구현보다 앞서 나간 자리이므로, 실제 데이터에서 STALE 을 찾지 마십시오.

토픽은 sensor.telemetry.raw (무손실 원본, 보간 없음)와 sensor.telemetry.clean (중복 제거 + 보간 완료)으로 갈라져 있고 둘 다 InfluxDB 에 적재됩니다(ADR-004).

보간 자체는 Interpolator.java 한 클래스가 합니다. 태그로 키를 나눈 KeyedProcessFunction 이고, 상태는 RocksDB 에 들어갑니다. 결측 판정 기준이 이렇습니다.

```
long gapMs = in.millis() - prev.millis();
// 스캔 주기의 1.5 배를 넘으면 결측 구간으로 판정
if (gapMs > scanIntervalMs * 1.5) {
    emitFill(prev, in, gapMs, out);
}
```

스캔 주기가 1 초이므로 1.5 초를 넘으면 구멍으로 봅니다. 정확히 1 초를 기준으로 삼으면 5.2 에서 본 지터에 매번 걸리므로 1.5 배 여유를 둔 것입니다.

몇 개를 채울지는 emitFill() 의 첫 줄이 정합니다. 이 한 줄이 뒤의 계산 전부의 뿌리이므로 반드시 같이 읽어야 합니다.

```
int missing = (int) Math.round((double) gapMs / scanIntervalMs) - 1;
if (missing <= 0) {
    return;
}
```

공백을 스캔 주기로 나누고 1 을 뺍니다. 15 초 공백을 1 초 주기로 나누면 15 인데, 그 15 칸 중 마지막 하나는 공백이 끝나며 실제로 도착한 값이 이미 차지하고 있습니다. 그래서 메워야 할 빈칸은 14 개입니다. 뒤에 실측으로 나오는 "14 건"이 여기서 나온 숫자입니다.

채우는 쪽은 이렇습니다.

```

boolean useLinear = "linear".equalsIgnoreCase(mode) && gapMs <= maxGapMs;
String quality = useLinear ? "INTERPOLATED_LINEAR" : "INTERPOLATED_LOCF";
int cap = (int) Math.min(missing, maxGapMs / scanIntervalMs);
for (int i = 1; i <= cap; i++) {
    long tsNs = prev.ts + (long) i * scanIntervalMs * 1_000_000L;
    double v;
    if (useLinear) {
        double frac = (double) i / (missing + 1);
        v = prev.value + (next.value - prev.value) * frac; // 시간차 비례
    } else {
        v = prev.value; // LOCF
    }
    out.collect(new Reading(tsNs, prev.site, prev.device, prev.tag, v, quality));
}

```

frac의 분모가 missing + 1인 것에 주의하십시오. 빈칸이 14개면 앞값과 뒷값 사이를 15등분한다는 뜻입니다. 구간의 양 끝(앞값·뒷값)은 이미 실측값이므로 그 사이를 나누어야 합니다. cap은 폭주 방지 한도로, 뒤의 상자에서 다룹니다.

설정은 flink/onnx-job/job.properties에 있습니다(5.1에서 본 대로 이 파일은 이미지에 구워지므로 바꾸면 재빌드가 필요합니다).

```

interpolation.mode      = linear
interpolation.max-gap-ms = 20000
scan.interval.ms       = 1000

```

실측 결과입니다. 서로 다른 두 측정이므로 구분해서 읽어야 합니다.

① dropout 시험(짧은 구간). dropout은 이 시뮬레이터의 고장 시나리오 이름으로, TT-101이 15초 동안 값 발행을 멈추게 합니다(plant.yaml의 faults.dropout). 주입했더니 raw 토픽에는 15.0초 공백이 그대로 남았고(대조군 LT-102는 연속), clean 토픽에는 14건이 보간되어 들어왔습니다. 같은 시험에서 clean 토픽 660건을 세어 보면 GOOD 646건 + INTERPOLATED_LINEAR 14건입니다. 찍어 본 예시 다섯 개가 70.132 · 70.150 · 70.168 · 70.186 · 70.204로 0.018씩 일정하게 올라갑니다(05-tier3-ml.md § 5.2). 14건 전체가 70.204에서 끝난다는 뜻이 아니라, 간격이 일정하다는 것이 선형 보간이 직선을 그었다는 증거입니다.

② 장시간 누적(InfluxDB). 별도로 InfluxDB에 쌓인 것을 quality 별로 세면 GOOD 15,858건(99.74%), INTERPOLATED_LINEAR 42건(0.26%)입니다(09-metrics.md § 9.6). 앞의 660건과는 측정 구간이 다른 수치이므로 두 숫자를 이어 붙여 읽으면 안 됩니다.

AR-100·기록용은 선형, 추론용은 LOCF. 같은 시스템 안에서 보간 방식이 두 가지입니다. 이것이 ADR-005의 결정입니다.

기록용(clean 토픽 → InfluxDB)은 선형 보간입니다. 히스토리안에 남는 값은 사후 분석에 쓰이므로 물리적으로 타당해야 하고, 몇 초 늦게 적재되는 것은 문제가 되지 않습니다. 히스토리안(historian)은 공장에서 쓰는 이름으로, 공정 데이터를 장기간 원본 그대로 보관해 두는 이력 저장소를 뜻합니다. 이 시스템에서는 InfluxDB가 그 역할이고, 7장에서 따로 다룹니다.

추론용(ONNX 텐서 조립)은 LOCF입니다. OnnxScorer.java는 1초마다 타이머가 울릴 때 태그별 최신값 맵(MapState<String, Double> latest)에서 12칸을 채우는데, 그 시점에 값이 없는 칸은 마지막으로 본 값을 그대로 씁니다. 공백이 닫히기를 기다렸다가 추론하면 실시간 이상 탐지의 지연 요건이 무너지기 때문입니다. 한 번도 관측되지 않은 태그는 학습 시 평균값으로 대체하되, if (seen < tagOrder.length) return;으로 전 태그가 최소 한 번씩 관측되기 전에는 아예 추론하지 않습니다.

전 구간에 한 가지 정책만 쓰면 둘 중 하나는 반드시 망가집니다. 전부 선형으로 하면 추론이 늦어 못 쓰고, 전부 LOCF로 하면 히스토리안에 계단 모양의 가짜 기록이 남습니다.

폭주 방지 장치도 함께 있습니다. max-gap-ms = 20000, 즉 20초를 넘는 공백은 선형 보간을 포기하고 LOCF로 대체하며(useLinear 조건의 gapMs <= maxGapMs), 채우는 개수도 cap = min(missing, maxGapMs / scanIntervalMs)로 최대 20개까지만 잘라 냅니다. 통신이 10분간 끊겼다가 살아났을 때 599건의 가짜 값을 만들어 내는 사고를 막기 위한 것입니다. 장시간 단절은 보간할 일이 아니라보고할 일입니다.

주의 — 이 시스템은 PDF 권고와 반대 방향으로 갔습니다. 설계 보고서의 종합 결론 3은 이렇게 적혀 있습니다. "단순 시각화용 데이터는 무결성을 위해 결측 상태를 그대로 보존하되, 실시간 머신러닝 모델이 개입하는 분석 경로에 한하여

Flink 스트림 내부에서 선형 또는 회귀 보간을 선행시켜야 한다." 즉 설계 보고서는 ML 경로에 선형 보간을 지정합니다. AR-100은 정반대입니다 — 추론 경로가 LOCF 이고, 선형 보간은 기록 경로(clean → InfluxDB)에 있습니다.

왜 뒤집었는지는 PDF 안에서 답이 나옵니다. 같은 문서의 보간 기법 표는 선형 보간의 단점으로 "미래 시점 관측치가 도착할 때까지 스트림 버퍼링 지연 발생"을 명시합니다. 15 초 공백이면 15 초를 기다려야 합니다. 그것을 추론 경로에 두면 PDF 자신이 같은 문장에서 요구한 수십 밀리초 지연 요건이 무너집니다. 결론의 배치 권고와 표의 지연 특성이 서로 맞지 않는 자리이고, 이 시스템은 표 쪽을 따랐습니다.

PDF를 함께 읽는 분은 이 지점에서 두 문서가 갈린다는 것을 알고 계셔야 합니다. 이 교본은 어느 쪽이 절대적으로 옳다고 말하지 않습니다. 다만 "보간 방식은 기법의 우열이 아니라 그 경로가 지연을 얼마나 견딜 수 있는지로 정해진다"는 것이 두 문서에서 공통으로 얻을 수 있는 원칙입니다.

주의 — 메우면 데이터가 좋아진다는 착각. 보간을 배우는 사람이 가장 자주 빠지는 함정입니다. 보간은 없던 정보를 만들어 내지 않습니다. 없는 값을 있어 보이게 만들 뿐입니다. 보간 비율이 올라가면 데이터 품질은 좋아지는 것이 아니라 나빠지고 있는 것입니다. 그래서 quality 별 건수는 그 자체로 감시 지표입니다. 위 실측의 0.26%가 0.26%로 유지되는 동안은 정상이고, 이 값이 몇 퍼센트로 올라가면 보간 로직을 손볼 것이 아니라 왜 데이터가 안 오는지를 찾아야 합니다.

또 하나, 보간된 값을 모델 학습 데이터로 쓰면 안 됩니다. 모델이 사람이 만들어 넣은 직선을 정상 패턴으로 학습하게 됩니다. 학습 데이터에서는 보간 구간을 통째로 빼는 것이 원칙입니다.

주의 — RocksDB는 null을 거부합니다. 이 시스템 구축 중 실제로 잡이 죽은 사례입니다.

```
Caused by: TimerException{java.lang.IllegalArgumentException: The record must not be null.}
at org.uengine.iiot.OnnxScorer.onTimer(OnnxScorer.java:153)
```

슬라이딩 텐서 버퍼를 한 칸씩 왼쪽으로 밀 때, 버퍼가 아직 다 차기 전에 시프트를 하면 기록된 적 없는 칸을 읽어 MapState.put(key, null)이 됩니다. RocksDB 상태 백엔드는 null 값을 받지 않으므로 그 자리에서 예외가 납니다. 해결은 간단합니다 — 차기 전에는 뒤에 덧붙이기만 하고, 가득 찬 뒤에만 시프트합니다. 지금 OnnxScorer.java에 그 분기가 들어가 있고 주석도 달려 있습니다.

여기서 배울 것은 버그 자체가 아닙니다. 상태 백엔드를 힙으로 두고 개발하다가 운영에서 RocksDB로 바꾸면 이런 문제가 운영에서 처음 터집니다. 힙 백엔드는 null을 그냥 받아들이기 때문입니다(5.3의 비교표 참조). 상태 백엔드는 개발 때부터 운영과 같은 것을 써야 합니다.

보간 위치에 대한 설계 논거

설계 보고서는 에지에서 다수 센서의 창·상관 상태를 유지하는 자원 부담과, 데이터베이스 적재 후 처리할 때의 실시간 지연을 함께 고려해 스트림 계층을 보간 위치로 제시합니다. 원본 이력은 보존하고, 빈틈 없는 입력이 필요한 경로에만 보간을 적용한다는 구분입니다. 또한 설계 보고서의 선형 보간 권고와 실제 구현의 "기록은 선형, 추론은 LOCF"는 동일하지 않습니다. 실제 선택의 이유는 미래 관측을 기다릴 수 없는 추론 지연이며 원문 ADR-005가 이를 설명합니다.

같은 자리의 다른 도구들 Esper / EQL — SQL과 비슷한 이벤트 질의 언어(EQL)로 시간 윈도우와 패턴 매칭("A 이벤트 발생 후 3분 이내에 B가 발생하지 않는 상황")을 직관적으로 선언하는 고성능 인메모리 CEP 엔진입니다. 표현력 자체는 지금도 훌륭합니다. 본질적 차이는 실행 모델에 있습니다 — Esper는 단일 프로세스 JVM 안에 임베디드 라이브러리로 내장되도록 설계되었습니다. 그래서 초당 수십만 건이 물리거나 고가용성 클러스터가 필요한 환경에서 노드 장애 시 상태 복구와 수평 확장에 치명적 한계를 드러냅니다. 기억(상태)이 한 프로세스 안에만 있으니 그 프로세스가 죽으면 기억도 죽습니다. 설계 보고서의 결론 2가 "레거시 단일 JVM 엔진인 Esper/EQL 대신 Flink를 스트림 처리의 핵심 코어로 채택해야 한다"인 이유가 이것입니다. ksqlDB — Kafka 위에서 스트림 조인·필터링·윈도우 집계를 순수 SQL 문법으로 처리합니다. 도입 장벽이 매우 낮아서, 단순한 윈도우 연산만 필요하다면 PDF도 대안으로 인정합니다. 본질적 차이는 다루는 것의 범위입니다 — 다중 이벤트 간의 정합적 순서 패턴을 감지하는 순수 CEP 작업이나 커스텀 텐서 조작에 구조적 한계가 있습니다. 이 시스템은 같은 잡 안에서 결측을 메우고 12×10 텐서를 조립해 신경망에 밀어 넣어야 하므로, SQL만으로 표현되는 영역을 벗어납니다. Apache StreamPipes — 산업용 IoT 스트림 분석을 노코드(No-code)로 만드는 플랫폼입니다. 센서 어댑터, 윈도우 변환 필터, 이상 탐지 알고리즘을 웹 에디터에서 그래픽 블록으로 조립합니다. 현장 도메인 엔지니어의 접근성이 가장 높다는 것이 분명한 강점입니다. 본질적 차이는 고성능 분산 분기 연산이나 복잡한 딥러닝 텐서 처리에 제약이 있다는 점입니다. 블록으로 표현되지 않는 것은 만들 수 없습니다. Kafka Streams — 별도 클러스터 없이 애플리케이션에 라이브러리로 엮는 방식입니다. 운영이 가볍지만 자원 관리를 직접 해야 하고, Flink 수준의 창·윈도우·패턴 매칭 표현력에는 미치지 못합니다. Spark Structured Streaming — 배치 엔진에서 출발해 마이크로 배치로 스트림을 처리합니다. 배치와 스트림 코드를 통일하기 좋지만, 처리 단위가 작은 배치이므로 지연의 하한이 Flink보다 높습니다. 수 밀리초 알람이 목표라면 맞지 않습니다. eKuiper — 에지에서 도는 초경량 규칙

엔진입니다. 설계 보고서가 EdgeX의 내부 분석 수단으로 언급하는 것이기도 합니다. 자리가 아예 다릅니다 — 게이트웨이에서 1차로 거르고 중계하는 데 적합하고, 중앙에서 대규모 상태 연산을 하는 용도가 아닙니다. TSDB 질의 시점 보관 — 보관을 스트림에서 하지 않고 InfluxDB나 TimescaleDB의 질의 함수에 맡기는 선택지입니다. 시각화만 할 것이라면 이쪽이 오히려 좋습니다. 원본을 건드리지 않으니 감사 추적이 자동으로 보존됩니다. 다만 실시간 ML 경로의 수십 밀리초 요건은 충족할 수 없습니다.

그래서 결론은 이렇습니다. Flink는 Flink CEP 라이브러리와 SQL의 MATCH_RECOGNIZE 구문으로 비결정적 유한 오토마타(NFA) 기반 패턴 매칭을 Esper의 EQL 수준으로 제공하면서, 그것을 분산 환경에서 합니다.

용어 둘을 여기서 풀어 둡니다. MATCH_RECOGNIZE는 표준 SQL에 들어 있는 행 패턴 인식 구문입니다. 정렬된 행들을 정규식처럼 훑어 "A 다음에 B" 같은 순서 패턴을 찾아냅니다. 그리고 그 밑에서 도는 NFA(비결정적 유한 오토마타)는 상태와 전이로 이루어진 기계인데, 한 입력에 대해 갈 수 있는 다음 상태가 여러 개일 수 있어서 후보 경로를 여러 개 동시에 들고 진행하는 구조입니다. 패턴 매칭에서는 "A는 봤고 B를 기다리는 중"인 후보가 동시에 여러 개 살아 있을 수 있으므로 이 구조가 필요합니다. 그리고 그 후보들이 바로 5.3에서 말한 상태입니다 — Flink에서는 이것이 RocksDB에 들어가고 체크포인트로 복구됩니다. Esper에서는 프로세스 메모리 안에만 있어서 프로세스와 함께 사라집니다.

표현력은 Esper 급, 확장성과 내결함성은 분산 시스템급 — 이 둘이 한 엔진에 있다는 것이 Flink가 이 자리를 차지한 이유이고, 그래서 Esper를 따로 두어 파이프라인 복잡도를 가중시킬 필요가 없습니다.

비교 항목	Esper / EQL	ksqlDB	Apache StreamPipes	Apache Flink (이 시스템)
실행 모델	단일 JVM 임베디드	Kafka Streams 기반 분산	독립 클러스터	분산 클러스터(슬롯 단위 확장)
순서 패턴 매칭 (CEP)	EQL — 표현력 높음	구조적 한계	사전 정의 블록	MATCH_RECOGNIZE / Flink CEP(NFA)
상태 복구	노드 장애 시 한계	Kafka 기반 복구	제한적	RocksDB + 체크포인트
ML 텐서 처리	해당 없음	구조적 한계	딥러닝 텐서 처리 제약	임베디드 ONNX 직접 호출
진입 장벽	중간	낮음	가장 낮음(노코드)	높음

5.8 이 장의 용어 빠른 확인

6 CEP — 사건의 순서를 읽기

원문 교본 174-197 쪽의 설명을 통합

CEP 는 각각의 값이 한계를 넘었는지에 더해 “무엇이 먼저, 얼마 뒤에 무엇이 일어났는가”를 판정합니다. AR-100 은 교반기 전류 상승 뒤 진동 상승을 10 초 패턴으로 묶습니다. 이 경보가 기존 개별 임계치 경보를 대체하거나 억제하는 것은 아닙니다.

단일 임계치가 놓치는 것

먼저 무엇이 문제였는지부터 봐야 합니다. 5 장까지 만든 탐지기는 두 종류였습니다. 하나는 엔지니어링 임계치입니다. 공정 설계자가 “이 센서 값은 여기서 여기 사이에 있어야 한다”고 미리 정해 둔 규격 이고, 위쪽 한계를 상한 규격치(Upper Specification Limit, USL), 아래쪽 한계를 하한 규격치(Lower Specification Limit, LSL)라고 부릅니다. 값이 이 범위를 벗어나면 즉시 알람을 냅니다. 다른 하나는 롤링 Z-Score 로, 최근 60 개 샘플의 평균과 표준편차를 계산해 현재값이 통계적으로 얼마나 튀었는지를 봅니다.

두 방식 모두 공통점이 있습니다. 한 순간, 한 태그만 봅니다. 판정에 쓰는 정보가 “이 센서의 이 값” 하나뿐입니다. 그래서 다음 같은 상황을 표현할 방법이 아예 없습니다.

교반기(M-101)의 베어링이 마모되기 시작했다고 합시다. 베어링이 뻑뻑해지면 모터가 같은 회전수를 유지하려고 힘을 더 씁니다. 그래서 전류가 먼저 올라갑니다. 마모가 더 진행되어 회전축에 유격이 생기면 그제서야 진동이 올라갑니다. 전류 상승과 진동 상승 사이에는 물리적인 시차가 있고, 그 시차가 짧다는 것 — 수 초 안에 뒤따랐다는 것 — 자체가 “같은 원인에서 나온 두 증상”이라는 증거입니다.

설계 보고서는 이 상황을 규칙 한 문장으로 못 박아 놓았습니다. 이 장이 구현하는 대상이 정확히 이 한 문장입니다(PDF p. 8, 1 계층 탐지의 세 번째 항목).

“복합 상태 규칙(CEP Pattern): '모터 전류가 정격의 120%를 초과한 후 10 초 이내에 베어링 진동 센서 의 진폭이 기준치를 상회하는 경우'와 같이 여러 센서 간의 시간적 종속 관계를 ksqldb 나 Flink CEP 를 통해 인메모리로 실시간 추적한다.”

여기 나오는 인메모리(in-memory)는 “판정에 필요한 중간 정보를 디스크나 데이터베이스가 아니라 메모리(RAM) 안에 들고 있으면서 처리한다”는 뜻입니다. 디스크에 썼다 읽었다 하면 초 단위 지연이 생기므로, 실시간 탐지는 메모리에서 합니다.

이 한 문장을 단일 임계치로 감시하면 어떻게 되는지 보겠습니다. 아래 표는 AR-100 의 실제 설정에서 나오는 결과입니다.

감시 방식	전류 9.7 A 관측	몇 초 뒤 진동 7.2 mm/s 관측	sensor.alerts 에 실제로 나가는 것
임계치 규칙 (TIER1_RULE)	THRESHOLD_USL 알람 — 초과하는 동안 매초	THRESHOLD_USL 알람 — 초과하는 동안 매초	IT-102 = 9.7 A / 규격 [-, 9.6] 같은 문장이 두 태그에서 반복해서
CEP 패턴 (TIER1_CEP)	부분 매치 보관 — 출력 없음	패턴 완성 → 알람 1 건	교반기 전류 9.69A ... 후 7 초 내 진동 7.21mm/s 상회 → 베어링 열화 의심

주의 — CEP 는 알람을 줄여 주지 않습니다. 흔한 오해가 “CEP 를 넣으면 알람 두 개가 진단 하나로 합쳐진다”는 것입니다. AR-100 에서는 그렇지 않습니다. flink/sql/tag_limits.csv 에 IT-102 의 USL 이 9.6, VT-101 의 USL 이 7.1 로 들어 있고, flink/sql/02_tier1_rules.sql 이 모든 태그를 이 표와 조인해 감시하므로, bearing_wear 고장을 넣으면 같은 구간에서 두 태그 의 THRESHOLD_USL 알람이 매초 함께 나갑니다. CEP 알람은 그 위에 한 건이 더 붙는 것입니다.

그러면 CEP 가 더하는 값은 무엇인가. 원인의 형태를 말해 주는 알람 하나입니다. THRESHOLD_USL 두 종류는 “이 값이 규격을 넘었다”만 말합니다. 왜 넘었는지, 둘이 관계가 있는지는 말하지 않습니다. CEP_BEARING 한 건만이 “전류가 먼저, 진동이 나중, 간격 7 초 → 베어링”이라고 말합니다. 운전원이 정비 지시를 내리는 데 쓰는 것은 이 한 건입니다.

알람 홍수 자체를 줄이는 것은 CEP 의 일이 아니라 알람 억제·집계 설계의 일이며, 이 프로젝트 는 거기까지는 하지 않았습니다.

그리고 결정적으로, WHERE IT-102 > 9.6 AND VT-101 > 7.1 같은 단순 논리곱(AND)으로는 이 일을 할 수 없습니다. AND 는 “동시에 참”만 표현하지 “먼저-나중”을 표현하지 못하기 때문입니다. 게다가 두 값은 서로 다른 행(row)으로 도착합니다. 한 행에는 IT-102 값만, 다른 행에는 VT-101 값만 들어 있어서 한 행 안에서 AND 를 걸 대상 자체가 없습니다.

복합 이벤트 처리(Complex Event Processing, CEP)는 바로 이 빈자리를 채우는 기술입니다. PDF 는 이렇게 정의합니다.

"센서 데이터 스트림 내에서 단순 수치 비교를 넘어, 특정 이벤트의 발생 순서나 시간적 상관관계를 규명하기 위해 복합 이벤트 처리(Complex Event Processing, CEP) 엔진의 도입이 검토된다."

여기서 "복합(Complex)"이라는 말은 "어렵다"는 뜻이 아닙니다. 여러 개의 낱개 이벤트를 조합해 하나의 상위 이벤트를 만들어낸다는 뜻입니다. 낱개 이벤트(전류 측정값 하나, 진동 측정값 하나)를 단순 이벤트(simple event)라 부르고, 그것들이 특정 패턴으로 배열됐을 때 엔진이 새로 만들어 내보내는 것을 복합 이벤트(complex event)라 부릅니다. AR-100 에서 CEP_BEARING 이라는 알람이 바로 이 복합 이벤트입니다. 센서가 보낸 적 없는, 엔진이 만들어낸 이벤트입니다.

NFA, 부분 매치를 들고 기다리는 기계

CEP 엔진이 순서를 어떻게 알아보는지가 이 장의 핵심입니다. 답은 오토마타(automaton)입니다. 우리말로로는 자동자·유한상태기계라고 옮기는데, 실물은 아주 단순한 개념입니다.

유한 오토마타(Finite Automaton)는 상태(state) 몇 개와, 상태 사이를 옮겨 다니는 전이(transition) 규칙으로 이루어진 기계입니다. 입력을 하나씩 먹으면서 현재 상태를 바꿔 나가다가, 정해진 수용 상태(accepting state)에 도달하면 "패턴을 찾았다"고 선언합니다. 자동문을 생각하면 됩니다. 상태는 닫힘과 열림 두 개, 전이는 "사람이 감지되면 닫힘→열림", "5 초간 아무도 없으면 열림→닫힘". 자동문은 지금까지 몇 명이 지나갔는지 기억하지 않습니다. 오직 현재 상태 하나만 들고 있습니다. 이것이 유한 오토마타입니다.

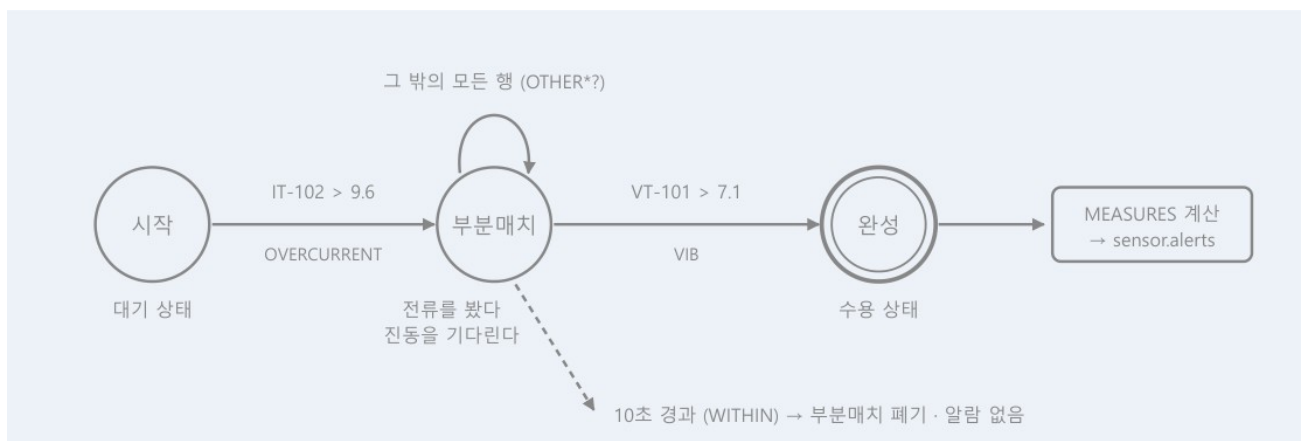
문제는 비결정성(non-determinism)입니다. 이름이 어려워 보이지만 뜻은 간단합니다.

결정적(Deterministic) 오토마타, DFA — 현재 상태에서 어떤 입력을 받았을 때 갈 수 있는 다음 상태가 딱 하나로 정해집니다. 갈림길이 없습니다. 비결정적(Non-deterministic) 오토마타, NFA — 같은 입력에 대해 갈 수 있는 길이 여럿일 수 있습니다. 기계는 어느 길이 맞는지 지금은 모르므로, 가능한 길을 전부 동시에 붙들고 진행합니다.

우리 패턴이 왜 비결정적인지 보겠습니다. "전류 초과 후 10 초 내 진동 초과"를 감시하는 중에 IT-102 = 9.8 이 들어왔습니다. 기계는 여기서 갈림길을 만납니다. 이 행을 패턴의 시작점으로 삼는 갈래와, 이 행을 시작점으로 쓰지 않고 지나가는 갈래가 동시에 열립니다. 어느 쪽이 맞는지 지금 알 수 없습니다. 이 행으로 시작한 시도는 10 초 안에 진동이 안 오면 실패로 끝날 수도 있고, 다음 초에 들어올 IT-102 행으로 시작한 시도가 성공할 수도 있습니다. 어느 시작점이 결국 완성될지는 미래를 봐야

주의 — 갈래를 살려 두는 이유는 "더 큰 값"을 기다려서가 아닙니다. DEFINE 절의 조건은 9.6 을 넘었는가 아닌가만 봅니다. 9.8 인지 10.5 인지는 비교하지 않고, 어느 쪽이 "더 나은 시작 점"이라는 개념 자체가 없습니다. 게다가 AR-100 이 쓰는 AFTER MATCH SKIP PAST LAST ROW 아래에서는 먼저 완성되는 매치가 채택되므로, 나중에 더 큰 값이 와도 앞선 시작점을 밀어내지 못합니다. NFA 가 여러 갈래를 드는 이유는 오직 어느 시작점이 결국 완성될지 모르기 때문입니다.

그 결과 NFA 는 부분 매치(partial match)라는 것을 메모리에 들고 다니게 됩니다. 부분 매치란 "여기 까지는 맞았고 나머지를 기다리는 중인 진행형 시도"입니다. 베어링 패턴에서는 "t=20 초에 전류 9.69 A 를 봤다. 이제 t=30 초까지 진동 7.1 초과를 기다린다"가 하나의 부분 매치입니다.



원문 도해 · CEP 부분 매치 상태와 완료 (원문 178 쪽)

왜 이 구조가 중요한가. CEP 엔진의 성능과 안정성은 전부 이 부분 매치 관리에 달려 있습니다. AR-100 에는 태그가 12 개이고 각 태그가 1 초에 한 번 값을 보냅니다(초당 12 건). 전류 초과가 계속되는 동안에는 초마다 새 부분 매치가

태어납니다. 이것을 정리하지 않으면 메모리가 무한히 늘어납니다. 그래서 CEP 문법에는 반드시 시간 제한이 들어갑니다. 10 초가 지난 부분 매치는 실패로 확정하고 버립니다. 이것이 뒤에 나올 WITHIN INTERVAL '10' SECOND 의 진짜 역할입니다 — 의미를 정하는 동시에 메모리를 유한하게 묶는 장치입니다.

초보가 반드시 헷갈리는 지점. "NFA 가 여러 길을 동시에 간다"는 말을 듣고 스레드(thread)를 여러 개 띄운다고 오해하는 경우가 많습니다. 스레드란 한 프로그램 안에서 동시에 돌아가는 실행 흐름의 단위인데, CEP 는 그런 식으로 동작하지 않습니다. 실제 구현은 부분 매치 목록을 하나 들고 있다가 새 행이 올 때마다 목록 전체를 한 번씩 훑는 방식입니다. 병렬 실행이 아니라 상태 집합의 갱신입니다. 그래서 CEP 의 비용은 "부분 매치 개수 × 유입 행 수"에 비례하고, 패턴이 험거울수록(시간 창이 길수록, 시작 조건이 혼잡수록) 급격히 비싸집니다.

"전류가 먼저"는 어떻게 보장되는가

이 절이 없으면 앞의 설명은 공중에 뜩니다. CEP 가 순서를 읽으려면 행들이 올바른 순서로 줄을 서 있어야 하는데, 분산 시스템에서 그것은 저절로 되지 않습니다. AR-100 이 이 문제를 어떻게 푸는지 보겠습니다. 먼저 용어 몇 개를 풀어둡니다.

소스와 싱크 Flink SQL 에서 소스(source)는 데이터가 들어오는 입구, 싱크(sink)는 데이터가 나가는 출구입니다. 둘 다 CREATE TABLE 로 선언하며, 겉모습은 똑같은 테이블이지만 SELECT ... FROM 의 대상이 되면 소스, INSERT INTO 의 대상이 되면 싱크로 쓰입니다. AR-100 의 flink/sql/01_sources.sql 에는 소스 두 개(Kafka 의 telemetry_raw , CSV 파일인 tag_limits)와 싱크 하나(Kafka 의 alerts)가 선언돼 있습니다.

토픽·파티션·오프셋·컨슈머 그룹 Kafka 용어입니다. 토픽(topic)은 메시지를 담는 이름 붙은 통이고, 토픽은 다시 파티션(partition) 여러 개로 쪼개져 있습니다. 파티션은 줄 세워진 로그입니다 — 메시지가 들어온 순서대로 번호가 붙어 쌓이고, 그 번호를 오프셋(offset)이라고 부릅니다. 0 번, 1 번, 2 번... 식입니다.

중요한 성질이 여기 있습니다. 순서는 파티션 안에서만 보장됩니다. 서로 다른 파티션에 들어간 두 메시지 사이에는 "누가 먼저인가"가 Kafka 차원에서 정의되지 않습니다.

읽는 쪽에서는 컨슈머 그룹(consumer group)이라는 이름표를 붙여 읽습니다. 같은 이름표를 단 여러 프로세스가 파티션을 나눠 갖고, "이 그룹은 이 파티션을 몇 번 오프셋까지 읽었다"는 기록도 그룹 단위로 남습니다. AR-100 의 CEP 를 포함한 SQL 잡들은 'properties.group.id' = 'flink-tier1' 이라는 그룹으로 읽습니다.

그리고 'scan.startup.mode' = 'latest-offset' 은 "잡을 제출한 그 순간 이후에 들어온 메시지부터 읽는다"는 뜻입니다. 반대말은 earliest-offset (토픽에 남아 있는 가장 오래된 메시지부터 전부 다시 읽기)입니다. 실무적 의미는 이렇습니다 — 잡을 다시 제출하면 과거 데이터는 다시 보지 않습니다. 실습 중에 "잡은 분명히 떠 있는데 알람이 안 나온다"면, 잡을 띄우기 전에 고장을 넣은 것은 아닌지 먼저 의심해야 합니다.

파티션 키가 tag 라는 사실 여기가 이 장에서 가장 자주 빠지는 대목입니다. AR-100 은 Telegraf 가 MQTT 에서 받은 데이터를 Kafka 로 넣는데, telegraf/bridge-edgex.conf 에 이렇게 적혀 있습니다.

```
routing_tag = "tag"          # 파티션 키 = 태그 → 태그 단위 순서 보장
```

파티션 키(partition key)는 "이 메시지를 몇 번 파티션에 넣을지" 정하는 값입니다. 키가 같으면 반드시 같은 파티션으로 갑니다. 여기서는 키가 태그 이름이므로 IT-102 는 언제나 한 파티션으로, VT-101 도 언제나 (다른) 한 파티션으로 들어갑니다. kafka/create-topics.sh 10 행의 주석도 같은 말을 합니다.

토픽	파티션	파티션 키
sensor.telemetry.raw	6	tag

그러면 이런 결론이 나옵니다. IT-102 와 VT-101 은 서로 다른 파티션으로 들어옵니다. 그리고 파티션 사이에는 순서 보장이 없습니다. 그러니 "전류가 먼저 왔다"는 것은 Kafka 가 보장해 주는 것이 아닙니다. Flink 가 이벤트 시간과 워터마크로 직접 세워야 합니다.

주의 — 두 개의 "파티션"은 다른 것입니다. Kafka 의 파티션 키는 tag , MATCH_RECOGNIZE 의 PARTITION BY 컬럼은 device 입니다. 이름이 같지만 층이 다릅니다. Kafka 파티션은 메시지를 어느 로그에 담을지, PARTITION BY 는 패턴을 어느 단위로 따로 찾을지를 정합니다. 두 태그를 한 패턴으로 묶어야 하므로 PARTITION BY tag 로 쓰면 안 되고(그러면 전류와 진동이 서로 다른 갈래로 갈라져 영역 만나지 못합니다), 두 태그가 공유하는 설비 식별자인 device 를 씁니다.

이벤트 시간과 워터마크 순서를 읽는 기술이므로 무엇을 순서로 볼지를 반드시 정해야 합니다. 두 가지 후보가 있습니다.

처리 시간(processing time) — Flink 서버가 그 행을 받은 시각. 네트워크가 막히거나 파티션마다 속도가 다르면 뒤바뀝니다. 이벤트 시간(event time) — 센서가 그 값을 측정할 시각. 데이터 안에 실려 옵니다.

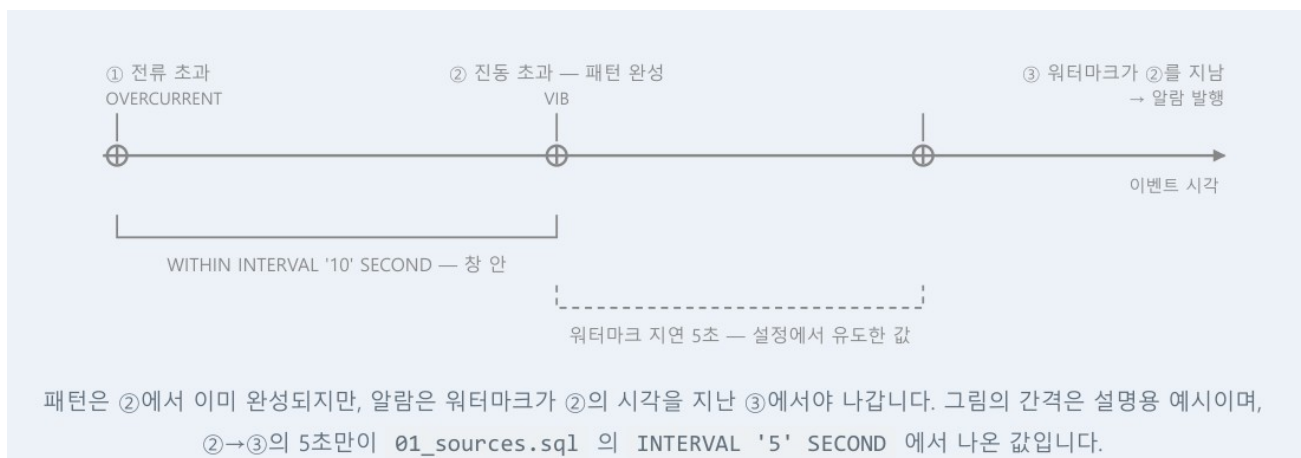
"전류가 먼저 올랐다"는 것은 측정 시점 기준의 이야기이지 서버 도착 순서의 이야기가 아닙니다. 그래서 반드시 이벤트 시간을 써야 합니다. `flink/sql/01_sources.sql` 의 정의입니다.

```
event_time AS TO_TIMESTAMP_LTZ(ts / 1000000, 3),
-- 센서 클럭 지터와 EdgeX 스캔 편차를 흡수
WATERMARK FOR event_time AS event_time - INTERVAL '5' SECOND
```

ts 는 나노초 단위 타임스탬프라서 1,000,000 으로 나눠 밀리초로 바꿉니다. 그리고 워터마크 (watermark)가 붙습니다. 워터마크란 "이 시각 이전의 데이터는 이제 다 왔다고 간주하겠다"는 선언입니다. 여기서는 최신 이벤트 시각보다 5 초 뒤쳐진 값으로 잡았습니다.

왜 5 초인가. 주석이 말하는 지터(jitter)는 "주기적으로 일어나야 할 일의 시각이 매번 조금씩 흔들리는 정도"입니다. 센서가 정확히 1.000 초마다 값을 내지 않고 0.98 초, 1.03 초처럼 흔들리는 것이 클럭 지터 이고, EdgeX 가 장치를 훑는 스캔 시점도 비슷하게 흔들립니다. 이런 흔들림 때문에 이벤트 시각이 살짝 뒤바뀐 채 도착할 수 있으므로, 판정을 5 초 미뤄서 늦게 온 행까지 제자리에 끼워 넣습니다.

워터마크가 없으면 Flink 는 언제 순서 판정을 확정해야 할지 알 수 없어 영원히 기다립니다. 반대로 워터마크를 너무 짧게 잡으면 늦게 도착한 행이 버려집니다. 워터마크 지연은 정확성과 지연시간의 맞바꿈이고, 이 시스템에서는 그 대가가 구체적인 숫자로 나타납니다.



원문 도해 · 패턴 완성과 워터마크에 따른 발행 시점 (원문 180 쪽)

알람은 진동 초과 후 최소 5 초 뒤에 나옵니다. 이유는 이렇습니다. 이벤트 시간 기준으로 동작하는 MATCH_RECOGNIZE 는 행이 도착하는 즉시 처리하지 않고, 워터마크가 그 행의 시각을 지날 때까지 모아 두었다가 시각 순으로 정렬해 NFA 에 먹입니다. 워터마크는 "최신 이벤트 시각 - 5 초"이므로, 시각 T 의 행이 처리되려면 T+5 초짜리 데이터가 도착해야 합니다. 센서가 1 초마다 값을 보내므로 복시계로 약 5 초가 걸립니다. 여기에 알람을 Kafka 에 쓰고 후단이 읽는 시간이 더 붙습니다.

이 5 초는 설정값에서 유도한 숫자이지 계측한 숫자가 아닙니다. 이 프로젝트의 지연 측정 기록 (docs/journal/09-metrics.md § 9.4)에는 ONNX 추론 지연 0.126~0.180 ms, EdgeX 스캔→Kafka 도달 1 초 이내는 있지만 CEP 알람 지연을 켜 항목은 없습니다. 실습에서 직접 재 볼 값으로 남겨 둡니다.

한산한 파티션이 시간을 멈춘다 — idle-timeout 여기까지가 교과서적인 설명이고, 이제 이 시스템에서 실제로 알람이 나오게 만드는 한 줄이 나옵니다. `01_sources.sql` 8 행입니다.

```
SET 'table.exec.source.idle-timeout' = '5s';
```

문제 상황은 이렇습니다. Flink 의 워터마크는 모든 입력 파티션의 워터마크 중 가장 작은 값입니다. 가장 뒤쳐진 파티션에 맞추는 것인데, 안 그러면 느린 파티션의 데이터가 전부 지각 처리되기 때문입니다. 그런데 `sensor.telemetry.raw` 는 파티션이 6 개이고 태그는 12 종이며, 태그 이름을 해시해 파티션을 고르므로 고르게 나뉘지 않습니다. 어떤 파티션에는 태그가 몇 개 몰리고, 어떤 파티션에는 하나 도 배정되지 않을 수 있습니다.

태그가 하나도 없는 파티션은 데이터가 오지 않으므로 그 파티션의 워터마크가 영원히 제자리입니다. 그러면 최솟값도 제자리이고, 전체 워터마크가 멈춥니다. 워터마크가 멈추면 정렬 버퍼에서 행이 빠져 나오지 못하고, CEP 매치는 완성돼도 영영 확정되지 않습니다. 잡은 RUNNING 인데 알람이 한 건도 나 오지 않는, 가장 진단하기 어려운 종류의 침묵입니다.

idle-timeout 은 "5 초 동안 데이터가 오지 않은 파티션은 놓고 있는(idle) 것으로 표시하고, 워터마크 최솟값 계산에서 빠라"는 설정입니다. 이 한 줄 덕분에 한산한 파티션이 시간을 붙들지 못합니다. 그 파티션에 다시 데이터가 들어오면 자동으로 계산에 복귀합니다.

주의. 이 설정은 CEP 만의 것이 아니라 01_sources.sql 에서 한 번 선언되어 같은 파일로 제출 되는 SQL 잡 3 개 전부 (임계치·Z-Score·CEP)에 적용됩니다. 실습에서 01_sources.sql 을 손덜 때 이 줄을 지우면, 증상은 "CEP 만 안 나온다"가 아니라 "이벤트 시간을 쓰는 것들이 전부 조용해진다"로 나타날 수 있습니다.

MATCH_RECOGNIZE 문법 해부

MATCH_RECOGNIZE 는 SQL 표준(SQL:2016)에 들어간 행 패턴 인식 구문입니다. Apache Flink 의 SQL 이 이것을 구현했고, 내부적으로는 앞에서 설명한 NFA 로 컴파일됩니다. 컴파일(compile)이란 사람이 쓴 코드를 기계가 실제로 실행할 형태로 번역하는 과정을 말합니다. 우리가 쓰는 것은 SQL 문장이지만, Flink 가 그것을 읽어 상태와 전이로 이루어진 오토마타 자료구조로 바꿔 놓고 실행한다는 뜻입니다.

즉 우리는 자바 코드를 한 줄도 쓰지 않고, SQL 문장 하나로 상태 기계를 기술합니다.

이름을 풀면 "match(맞는 것을) recognize(알아보라)"입니다. 이 구문은 테이블(스트림)을 받아서, 그 안에서 연속된 행들의 배열이 정해진 패턴과 맞는 구간을 찾아 각 구간마다 결과 행을 내놓습니다. 일반 SQL 의 WHERE 가 한 행씩 독립으로 판정하는 것과 근본적으로 다릅니다.

AR-100 의 실제 파일 flink/sql/04_tier1_cep.sql 에 들어 있는 문장 전체입니다.

```
MATCH_RECOGNIZE (
  PARTITION BY device
  ORDER BY event_time
  MEASURES
    LAST(VIB.ts)      AS ts,
    LAST(VIB.site)    AS site,
    'VT-101'         AS tag,
    LAST(VIB.value) AS value,
    CONCAT('교반기 전류 ', CAST(ROUND(LAST(OVERCURRENT.value), 2) AS STRING),
          'A (정격 120% 초과) 후 ',
          CAST(TIMESTAMPDIFF(SECOND, LAST(OVERCURRENT.event_time),
                             LAST(VIB.event_time)) AS STRING),
          '초 내 진동 ', CAST(ROUND(LAST(VIB.value), 2) AS STRING),
          'mm/s 상회 → 베어링 열화 의심') AS detail
  ONE ROW PER MATCH
  AFTER MATCH SKIP PAST LAST ROW
  PATTERN (OVERCURRENT OTHER*? VIB) WITHIN INTERVAL '10' SECOND
  DEFINE
    OVERCURRENT AS OVERCURRENT.tag = 'IT-102' AND OVERCURRENT.value > 9.6,
    VIB          AS VIB.tag      = 'VT-101' AND VIB.value      > 7.1
)
```

이제 절(clause)마다 하나씩 뜯어봅니다.

PARTITION BY device — 누구의 사건인가 스트림에는 여러 설비의 데이터가 섞여 흐릅니다. A 설비의 전류 상승과 B 설비의 진동 상승을 묶어 진단을 내면 명백한 오진입니다. PARTITION BY 는 스트림을 이 컬럼의 값별로 갈라 놓고, 각 갈래 안에서만 패턴을 찾으라는 지시입니다. 갈래마다 독립적인 NFA 인스턴스와 독립적인 부분 매치 목록이 생깁니다.

AR-100 은 simulator/plant.yaml 의 device: reactor-line-01 하나뿐이라 실질적으로 갈래가 하나 입니다. 문법상 이 줄을 빼면 여러 설비로 확장할 때 즉시 오진이 납니다.

또 하나 중요한 점은, 이 PARTITION BY 컬럼이 Flink 의 병렬 처리 단위와 같다는 것입니다. 갈래가 늘면 여러 태스크에 나눠 담아 동시에 처리할 수 있습니다. 다만 이 시스템의 실제 병렬도를 알아 두어야 이야기가 땅에 붙습니다.

flink/conf/config.yaml 의 값입니다.

```
taskmanager:
  numberOfTaskSlots: 8
parallelism:
  default: 2
```

기본 병렬도가 2 이므로 CEP 연산자도 서브태스크 2 개로 돛니다. 그런데 device 의 값이 한 종류뿐이므로 실제로는 그중 한 서브태스크만 일하고 나머지는 눕니다. 확장성은 "지금 병렬로 돌고 있다"가 아니라 "설비를 늘리면 코드 수정 없이

갈래가 나뉜다"는 의미로 이해해야 합니다. 슬롯 8 개는 잡 4 개 × 병렬도 2 에 맞춘 값입니다(설정에서 계산한 수치입니다).

ORDER BY event_time — 무엇을 기준으로 줄을 세우는가 Flink 에서 MATCH_RECOGNIZE 의 ORDER BY 첫 컬럼은 반드시 시간 속성이어야 하며, 오름차순이어야 합니다. 여기에 앞 절에서 설명한 event_time 과 워터마크가 그대로 걸립니다. 이 한 줄이 "전류가 먼저"라는 주장의 근거 전부입니다.

DEFINE — 각 패턴 변수가 무슨 뜻인가 읽는 순서상 DEFINE 을 먼저 보는 것이 이해가 빠릅니다. 여기서 패턴 변수 (pattern variable)를 정의 합니다. 패턴 변수란 "이런 조건을 만족하는 행"에 붙이는 이름입니다.

```
OVERCURRENT AS OVERCURRENT.tag = 'IT-102' AND OVERCURRENT.value > 9.6,
VIB          AS VIB.tag          = 'VT-101' AND VIB.value          > 7.1
```

OVERCURRENT 는 "태그가 IT-102 이면서 값이 9.6 을 넘는 행", VIB 는 "태그가 VT-101 이면서 값이 7.1 을 넘는 행"입니다. 이름은 마음대로 지어도 됩니다. 조건 안에서 자기 이름으로 컬럼을 참조한다 는 점에 주의하세요(OVERCURRENT.value).

여기가 CEP 의 결정적 특성입니다. 조건 자체는 여전히 단순 임계치입니다. 9.6 과 7.1 이라는 숫자는 앞 장의 USL 과 똑같은 숫자입니다. CEP 가 새로 더하는 것은 그 조건들을 어떤 순서로 만족해야 하는가뿐입니다. 이 점을 짚어 두지 않으면 학습자는 CEP 를 "더 똑똑한 임계치"로 오해합니다. 그렇지 않습니다. 임계치는 그대로 두고 시간 축을 추가하는 기술입니다.

PATTERN — 어떤 순서로 나와야 하는가

```
PATTERN (OVERCURRENT OTHER*? VIB) WITHIN INTERVAL '10' SECOND
```

이 한 줄이 이 장에서 가장 중요합니다. 괄호 안은 정규표현식(regular expression)과 같은 문법입니다. 정규표현식은 원래 글자의 나열에서 패턴을 찾는 표기법입니다. 예를 들어 ab*c 라고 쓰면 "a 가 나오고, b 가 0 번 이상 반복되고, 그다음 c 가 나오는 부분"을 뜻해서 ac , abc , abbbc 를 모두 찾아 냅니다. MATCH_RECOGNIZE 는 이 발상을 그대로 가져와, 찾는 대상을 글자 대신 행으로 바꾼 것입니다. 글자 하나에 해당하는 것이 DEFINE 으로 이름 붙인 패턴 변수입니다.

기호	이름	뜻
A B	연접	A 로 표시된 행 바로 다음 행이 B 여야 한다
A*	탐욕적 반복	0 회 이상 반복. 가능한 한 많이 먹는다
A*?	게으른(reluctant) 반복	0 회 이상 반복. 가능한 한 적게 먹는다
A+ / A?	1 회 이상 / 0~1 회	같은 계열의 수량자
WITHIN	시간 제한	첫 행부터 마지막 행까지 이 시간 안에 끝나야 한다

그렇다면 OTHER 는 무엇일까요. DEFINE 에 OTHER 가 없습니다. 이것이 의도입니다. MATCH_RECOGNIZE 에서 DEFINE 에 조건이 없는 패턴 변수는 모든 행에 매칭됩니다(조건이 항상 참). 즉 OTHER*? 는 "아무 행이나 필요한 만큼만 건너뛰어도 된다"는 뜻입니다.

왜 이 장치가 필요한지는 AR-100 이 실제로 걸려 넘어진 함정을 봐야 이해됩니다. 아래 AR-100 에서는 항목의 "0 초 함정"을 반드시 읽으시기 바랍니다.

WITHIN INTERVAL '10' SECOND 는 설계 보고서 원문의 "10 초 이내"를 그대로 옮긴 것입니다. 첫 행 (OVERCURRENT)의 이벤트 시간과 마지막 행 (VIB)의 이벤트 시간 차가 10 초를 넘으면 그 시도는 매치 로 인정되지 않습니다. 앞서 말한 대로 이 절은 의미와 메모리 한계를 동시에 정합니다.

MEASURES — 찾았으면 무엇을 계산해 내보낼 것인가 패턴이 완성되면 그 구간에 속한 행들이 손안에 있습니다.

MEASURES 는 그 행들로부터 출력 컬럼을 만듭니다. 행 하나가 아니라 여러 행에 걸친 계산이 가능하다는 점이 일반 SELECT 와 다릅니다.

여기서 쓰이는 함수가 탐색 함수(navigation function)입니다.

FIRST(변수.컬럼) — 그 변수로 표시된 행 중 처음 것의 값 LAST(변수.컬럼) — 그 변수로 표시된 행 중 마지막 것의 값

AR-100 의 MEASURES 를 읽어 보면, 알람의 시각·사이트·값은 진동 행에서 가져옵니다(LAST(VIB.ts) , LAST(VIB.value)). 패턴이 완성된 시점이 진동 초과 시점이기 때문입니다. 그리고 detail 문자열에 두 행의 정보를 함께 담습니다.

```
TIMESTAMPDIFF(SECOND, LAST(OVERCURRENT.event_time), LAST(VIB.event_time))
```

이 한 조각이 전류 초과와 진동 초과 사이의 실제 간격(초)을 계산합니다. 서로 다른 두 행의 시각을 빼는 연산인데, 일반 SQL 로 전혀 못 하는 것은 아닙니다. 두 가지 우회로가 있습니다.

자기조인(self-join) — 같은 테이블을 자기 자신과 한 번 더 붙여, 서로 다른 두 행을 한 줄에 나란히 놓고 시각을 빼는 기법입니다. 조인 조건에 $b.시각 - a.시각 < 10$ 초 같은 것을 넣으면 흉내는 납니다. 윈도우 함수 LAG / LEAD — 현재 행에서 몇 칸 앞·뒤 행의 값을 끌어오는 함수입니다. 이것으로도 직전 행의 시각을 가져올 수 있습니다.

그러니 정확한 표현은 "불가능하다"가 아니라 "빠르게 다루기 어려워진다"입니다. "전류 초과 후, 임의 개수의 다른 행을 건너뛰 다음, 처음 오는 진동 초과"라는 조건을 조인 조건으로 옮기는 순간 중복 매치를 걸러 낼 장치를 직접 짜야 하고, 조건이 세 개·네 개로 늘거나 "B 가 오지 않음"을 표현해야 하면 SQL 이 급격히 불어납니다. MATCH_RECOGNIZE 는 그 관리를 문법이 대신해 줍니다.

ONE ROW PER MATCH — 매치 하나에 결과 몇 줄인가 매치 하나당 요약 행 하나를 내보낸다는 선언입니다. SQL 표준에는 매치에 포함된 모든 행을 그대로 내보내는 ALL ROWS PER MATCH 모드도 정의돼 있습니다.

주의 — 이것은 설계 선택이 아닙니다. Flink SQL 은 ALL ROWS PER MATCH 를 지원하지 않습니다. Flink 1.20 공식 문서가 "Currently, the only supported output mode is ONE ROW PER MATCH " 라고 명시합니다(AR-100 이 쓰는 버전은 .env 의 FLINK_VERSION=1.20.1 입니다). 즉 이 줄은 쓸 수 있는 유일한 값을 명시한 것이며, 생략해도 동작은 같습니다. 적어 두면 읽는 사람이 출력 형태를 바로 알 수 있으므로 남겨 둔 것입니다. "AR-100 이 알람을 하나로 만들려고이 모드를 골랐다"는 설명은 틀렸습니다.

AFTER MATCH SKIP PAST LAST ROW — 찾은 다음 어디서 다시 시작하는가 매치를 하나 찾고 나면, 다음 탐색을 어느 행부터 시작할지 정해야 합니다. 이 선택에 따라 알람 개수 가 달라집니다.

전략	다음 탐색 시작점	결과
SKIP PAST LAST ROW	매치의 마지막 행 다음 행	구간이 겹치지 않음. 알람 최소
SKIP TO NEXT ROW	매치의 첫 행 다음 행	구간이 겹칠 수 있음 → 알람 증가
SKIP TO LAST 변수	지정한 변수의 마지막 행	중간 지점부터 재탐색

베어링 열화는 한 번 진단하면 족한 사건이므로 AR-100 은 SKIP PAST LAST ROW 를 씁니다. 전류가 계속 높은 상태에서 진동도 계속 높다면 SKIP TO NEXT ROW 에서는 겹치는 매치가 생겨 알람이 늘어날 것으로 예상됩니다. 다만 이 프로젝트는 그 건수를 센 기록이 없습니다 — 예상이지 실측이 아닙니다. 뒤의 실습 과제에서 직접 세어 보고 이 문단의 예상이 맞는지 판정하는 것이 이 항목의 학습 목표입니다.

부분 매치는 어떻게 살아남는가

부분 매치는 메모리에만 있는 임시 정보처럼 보이지만, 사실은 이 시스템이 반드시 지켜야 하는 데이터입니다. "전류를 봤고 진동을 기다리는 중"이라는 정보가 서버 재기동으로 사라지면, 바로 그 순간 진행 중이던 고장의 진단이 통째로 없어집니다. 어떻게 지키는지 보려면 Flink 의 구성 요소부터 알아야 합니다.

잡 · JobManager · TaskManager · 슬롯 잡(job) — 제출해서 계속 돌고 있는 처리 프로그램 하나. AR-100 에는 잡이 4 개 있고, 그중 하나가 CEP 입니다. JobManager — 잡을 받아 실행 계획을 짜고, 작업을 나눠 주고, 체크포인트를 지휘하고, 장애 시 재시작을 결정하는 관리자 프로세스. 관제실에 해당합니다. TaskManager — 실제 데이터를 처리하는 일꾼 프로세스. 작업장에 해당합니다. 슬롯(slot) — TaskManager 한 대가 동시에 맡을 수 있는 작업 자리의 개수. 작업대 수라고 보면 됩니다. AR-100 은 numberOfTaskSlots: 8 입니다.

이들은 JVM(Java Virtual Machine, 자바 가상 머신) 위에서 돕니다. JVM 은 자바로 만든 프로그램을 실행해 주는 소프트웨어이고, JVM 이 프로그램용으로 쓰는 메모리 영역을 힙(heap) 메모리라고 부릅니다. 부분 매치 같은 진행 중 데이터는 기본적으로 이 힙 위에 있습니다. 프로세스가 죽으면 힙은 통째로 사라집니다. 이 장 뒤에서 Esper 의 한계로 지적되는 것이 정확히 이 지점입니다.

체크포인트 — 진행 중 상태의 정기 사진

체크포인트(checkpoint)는 "지금 이 순간 모든 일꾼이 들고 있는 진행 중 데이터를, 어디까지 읽었는지와 함께 통째로 저장해 둔 사진"입니다. 일정 간격으로 찍어 두었다가, 장애가 나면 가장 최근 사진 으로 되돌아가 그 지점부터 다시 흘립니다. Flink 에서 이렇게 저장되는 진행 중 데이터를 상태(state) 라고 부르고, 부분 매치는 정식 상태로 취급되어 체크포인트에 들어갑니다.

flink/conf/config.yaml의 실제 설정입니다. 줄마다 뜻을 붙입니다.

```
state:
  backend:
    type: rocksdb
    incremental: true
  checkpoints:
    dir: file:///opt/flink/checkpoints
execution:
  checkpointing:
    interval: 10s
    mode: EXACTLY_ONCE
    timeout: 60s
    min-pause: 5s
```

패턴 상태의 장애 복구

부분 매치도 Flink 상태이므로 체크포인트와 재시작 전략으로 되살려야 합니다. 설정의 정보는 5장의 RocksDB·10 초 체크포인트·지수 백오프 설명에 있습니다. 단순히 체크포인트를 켜다는 이유로 잡이 자동 재시작하거나 외부 알람 중복까지 사라지는 것은 아닙니다. 원문이 설명하는 상태 복구와 최종 싱크 전달 보장은 구분해서 읽습니다.

```
restart-strategy:
  type: exponential-delay
  exponential-delay:
    initial-backoff: 5s
    max-backoff: 2min
    backoff-multiplier: 2.0
    reset-backoff-threshold: 10min
    jitter-factor: 0.1
heartbeat:
  timeout: 120000          # 50s → 120s (밀리초 단위)
taskmanager:
  memory:
    process:
      size: 4608m          # 3072m 에서 상향
```

지수 백오프(exponential backoff)가 무엇인지 풀어 두겠습니다. 실패한 일을 다시 시도할 때 기다리는 시간을 매번 배로 늘려 가는 방식입니다. 5 초 기다렸다 재시도, 실패하면 10 초, 다음은 20 초, 40 초... 이렇게 늘리되 상한에서 멈춥니다. 왜 늘리는가 하면, 원인이 안 풀린 상태에서 초마다 재시도하면 재시작 자체가 시스템에 부하를 더해 상황을 악화시키기 때문입니다.

키	값	뜻
initial-backoff	5s	첫 재시도까지 기다리는 시간
backoff-multiplier	2.0	실패할 때마다 대기 시간에 곱하는 배수. 5 초 → 10 초 → 20 초...
max-backoff	2min	대기 시간의 상한. 무한정 늘어나지 않게 막습니다
reset-backoff-threshold	10min	10 분 동안 멀쩡히 돌면 대기 시간을 처음값으로 되돌립니다
jitter-factor	0.1	대기 시간에 $\pm 10\%$ 무작위 흔들림을 섞습니다. 여러 잡이 같은 순간에 한꺼번에 재시도해 몰리는 일을 막습니다

나머지 두 줄도 사건의 일부입니다. heartbeat.timeout 을 50 초에서 120 초로 늘린 것은 일시적인 느려짐을 죽음으로 오판하지 않게 하려는 것이고, TaskManager 프로세스 메모리를 3072m 에서 4608m 으로 올린 것은 메모리 압박이 하트비트 지연의 원인일 수 있기 때문입니다. "재시작 전략을 켜다"만 기억하면 절반만 배운 것입니다. 재발을 줄이는 조정과 재발했을 때 살아나는 장치를 함께 넣는 것이 이 사건의 교훈입니다.

주의. "체크포인트를 켜는데 왜 복구가 안 되지"로 헤매는 원인이 대부분 이것입니다. PDF 가 말하는 "정확히 한 번 상태 복구"는 체크포인트 + 재시작 전략 두 가지가 모두 있어야 성립합니다.

AR-100에서는. 이 시스템의 CEP 는 flink/sql/04_tier1_cep.sql 파일 하나, SQL 문 한 개로 구현 되어 있습니다. 자바 코드가 없습니다. 잡 이름은 파일 첫 줄의 SET 'pipeline.name' = 'AR100-Tier1-CEP'; 로 붙습니다.

임계치 두 개가 어디서 나왔는가 교반기 M-101 의 정격 전류는 simulator/plant.yaml 의 physics.agitator.rated_current_a: 8.0 입니다. PDF 가 말한 "정격의 120%"는 곧 $8.0 \times 1.2 = 9.6$ A 이고, 같은 파일 20 행의 IT-102 태그 정의에 usl: 9.6 으로 들어가 있습니다.

진동 기준치 7.1 mm/s 는 21 행 VT-101 의 `usl: 7.1` 입니다. 이 값은 전류처럼 계산식으로 유도되지 않습니다. 대신 판단 근거는 같은 파일의 정상값에서 나옵니다.

```
agitator:
  rated_current_a: 8.0
  base_vibration_mms: 2.1
```

정상 운전 진동의 기준값이 2.1 mm/s 이고(`simulator/plant.py` 의 `_vibration` 은 여기에 레벨 보정 $0.82 + 0.30 \times$ 레벨 을 곱하므로 실제로는 2 mm/s 안팎), 임계치 7.1 은 그 약 3.4 배입니다. 같은 함수에서 베어링 마모가 최대로 누적되면 진동에 최대 4 배($1.0 + 3.0 \times \text{vib_wear}$)가 곱해지므로, 7.1 은 "마모가 상당히 진행돼야 닿는, 그러나 최대치에는 못 미치는" 지점에 놓여 있습니다. 실측 알람에서 관측된 값이 7.21 mm/s 인 것과 맞습니다. 초보가 "7.1 이 큰 값인가 작은 값인가"를 판단할 근거는 이것입니다 — 정상 3 배 남짓.

주의 — `tag_limits.csv` 를 직접 고치지 마세요. Flink 가 읽는 `flink/sql/tag_limits.csv` 는 파생 파일입니다.

```
IT-102,A,,9.6
VT-101,mm/s,,7.1
```

`docs/journal/10-config-reference.md` 의 설정 파일 지도는 `simulator/plant.yaml` 에 ★ 단일 출처 표시를 붙이고, `flink/sql/tag_limits.csv` 는 (자동 생성) 으로 표기합니다. ADR-009 도 `plant.yaml` → (스크립트) → `flink/sql/tag_limits.csv` 의 생성 경로를 그려 둡니다. 규격을 바꾸려면 `plant.yaml` 을 고치고 재생성 절차 (`make regen-edgex && make regen-fuxa`)를 거쳐야 합니다. CSV 만 손대면 두 가지가 어긋납니다. 첫째, 재생성할 때 덮어써집니다. 둘째, 시뮬레이터는 `plant.yaml` 을 읽으므로 물리 모델이 만드는 값과 Flink 가 감시하는 규격이 서로 다른 세계가 됩니다.

입력은 무손실 원본 토픽에서 받고, 두 태그만 걸러냅니다

```
FROM (
  SELECT * FROM telemetry_raw
  WHERE tag IN ('IT-102', 'VT-101')
)
```

`telemetry_raw` 는 `01_sources.sql` 에 정의된 Kafka 소스로, 토픽은 `sensor.telemetry.raw` (파티션 6 개), 컨슈머 그룹은 `flink-tier1` , 시작 위치는 `latest-offset` — 즉 잡을 제출한 뒤에 들어온 데이터만 봅니다. 보관값이 섞이지 않은 원본 토픽을 쓰는 것이 의도입니다. 추정치로 진단을 내면 안 되기 때문입니다.

이 사전 필터가 중요합니다. 스트림에는 태그 12 종이 초당 12 건으로 섞여 들어오는데, 미리 두 태그만 남기면 NFA 가 훑어야 할 행이 1/6 로 줄어듭니다. 다만 앞서 설명한 워터마크는 필터 이전, 소스 단계에서 계산되므로 `idle-timeout` 의 역할은 이 필터와 무관하게 그대로 필요합니다.

출력은 통합 알람 토픽으로 나갑니다 `alerts` 싱크(토픽 `sensor.alerts` , 파티션 3 개, 보존 7 일)에 다음 필드로 기록됩니다.

필드	값	의미
<code>alert_type</code>	<code>CEP_BEARING</code>	알람 종류
<code>severity</code>	<code>CRITICAL</code>	심각도
<code>detector</code>	<code>TIER1_CEP</code>	어느 탐지기가 잡았는가
<code>tag</code>	<code>VT-101</code>	패턴을 완성시킨 센서 태그
<code>device</code>	<code>reactor-line-01</code>	PARTITION BY 컬럼이라 자동 포함

`detector` 필드가 있기 때문에 나중에 "이 알람은 규칙이 잡았나, Z-Score 가 잡았나, CEP 가 잡았나, 오토인코더가 잡았나"를 구분할 수 있습니다. 검증 스크립트 `scripts/verify.py` 는 정확히 이 필드로 판정합니다.

```
("bearing_wear", 50, "TIER1_CEP", "전류→진동 시간 종속 패턴은 CEP 가 처리")
```

가운데 숫자 50 은 고장을 넣고 알람을 기다리는 초 단위 대기 시간입니다. 앞에서 계산한 지연 요소들 — 전류가 9.6 을 넘기까지의 램프, 진동의 6 초 지연, 진동이 7.1 을 넘기까지의 램프, 워터마크 5 초 — 을 모두 담을 만큼 넉넉하게 잡은 값입니다.

고장 시나리오 — `bearing_wear` 가 시간차를 실제로 만듭니다 CEP 를 시연하려면 실제로 시차가 있는 고장이 있어야 합니다. `simulator/plant.yaml` 123~127 행의 정의입니다.

```
bearing_wear:
  desc: "IT-102 상승 후 10 초 내 VT-101 상승 → Flink CEP MATCH_RECOGNIZE 검증"
  default_duration_s: 45
  target: [IT-102, VT-101]
  lag_s: 6
```

그리고 simulator/plant.py 의 물리 모델이 이 시차를 만들어 냅니다.

```
if "bearing_wear" in self.faults:
    f = self.faults["bearing_wear"]
    self.bearing_wear = min(1.0, self.bearing_wear + dt_s / 25.0) # 전류: 즉시
    if f.elapsed >= f.lag_s:
        self.vib_wear = min(1.0, self.vib_wear + dt_s / 23.0) # 진동: 지연 + 완만
```

전류는 고장 주입 즉시 25 초에 걸쳐 누적되기 시작하고(`_agitator_current` 에서 $1.0 + 0.62 * bearing_wear$ 배수로 반영), 진동은 `lag_s=6` 초가 지난 뒤부터 23 초에 걸쳐 누적됩니다

(`_vibration` 에서 $1.0 + 3.0 * vib_wear$). 두 누적기를 분리한 것이 핵심입니다.

주의 — "0 초 함정". 처음 구현은 PATTERN (OVERCURRENT VIB) 였습니다. 알람은 떴습니다. 그런데 간격이 언제나 0 초로 나왔습니다.

```
교반기 전류 9.8A (정격 120% 초과) 후 0 초 내 진동 7.12mm/s 상회
^^^^
```

원인은 두 가지가 겹친 것이었습니다. (1) 문법 — PATTERN (A B) 는 "A 바로 다음 행이 B"를 요구합니다. 그런데 스트림에는 12 개 태그가 1 초마다 섞여 오므로, 두 조건이 같은 스캔 안 에서 동시에 참일 때만 매칭됐습니다. (2) 시뮬레이터 — 초기 물리 모델은 전류와 진동이 같은 열화 변수 하나를 공유해 거의 동시에 임계를 넘었습니다.

문법은 OTHER*? 를 끼워 중간 행을 건너뛸 수 있게 고쳤고, 시뮬레이터는 `vib_wear` 를 별도 누적기로 분리했습니다. 간격이 0 초로 나오는 CEP 는 CEP 가 아닙니다. WHERE a AND b 로도 되는 일을 비싼 상태 기계로 한 셈입니다. CEP 시연을 검증할 때는 "패턴이 매칭됐다"가 아니라 "시간 간격이 실제로 0 이 아니다"를 확인해야 합니다.

6 회 반복 실측 — 시차가 재현되는가 고쳐 놓고 끝내지 않고, 시차가 매번 10 초 창 안에 들어오는지 6 회 반복 시험했습니다. 아래 표는 시뮬레이터 쪽에서 두 태그가 각자 임계치를 넘은 시점을 쟀 것이며, 시각은 고장 주입 시점을 0 으로 둔 값입니다.

시행	IT-102 초과 시점	VT-101 초과 시점	간격	10 초 창 성립
1	t=21s	t=24s	3s	성립
2	t=17s	t=23s	6s	성립
3	t=21s	t=23s	2s	성립
4	t=20s	t=24s	4s	성립
5	t=20s	t=23s	3s	성립
6	t=19s	t=23s	4s	성립

6/6 시행에서 순서와 창이 모두 성립했습니다. 간격은 2~6 초에 분포했습니다.

그리고 아래는 위 표와는 다른 실행에서, 전체 검증 스크립트를 돌렸을 때 `sensor.alerts` 에 실제로 실린 알람입니다.

```
✓ bearing_wear → TIER1_CEP
교반기 전류 9.69A (정격 120% 초과) 후 7 초 내 진동 7.21mm/s 상회 → 베어링 열화 의심
```

주의 — 표의 2~6 초와 여기 7 초는 같은 시험의 결과가 아닙니다. 표는 재현성을 보려고 시차만 6 회 쟀 기록이고, 7 초는 그와 별개로 전체 파이프라인을 돌린 실행에서 알람 자신이 계산해 문자열에 넣은 값입니다. 시뮬레이터에는 계측 노이즈가 있어 초과 시점이 실행마다 흔들리고, 6 회는 분포를 확정할 만큼 많지 않습니다. 두 값 모두 WITHIN 의 10 초 창 안이라는 점이 확인된 사실이고, "표의 최댓값을 넘었으니 어느 한쪽이 틀렸다"고 읽으면 안 됩니다.

이 알람 문장 하나에 두 센서, 두 시각, 하나의 진단이 들어 있습니다. 단일 임계치로는 만들 수 없는 출력입니다. 다만 앞에서 짚은 대로, 같은 구간에서 IT-102 와 VT-101 의 THRESHOLD_USL 알람 도 함께 나가고 있습니다. `verify.py` 가 이 구간에서 수집한 알람들의 `detector` 집합을 모아 그 안에 TIER1_CEP 이 있지만 확인하는 것도 그래서입니다 — 여러 탐지기가 동시에 우는 것이 정상 동작입니다.

주의 — PARTITION BY 컬럼을 MEASURES 에 다시 쓰면 잡이 뜨지 않습니다

```
ValidationException: Columns ambiguously defined: {device}
```

PARTITION BY device 로 지정한 컬럼은 출력에 자동으로 포함됩니다. 여기에 MEASURES 에서 'M-101' AS device 를 또 정의해 이름이 충돌했습니다.

해결은 그 줄을 없애는 것이었습니다. 그 결과 지금 MEASURES 의 tag 자리에는 'VT-101' , 즉 설비 식별자가 아니라 센서 태그가 들어갑니다. 설비 식별자 M-101 은 알람 어느 필드에 도 남지 않았습니 — detail 문자열의 "교반기"라는 말이 유일한 흔적입니다. device 필드에는 PARTITION BY 가 넣어 주는 reactor-line-01 이 들어갑니다. 파일을 열어 확인할 때 이 점을 헷갈리지 마세요.

이 잡은 4 개 중 하나입니다 flink/submit-jobs.sh 는 SQL 파일 4 개를 이어 붙여 제출합니다. 01_sources.sql 은 잡을 만들지 않고 테이블만 선언하므로, SQL 쪽에서 생기는 잡은 3 개(임계치·Z-Score·CEP)입니다. 여기에 ONNX 추론 잡 1 개가 더해져 총 4 개입니다.

```
EXPECTED_SQL_JOBS=3    # 임계치 / Z-Score / CEP
EXPECTED_TOTAL=4       # SQL 3 + ONNX 1
running=$(curl -sf "http://$JTM/jobs/overview" | grep -o '"state":"RUNNING"' | wc -l)
```

마지막 줄이 상태를 확인하는 방식인데, 여기에도 사연이 있습니다. docs/journal/04-tier3-stream.md § 4.5 에 남은 사건입니다.

```
# 제출기가 이렇게 보고했지만
▶ Tier-1 규칙 탐지 SQL 제출...
✓ Tier-1 SQL 제출 완료
# 실제로는
$ curl -s localhost:27081/jobs/overview
총 0 개 잡
```

원인은 이랬습니다. 제출기는 SQL Client 의 출력에서 grep "`^\[ERROR\]`" 로 실패를 찾고 있었는데, SQL Client 가 터미널을 색칠하려고 붙이는 ANSI 색상코드(`\033[31;1m` 같은 보이지 않는 제어 문자)가 `[ERROR]` 앞에 붙어 있어서 "행의 시작"이라는 조건이 깨졌습니다. 오류가 분명히 났는데 grep 이 못 찾았고, 제출기는 실패를 성공으로 보고했습니다.

고친 방법이 두 겹입니다. 첫째, sed -E 's/\x1b\[[0-9;]*m/g' 로 색상코드를 먼저 지우고 나서 검사합니다. 둘째, 그것으로도 부족하다고 보고 최종 판정을 로그 텍스트가 아니라 잡의 실제 상태로 옮겼습니다. 여기서 쓰는 REST API 는 프로그램끼리 HTTP 로 주고받는 창구를 말합니다 — 사람이 브라우저로 여는 화면(`http://localhost:27081`) 대신, 같은 정보를 기계가 읽기 좋은 JSON 형식으로 돌려주는 주소(`/jobs/overview`)를 curl 로 호출해 RUNNING 인 잡이 몇 개인지 세는 것입니다.

이것이 ADR-008 의 결론입니다. "로그 텍스트는 포맷·색상·로케일에 따라 깨진다. 상태는 깨지지 않는다." 이 원칙은 CEP 와 직접 관계가 없어 보이지만, 앞에서 본 침묵형 장애들(워터마크 정지, 재시작 안 됨)을 잡아내는 마지막 그물이 바로 이 확인입니다.

참고로 같은 bearing_wear 고장은 2 계층 오토인코더(Autoencoder)도 잡아냅니다. 오토인코더는 12 개 태그 × 10 스텝 = 120 개 숫자를 한 덩어리로 받아 그것을 작은 크기로 압축했다가 다시 원래 대로 복원해 보는 신경망이고, 정상 데이터로만 학습합니다. 정상과 닮은 입력은 잘 복원되고 낯선 입력은 복원이 어긋나므로, 원본과 복원본의 차이인 재구성오차(reconstruction error)가 크면 이상으로 판정합니다.

학습 로그의 자가검증 결과(docs/journal/05-tier3-ml.md)는 이렇습니다.

```
정상      평균오차  0.02053  탐지율  0.5%   (오탐률)
bearing_wear      평균오차 146.13240  탐지율 100.0%
임계치 = 0.07007
```

146.1 이 큰 값인지 작은 값인지는 이 세 줄을 나란히 놓아야 판단됩니다. 정상 운전의 오차가 0.02, 이상이라고 판정하는 선이 0.07 인데 bearing_wear 의 오차는 146 입니다. 임계치의 2 천 배가 넘습니다. 아슬아슬하게 넘긴 것이 아니라 완전히 다른 영역에 있다는 뜻이고, 그래서 탐지율이 100% 입니다. 참고로 같은 표에서 drift 시나리오의 오차 55.8, noise 는 0.031 로 임계치 아래라 Tier-1 의 Z-Score 가 담당합니다.

두 계층이 겹쳐서 잡는 것은 낭비가 아니라 이중화입니다. 다만 오토인코더는 "무언가 평소와 다르다"와 기여도가 큰 센서 몇 개를 말해 줄 뿐이고, CEP 알람만이 "전류가 먼저, 진동이 나중, 간격 몇 초"라는 원인의 형태를 문장으로 말해 줍니다.

같은 자리의 다른 도구들 Esper / EQL — 원조 CEP 엔진. 설계 보고서 원문: "Esper 는 이벤트 구동 아키텍처(EDA)를 구축하기 위해 개발된 고성능 인메모리 CEP 엔진으로, SQL 과 유사한 이벤트 쿼리 언어(EQL)를 사용하여 시간 윈도우, 패턴 매칭 (예: 'A 이벤트 발생 후 3 분 이내에 B 가 발생하지 않는 상황'), 인과관계 필터링을 직관적으로 선언할 수 있다." 여기 나오는 EDA(Event-Driven Architecture, 이벤트 구동 아키텍처)는 시스템을 "무슨 일이 일어났다"는 사건 알림을 주고받는 방식으로 짜는 설계 양식입니다. A 가 B 를 직접 불러 시키는 대신, A 는 사건을 발행하고 관심 있는 쪽이 알아서 받아 처리합니다. 이 교본에서 Kafka 를 가운데 두고 각 계층이 토픽을 읽고 쓰는 구조가 바로 EDA 입니다. Esper 의 표현력은 지금 봐도 높습니다. "B 가 발생하지 않는 상황"까지 선언할 수 있다는 점에 주목하세요 — 오지 않은 것을 감지하는 일은 생각보다 까다롭습니다. 한계는 표현력이 아니라 구조입니다. PDF 는 이어서 이렇게 못 박습니다. "Esper 는 태생적으로 단일 프로세스 JVM 내에 임베디드 라이브러리 형태로 내장되어 구동되도록 설계되었다. 이로 인해 초당 수십만 건의 대규모 트래픽이 집중되거나 고가용성 클러스터링이 요구되는 엔터프라이즈 환경에서는 노드 장애 시 상태 복구와 수평적 확장(Scale-out)에 치명적인 한계를 드러낸다." 앞에서 JVM 과 힘을 풀어 두었으니 이제 이 문장이 구체적으로 읽힙니다. 부분 매치가 그 JVM 의 힙 메모리에만 있으므로 (1) 프로세스가 죽으면 진행 중이던 모든 시도가 사라지고(체크포인트에 해당하는 장치가 없습니다), (2) 서버를 늘려도 각 서버가 서로 다른 부분 매치를 들고 있어 합칠 방법이 없습니다. 설비 가 100 대로 늘었을 때 "전류는 1 번 서버가 보고 진동은 2 번 서버가 봤다"면 패턴은 영영 완성되지 않습니다. ksqldb — 쉽지만 순수 CEP 에는 못 미칩니다. PDF 는 장점을 분명히 인정합니다. "ksqldb 는 Kafka 상단에서 직관적인 스트림 조인, 필터링, 윈도우 집계를 순수 SQL 문법으로 처리할 수 있어 시스템 도입 장벽이 매우 낮다." 별도 클러스터 없이 Kafka 위에 얹히고, 배우는 데 며칠이면 충분합니다. 그러나 "반면 다중 이벤트 간의 정합적 순서 패턴을 감지하는 순수 CEP 작업이나 커스텀 텐서 조작에는 구조적 한계가 존재 한다." 여기 나오는 텐서(tensor)는 숫자를 격자 모양으로 늘어놓은 덩어리를 부르는 말입니다. 숫자 하나는 스칼라, 한 줄로 늘어놓으면 벡터, 표처럼 가로세로로 놓으면 행렬이고, 그 이상 차원까지 아우르는 일반 이름 이 텐서입니다. 신경망은 입력을 반드시 이 형태로 받으므로, "텐서 조작"이란 센서 값들을 신경망이 먹을 수 있는 정해진 모양의 숫자 덩어리로 조립하는 일을 뜻합니다. AR-100 의 2 계층이 12 태그 × 10 스텝 = 120 개짜리 덩어리를 만드는 것이 바로 그 작업이고, ksqldb 에는 이런 조립을 SQL 안에서 해낼 수단이 없습니다. 패턴 쪽도 마찬가지입니다. ksqldb 에는 MATCH_RECOGNIZE 에 해당하는 행 패턴 인식 구문이 없습니다. "A 후 10 초 내 B"를 흉내 내려면 스트림을 자기 자신과 윈도우 조인하고 시각 차를 조건으로 거는 식이 되는 데, 조건이 세 개 이상으로 늘거나 "B 가 오지 않음"을 표현해야 하면 SQL 이 급격히 복잡해집니다. 윈도우 집계와 조인까지가 ksqldb 의 자리입니다. Apache Flink (Flink CEP / MATCH_RECOGNIZE) — 이 시스템의 선택. PDF 의 판단입니다. "Apache Flink 는 Flink CEP 라이브러리를 통해 Esper 의 EQL 수준에 필적하는 고도의 비결정적 유한 오토마타 (NFA) 기반 패턴 매칭 기능을 분산 환경에서 제공한다. 더욱이 Flink 는 RocksDB 기반의 분산 상태 저장소 와 Chandy-Lamport 알고리즘 기반의 비동기 체크포인트를 지원하여 노드 장애 시에도 '정확히 한 번 (Exactly-once)' 상태 복구를 완벽히 보장한다." 여기 나오는 Chandy-Lamport 알고리즘은 분산 시스템의 스냅샷 기법입니다. 흐름을 멈추지 않고도 "모든 노드의 그 순간 상태"를 서로 어긋남 없이 찍어내는 방법이라고 이해하면 됩니다. 원리는 데이터 흐름 속에 표식(barrier)을 하나 흘려보내고, 각 노드가 그 표식을 만나는 순간의 자기 상태를 저장하는 것입니다. 덕분에 초당 수만 건이 흐르는 중에도 처리를 세우지 않고 10 초마다 상태를 저장할 수 있습니다. 앞 절에서 본 interval: 10s 와 mode: EXACTLY_ONCE 가 이 문장의 실물입니다. 그래서 PDF 는 결론짓습니다. "현대적인 아키텍처 관점에서 Esper 를 별도로 배치하여 파이프라인의 복잡도를 가중시킬 필요가 없으며, Apache Flink(또는 상대적으로 단순한 SQL 윈도우 연산일 경우 ksqldb)를 Kafka 에 직접 컨슈머로 결합하여 결측치 보간, 스트림 정제, 복합 이벤트 탐지를 단일 분산 계층에서 통합 수행하는 구성이 가장 합리적이다." Apache StreamPipes — 노코드 산업용 스트림 분석. 설계 보고서 원문: "Apache StreamPipes 는 산업용 IoT 데이터 스트림의 분석을 노코드(No-code) 환경으로 구축하기 위해 탄생한 오픈소스 플랫폼이다. 센서 데이터 어댑터, 윈도우 변환 필터, 이상 탐지 알고리즘 등을 웹 파이프라인 에디터 상에서 그래픽 블록 조립 형태로 구성할 수 있어 현장 도메인 엔지니어의 접근성이 매우 높다. 그러나 고성능 분산 분기 연산이나 복잡한 딥러닝 텐서 처리에는 제약이 따른다." SQL 을 못 쓰는 현장 엔지니어가 화면에서 블록을 이어 붙여 직접 규칙을 만든다는 것은 큰 장점입니다. PDF 비교표에 따르면 데이터 수집 백본으로 내장 Kafka/Pulsar 엔진을 가지고 있고 PLC·MQTT 전용 커넥터도 있습니다. 여기서 Pulsar(Apache Pulsar) 는 Kafka 와 같은 자리에 놓이는 또 다른 메시지 브로커입니다 — 토픽에 메시지를 쌓고 여러 소비자가 나눠 읽는 구조는 같고, 저장 계층을 분리한 설계 등에서 차이가 납니다. "Kafka 아니면 Pulsar 를 골라 쓸 수 있다"는 뜻으로 읽으면 됩니다. 대신 같은 비교표에서 스트림 분석 항목은 "사전 정의된 스트림 분석 블록", SCADA 제어 적합성은 "분석 위주(현장 제어 기능 부재)", 분산 확장성은 "중간(독립 클러스터링 지원)"으로 평가됩니다. 사용자 제안 스택(Flink-Kafka-FUXA)은 같은 줄에서 각각 "Flink 기반 분산 상태 연산 및 ONNX 실시간 서빙", "FUXA 결합을 통한 정밀 P&ID 양방향 제어", "극대화(Kafka 파티션 및 Flink 슬롯 선택 확장)"입니다. 여기 나오는 ONNX(Open Neural Network Exchange) 는 학습한 신경망 모델을 도구에 매이지 않는 공통 파일 형식으로 저장하는 규격입니다. PyTorch 로 학습한 모델을 .onnx 파일로 내보내면 자바로 짠 Flink 잡이 그 파일을 그대로 읽어 추론할 수 있고, 별도 추론 서버를 띄워 네트워크로 호출할 필요가 없어집니다. AR-100 의 4 번째 잡이

하는 일입니다. AR-100 이 StreamPipes 를 쓰지 않은 이유도 같습니다 — Flink SQL 이 같은 역할을 더 높은 확장성으로 수행하기 때문입니다. Nussknacker — Flink 위에 얹는 저코드 UI. PDF 참고문헌 49 번에 "Nussknacker: Low-Code Apache Flink"로 올라 있으나 본문 비교 대상은 아닙니다. 엔진은 Flink 그대로 두고 규칙 작성만 화면으로 하는 방식이라, "Flink 의 확장성은 원하되 SQL 작성 부담을 줄이고 싶다"는 자리에 놓입니다. 애플리케이션 코드로 직접 구현. 컨슈머를 하나 짜서 "전류 초과를 보면 변수에 시각을 저장해 두고, 진동 초과가 오면 차이를 계산한다"고 쓰면 수싵 줄이면 됩니다. 조건이 두 개일 때는 이게 제일 빠릅니다. 문제는 그다음입니다. 설비가 늘면 설비별 상태 맵을 직접 관리해야 하고, 프로세스가 죽으면 그 맵이 날아가고, 늦게 도착한 데이터를 어떻게 다룰지(워터마크에 해당하는 판단을) 직접 정해야 하고, 메모리 누수를 막으려면 만료 처리를 직접 짜야 합니다. CEP 엔진이 대신해 주는 것이 정확히 이 네 가지입니다.

세 갈래 정리

항목	Esper / EQL	ksqlDB	Apache Flink (MATCH_RECOGNIZE)
실행 형태	단일 JVM 임베디드 라이브러리	Kafka 위 SQL 엔진	독립 분산 클러스터
패턴 표현력	높음 (부재 패턴 포함)	낮음 (윈도우 조인 수준)	높음 (NFA 기반, EQL 수준에 필적)
수평 확장	치명적 한계	가능	파티션·슬롯 선형 확장
장애 시 상태 복구	힙 메모리 소실	Kafka 로그 기반	RocksDB + 체크포인트, exactly-once (재시작 전략을 켜야 실제로 복구됨)
학습 난이도	중간 (EQL 별도 학습)	낮음 (순수 SQL)	중간 (SQL + 시간·상태 개념)
AR-100 채택	미채택 (설계 보고서 결론 2)	미채택 (Flink 가 상위 호환)	채택

PDF 의 5 개 결론 중 결론 2 는 이 장 전체를 한 문장으로 요약합니다.

"Esper 의 현대적 대체: 레거시 단일 JVM 엔진인 Esper/EQL 대신, Kafka 와 완벽히 결합하여 수평 확장과 무손실 상태 복구를 보장하는 Apache Flink(Flink CEP)를 스트림 처리의 핵심 코어로 채택해야 한다."

주의할 점은 이 결론이 "Esper 의 문법이 틀렸다"는 말이 아니라는 것입니다. EQL 이 만든 선언형 패턴 언어라는 발상은 그대로 SQL 표준의 MATCH_RECOGNIZE 로 살아남았습니다. 교체된 것은 언어가 아니라 그 언어를 돌리는 그릇입니다. 한 대의 JVM 힙에서 분산 상태 저장소로 바뀐 것뿐입니다.

이 장을 한 문단으로 CEP 는 임계치를 똑똑하게 만드는 기술이 아닙니다. 임계치는 그대로 둔 채 시간 축을 얻는 기술입니다. 그 시간 축은 저절로 생기지 않습니다. Kafka 는 파티션 안에서만 순서를 보장하고 IT-102 와 VT-101 은 다른 파티션에 있으므로, 순서는 이벤트 시간과 워터마크로 Flink 가 직접 세웁니다. 그 대가로 알람은 설정상 최소 5 초 늦고, 한산한 파티션이 시간을 붙들지 못하도록 idle-timeout 한 줄이 필요하며, 진행 중인 부분 매치를 지키려면 체크포인트와 재시작 전략이 함께 있어야 합니다. 그렇게 해서 얻는 것은 알람 수 의 감소가 아니라 원인의 형태를 말해주는 알람 한 건입니다.

직접 확인해 볼 것 flink/sql/04_tier1_cep.sql 에서 OTHER*? 를 지우고 다시 제출한 뒤, detail 문자열의 초 값이 어떻게 바뀌는지 관찰하고 이유를 설명해 보세요. WITHIN INTERVAL '10' SECOND 를 '2' SECOND 로 줄이면 bearing_wear 가 탐지되는지 확인하고, plant.yaml 의 lag_s: 6 및 6 회 실측표의 간격 2~6 초와 연결지어 설명해 보세요. AFTER MATCH SKIP PAST LAST ROW 를 SKIP TO NEXT ROW 로 바꾸고 45 초짜리 고장을 한 번 주입했을 때 CEP_BEARING 알람이 몇 건 나오는지 세어 보세요. 본문은 "늘어날 것"이라고만 예상했을 뿐 실측하지 않았습니다. 여러분의 숫자가 이 교본의 빈칸을 채웁니다. 같은 고장 한 번에 sensor.alerts 로 나가는 알람을 detector 별로 세어 보세요. TIER1_RULE 이 몇 건, TIER1_CEP 이 몇 건인지 비교하면 "CEP 가 알람을 줄여 주지 않는다"는 본문의 설명을 눈으로 확인 할 수 있습니다. 01_sources.sql 8 행의 SET 'table.exec.source.idle-timeout' = '5s'; 를 주석 처리하고 잡을 다시 제출한 뒤, 알람이 나오는지 관찰해 보세요. 잡 상태는 RUNNING 인데 알람만 사라지는 침묵을 직접 만들어 보는 것이 목적입니다. 진동 초과 시각(원본 토픽의 ts)과 알람이 sensor.alerts 에 실린 시각의 차이를 재서, 본문이 설정에서 유도한 "최소 5 초"와 비교해 보세요. docker compose kill flink-taskmanager 로 태스크매니저를 죽인 뒤, 진행 중이던 부분 매치가 체크포인트에서 복구되어 알람이 정상적으로 나오는지 Flink UI(http://localhost:27081)에서 관찰해 보세요. 이어서 flink/conf/config.yaml 의 restart-strategy 블록을 지우고 같은 실험을 반복해, 체크포인트 만으로는 복구되지 않는다는 것을 확인해 보세요.

7 머신러닝 이상 탐지 — 태그의 관계를 읽기

원문 교본 197~233 쪽의 설명을 통한

이 장의 “규칙이 못 잡는 것”은 단일 규격 한계만 보는 경우와 구분해서 읽어야 합니다. 원문 앞부분은 drift 를 오토인코더만 잡는 예로 설명하지만, 225 쪽과 293 쪽의 후속 실측에서는 Z-Score 도 주입 4 초 뒤에 탐지했습니다. 오토인코더의 차이는 여러 태그의 정상 관계와 재구성 오차 기여도를 함께 제공하는 데 있습니다. “모든 규칙이 침묵한다”는 뜻으로 축약하지 않습니다. [원문 18, 197, 225, 293 쪽]

두 종류의 Tier 와 검출기의 관계

원문에는 두 번호 체계가 함께 쓰입니다. 아키텍처의 Tier 1~4 는 예지·백본·Flink 처리·저장/관제 층을 뜻합니다. 반면 TIER1_RULE·TIER1_ZSCORE·TIER1_CEP 와 TIER2_ML 은 검출기 이름이며 모두 처리 계층 안에서 돌아갑니다. 번호만 보고 ML 이 Kafka 와 같은 층이라고 해석하면 안 됩니다.

검출기	읽는 차이
규격 규칙	사람이 정한 상·하한을 넘었는가
Z-Score	한 태그가 최근 자신의 평균·흔들림에서 얼마나 벗어났는가
CEP	정의한 서로 다른 사건이 정해진 순서·시간 안에 나타났는가
오토인코더	여러 태그와 시간 묶음이 학습한 정상 구조로 얼마나 잘 복원되는가

각 검출기는 서로 보완하며 같은 고장에 함께 반응할 수 있습니다. ML 은 숫자를 단순히 더 빠르게 읽는 기능이 아니라 입력 관계를 학습한 모델입니다. 모델이 내놓는 기여 태그는 볼 곳을 좁히는 정보이지 물리적 원인을 확정하는 판정은 아닙니다.

신경망·가중치·텐서·손실

오토인코더를 설명하려면 먼저 네 단어를 풀어야 합니다. 이 넷을 모르면 뒤의 모든 문장이 막힙니다.

신경망(neural network). 숫자 여러 개를 받아 숫자 여러 개를 내놓는 함수입니다. 안쪽은 층(layer)을 여러 겹 쌓아 만듭니다. 층 하나가 하는 일은 단순합니다. 들어온 숫자들에 저마다 정해진 계수를 곱해서 전부 더하고, 거기에 상수를 하나 더합니다. 그 계수를 가중치(weight), 더하는 상수를 편향(bias) 이라고 부릅니다. 파이토치에서 이 층의 이름이 nn.Linear 입니다.

가중치를 갱신한다는 게 무슨 일인가. 처음에 가중치는 무작위 숫자입니다. 그 상태로 데이터를 넣으면 출력이 엉망입니다. 그래서 얼마나 틀렸는지를 숫자 하나로 쟀 값을 만듭니다. 그것이 손실(loss)입니다. 손실이 커질수록 더 틀린 것입니다. 그다음 "가중치를 아주 조금 늘리면 손실이 늘어나는가 줄어드는가"를 가중치마다 계산해서, 줄어드는 방향으로 전부 조금씩 옮깁니다. 이 한 번의 이동이 갱신이 고, 이것을 수천 번 반복하는 것이 학습입니다. 얼마나 크게 옮길지를 정하는 값이 학습률(learning rate)이고 AR-100 은 0.001 입니다.

AR-100 의 손실은 평균제곱오차(MSE)입니다(ml/train.py 의 nn.MSELoss()). 즉 여기서 말하는 손실 은 곧 재구성 오차이고, 둘은 같은 계산에 붙은 다른 이름입니다. 학습 중에 보는 것을 손실이라 부르고, 운영 중에 이상 판정에 쓰는 것을 재구성 오차라고 부릅니다. 뒤에서 "학습 손실이 낮으니 모델이 좋다"가 왜 틀린 판단인지 다룰 텐데, 그 이야기의 손실이 바로 이 숫자입니다.

비선형(non-linear). 입력을 2 배로 키웠을 때 출력도 정확히 2 배가 되는 관계를 선형이라고 합니다. 직선입니다. 그렇지 않은 모든 관계가 비선형입니다. 문제는 nn.Linear 를 아무리 여러 겹 쌓아도 결과는 여전히 하나의 선형 변환이라는 점입니다. 곱하고 더하는 일을 반복해 봐야 곱하고 더하는 일이기 때문입니다. 그래서 층 사이에 활성화 함수를 끼웁니다. AR-100 이 쓰는 것은 ReLU(Rectified Linear Unit)로, 음수가 들어오면 0 으로 깎고 양수는 그대로 통과시키는 함수입니다. 이 꺾임 하나가 들어가면 층을 쌓을수록 표현할 수 있는 모양이 늘어납니다. "온도가 70 도까지는 pH 에 별 영향이 없다가 그 위 부터 급격히 떨어진다" 같은 꺾이는 관계는 ReLU 가 있어야 배울 수 있습니다.

텐서(tensor). 숫자를 축을 따라 나열한 덩어리를 부르는 이름입니다. 숫자 하나는 스칼라(scalar), 한 줄로 늘어선 것은 벡터(vector), 가로세로 격자는 행렬(matrix), 축이 몇 개든 상관없이 일반화한 것이 텐서입니다. 벡터도 텐서고 행렬도 텐서입니다. 딥러닝 라이브러리는 입력·출력·가중치를 전부 텐서로 다루기 때문에 "입력 텐서"라는 말이 계속 나옵니다.

AR-100 에서 실제로 오가는 텐서의 모양은 이렇습니다.

언제	텐서 모양	읽는 법
학습 중 한 묶음	(256, 120)	120 개짜리 입력 256 개를 한 판에 올린 것
추론 한 번	(1, 120)	120 개짜리 입력 1 개. 자바 코드의 <code>new float[][]{flat}</code> 이 이것
첫 층의 가중치	(120, 64)	120 개 입력을 64 개 출력으로 보내는 계수표

앞쪽 축(256 이나 1)을 배치(batch) 축이라고 합니다. 한 번에 몇 개를 같이 계산할지를 나타내는 자리 이고, 데이터의 의미와는 무관합니다. 이 축이 뒤에서 `dynamic_axes` 이야기로 다시 나옵니다.

모델이 얼마나 큰가. AR-100 의 신경망은 120→64→16→64→120 구조입니다(`ml/train.yaml` 의 `hidden: [64, 16]`). 가중치와 편향의 개수를 층별로 세면 $120 \times 64 + 64 = 7,744$, $64 \times 16 + 16 = 1,040$, $16 \times 64 + 64 = 1,088$, $64 \times 120 + 120 = 7,800$ 으로 합계 17,672 개입니다. 하나가 4 바이트(float32)이므로 약 70.7 KB, 즉 0.07 MB 입니다. 실제 저장된 파일 크기와 정확히 맞습니다(학습 로그: ONNX 저장 완료: `/opt/models/model.onnx` (0.07 MB)). "딥러닝"이라는 말에서 연상되는 크기와 전혀 다릅니다. 이 정도 크기라야 스트림 처리 중에 매초 돌릴 수 있습니다.

오토인코더의 구조

오토인코더(Autoencoder)는 입력을 그대로 다시 만들어내도록 훈련시킨 신경망입니다. 이름 그대로 "스스로(auto) 부호화하는(encode) 장치"입니다. 입력과 출력이 같은 것을 목표로 한다니 쓸모없어 보이는데, 가운데에 병목을 두는 순간 의미가 생깁니다.

구조는 두 덩어리입니다.

인코더(Encoder, 부호기) — 입력 벡터를 점점 좁은 층으로 통과시켜 저차원 잠재 벡터(latent vector) 로 압축합니다. 여기서 차원(dimension)은 숫자의 개수입니다. 120 개의 숫자를 16 개로 줄인다면 120 차원에서 16 차원으로 압축한 것입니다. 잠재(latent)라는 말은 "겉으로 드러나지 않는, 속에 들어 있는"이라는 뜻이고, 잠재 벡터는 원래 데이터를 설명하는 데 꼭 필요한 최소한의 요약입니다.

디코더(Decoder, 복호기) — 그 16 개 숫자만 가지고 원래의 120 개 숫자를 다시 만들어냅니다.

120 개를 16 개로 줄였다가 되살리라고 하면, 신경망은 모든 숫자를 외울 수 없습니다. 통로가 좁아서 불가능합니다. 그래서 신경망은 버려도 되는 것과 버리면 안 되는 것을 스스로 학습합니다. 버려도 되는 것은 계측 노이즈이고, 버리면 안 되는 것은 태그들이 서로 맞물리는 규칙입니다. "유량이 올라가면 펌프 전류도 올라간다"는 관계를 알고 있으면, 유량 하나만 기억해도 펌프 전류를 복원할 수 있습니다. 압축은 바로 그런 관계를 찾아내라는 압박입니다.

그래서 훈련이 끝난 오토인코더는 사실상 정상 운전의 물리 법칙을 압축해 담은 상자가 됩니다.

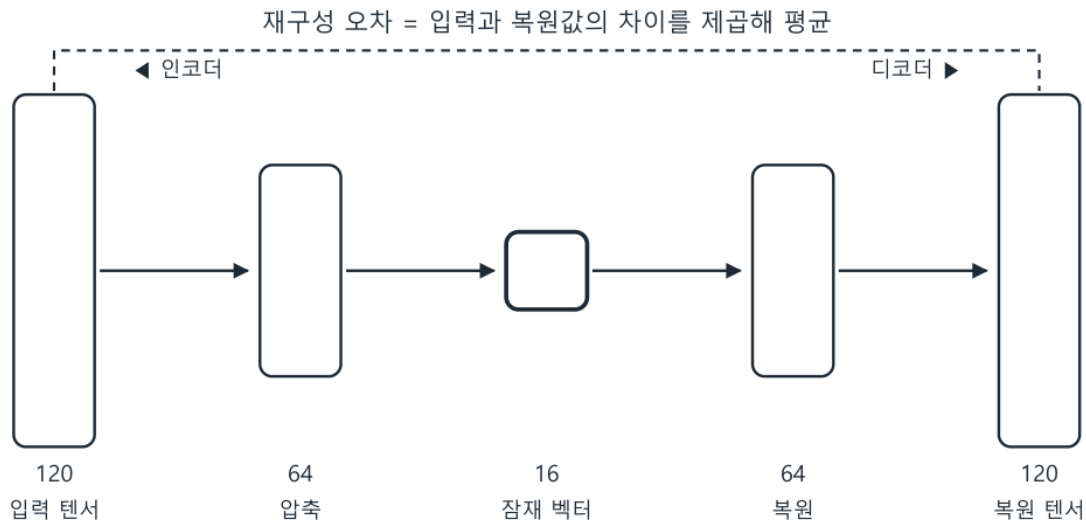
AR-100 의 모델 정의 전문입니다(`ml/train.py`).

```
class Autoencoder(nn.Module):
    """Dense Autoencoder. 인코더로 저차원 잠재벡터까지 압축한 뒤 복원한다."""
    def __init__(self, dim: int, hidden: list[int]):
        super().__init__()
        h1, h2 = hidden
        self.encoder = nn.Sequential(
            nn.Linear(dim, h1), nn.ReLU(),
            nn.Linear(h1, h2), nn.ReLU(),
        )
        self.decoder = nn.Sequential(
            nn.Linear(h2, h1), nn.ReLU(),
            nn.Linear(h1, dim),
        )
    def forward(self, x):
        return self.decoder(self.encoder(x))
```

`super().__init__()` 줄을 빼면 안 됩니다. 파이토치의 `nn.Module` 은 `self.encoder = ...` 같은 속성 할당을 가로채서 내부 목록에 등록하는데, 그 목록을 만드는 일이 부모 클래스의 초기화입니다. 이 줄이 없으면 첫 속성 할당에서 바로 실패합니다. `nn.Sequential` 은 여러 층을 적은 순서대로 이어 붙여 하나로 묶어 주는 상자이고, `forward` 는 "입력이 들어오면 인코더를 태우고 그 결과를 디코더에 태운다"는 흐름 자체입니다.

초보가 헷갈리는 지점 1. "출력이 입력과 같아야 하면, 그냥 입력을 복사해서 내보내면 되는 것 아닌 가?" 병목 때문에 못 합니다. 16 개 숫자로 120 개를 복사할 수 없습니다. 병목을 없애고 120→120 신경망을 만들면 모델은 진짜로 복사만 배우고, 그 순간 이상탐지 능력이 0 이 됩니다. 병목의 좁기가 이 기술의 전부입니다.

초보가 헷갈리는 지점 2. 디코더의 마지막 층에는 ReLU 가 없습니다. 이 신경망에 들어가는 숫자는 원래 센서값이 아니라 평균이 0 이 되도록 눈금을 바꾼 값이기 때문입니다(어떻게 바꾸는지는 아래 "재구성 오차와 임계치" 항목에서 정규화로 설명합니다). 평균이 0 이면 값의 절반이 음수입니다. 마지막 에 ReLU 를 붙이면 그 음수 영역을 전부 0 으로 깎아 버려서 복원이 원리적으로 불가능해집니다.



원문 도해 · 120→64→16→64→120 오토인코더 (원문 203 쪽)

왜 정상 데이터만으로 학습하는가

지도 학습(Supervised Learning)은 "이건 정상, 이건 고장"이라는 정답표(label, 라벨)를 붙인 데이터 로 훈련하는 방식입니다. 강아지 사진에 "강아지"라고 써 붙여 학습시키는 것과 같습니다. 이 방식이 공장에서 거의 안 통하는 이유는 명확합니다.

고장 데이터가 없습니다. 잘 운영되는 설비일수록 고장이 드뭅니다. 수천만 건의 정상 데이터에 고장이 열 건이면 학습이 성립하지 않습니다. 고장 종류를 미리 다 알 수 없습니다. 지도 학습은 학습 때 본 고장만 알아봅니다. 처음 보는 고장 앞에서는 "정상"이라고 답합니다. 라벨을 붙이려면 사람이 과거 데이터 전부를 열어 보고 판정해야 합니다. 현실적으로 불가능합니다.

비지도 학습(Unsupervised Learning)은 정답표 없이 데이터의 구조만 배웁니다. 오토인코더는 여기에 속합니다. 학습에는 정상 운전 데이터만 넣습니다. 고장 데이터는 한 건도 필요 없습니다.

논리는 이렇습니다. 모델은 정상 운전의 관계 구조만 배웠으므로, 정상 입력은 잘 복원합니다. 반대로 관계가 무너진 입력이 들어오면 배운 적 없는 모양이라 제대로 복원하지 못합니다. 설계 보고서의 표현 그대로입니다. "공정에 비정상 징후가 발생하면 학습된 정상 패턴과의 괴리로 인해 모델이 입력 신호를 제대로 복원하지 못하며, 이로 인해 원본 신호와 복원 신호 간의 재구성 오차(Reconstruction Error)가 급격히 증가한다."

그래서 이 방식은 처음 보는 고장도 잡습니다. 무엇이 고장인지 배운 적이 없고, 오직 정상이 어떤 모습인지만 배웠기 때문입니다. 이것이 오토인코더가 설비 이상탐지의 표준 도구가 된 이유입니다.

초보가 헷갈리는 지점. "정상 데이터에 고장 구간이 조금 섞여 있으면 어떻게 되나?" 모델이 그 고장까지 "정상"으로 배웁니다. 그래서 학습 데이터 구간을 고르는 일이 모델 구조를 고르는 일보다 중요합니다. 실제 공장에서는 정비 이력과 생산 기록을 대조해 "이 기간은 확실히 정상"인 구간만 뽑아 씁니다.

재구성 오차와 임계치

정규화(normalization)가 먼저입니다. 태그마다 단위와 크기가 다릅니다. 온도는 70 언저리, 압력은 3 언저리, pH 는 7.4 언저리입니다. 이대로 넣으면 숫자가 큰 태그가 오차를 독차지합니다. 그래서 각 태그를 z-정규화합니다. 학습 데이터의 평균 μ 와 표준편차 σ 로 빼고 나눠, 모든 태그를 "평균 0, 표준편차 1"의 같은 눈금으로 바꿉니다.

$$z_i = (x_i - \mu_i) / \sigma_i$$

정규화를 거치면 값이 대체로 -3에서 +3 사이에 놓이고 절반가량이 음수가 됩니다. 앞에서 디코더 마지막 층에 ReLU를 붙이면 안 된다고 한 이유가 이것입니다.

여기서 결정적인 규약이 하나 붙습니다. μ 와 σ 는 학습 때 계산해서 저장해 두고, 추론할 때 똑같은 값을 씁니다. 추론 시점에 그때그때 다시 계산하면 학습 분포와 추론 분포가 어긋나 모델이 무의미해집니다. 여기서 분포(distribution)는 값들이 어느 구간에 얼마나 몰려 있는지의 모양을 뜻합니다. 같은 pH 7.4라도 "평균 7.4, 폭 0.1인 분포"에서 나온 7.4와 "평균 7.9, 폭 0.3인 분포"에서 나온 7.4는 의미가 전혀 다릅니다. 모델은 학습할 때 본 분포 위에서만 옹골치게 동작하므로, 눈금을 바꾸는 계수도 그때 것을 그대로 써야 합니다. AR-100은 이 계수를 `model_meta.json`에 넣어 모델 파일과 함께 배포합니다.

재구성 오차는 평균제곱오차(MSE, Mean Squared Error)입니다. 입력 벡터 z 와 복원 벡터 $\hat{z}$ (z-햇)의 차이를 각 자리마다 제곱해서 전부 더한 뒤 자릿수로 나눕니다.

$$D = \text{윈도우 } 10 \text{ 스텝} \times \text{태그 } 12 \text{ 개} = 120$$

$$E = \frac{1}{D} \cdot \sum_{i=1}^D (z_i - \hat{z}_i)^2$$

판정: $E > \theta$ 이면 이상

제공하는 이유는 두 가지입니다. 부호를 없애기 위해서(위로 벗어나든 아래로 벗어나든 벗어남입니다), 그리고 크게 벗어난 자리에 더 큰 벌점을 주기 위해서입니다.

임계치 θ (세타)를 어떻게 정하는가. 이것이 실무에서 가장 자주 틀리는 지점입니다. 임의로 "0.05 쯤"이라고 정하면 안 됩니다. AR-100은 데이터가 정하게 합니다.

1. 학습이 끝난 모델에 검증용으로 떼어 둔 15%를 넣어 재구성 오차 분포를 만듭니다. 2. 그 분포의 99.5 백분위(percentile)를 구합니다. 백분위는 "이 값보다 작은 데이터가 전체의 몇 %인가"를 뜻합니다. p99.5는 그 데이터의 99.5%가 이 값 아래에 있다는 뜻입니다. 3. 거기에 안전계수(safety factor) 1.15를 곱합니다. 15%의 여유를 더 줘서 오탐(false positive, 정상을 이상이라 하는 것)을 억제합니다.

$$\theta = \text{percentile}(\text{정상 재구성오차}, 99.5) \times 1.15$$

초보가 헷갈리는 지점 1 — "검증용은 모델이 본 적 없는 데이터다"는 이 구현에서는 성립하지 않습니다. 교과서는 보통 그렇게 설명하지만, `ml/train.py`의 분할은 이렇게 되어 있습니다.

```
idx = np.random.permutation(len(X))
split = int(len(X) * (1 - m["val_split"]))
Xtr = torch.from_numpy(X[idx[:split]])
Xva = torch.from_numpy(X[idx[split:]])
```

여기서 X 의 한 줄은 1초씩 밀면서 잘라 낸 10초짜리 창입니다. 이웃한 두 창은 10스텝 중 9스텝이 같습니다. 그것을 무작위로 섞어 나눴으니, 검증 창 하나는 거의 언제나 학습 창과 대부분을 공유합니다. 시간 축으로 앞뒤를 자른 것이 아니라서 검증 집합은 시간적으로 분리돼 있지 않습니다. 따라서 여기서 나온 오차 분포는 실제보다 낙관적일 수 있고, 임계치도 그만큼 작게 나올 수 있습니다. 실무에서 시계열을 다룰 때 앞 85%로 학습하고 뒤 15%로 검증하는 이유가 이것입니다.

그럼 이 예제의 임계치는 못 믿을 값인가. 다행히 아닙니다. 별도로 학습에 전혀 쓰지 않은 새 정상 구간 400초를 다시 생성해 오탐률을 재는 자가검증이 파이프라인에 들어 있고(아래에서 설명합니다), 거기서 오탐률이 0.0~0.5%로 나왔습니다. 임계치 산출 과정에 낙관 편향이 있더라도 결과적으로 쓸 만한 값이었음을 독립된 데이터로 교차 확인한 셈입니다. 이 두 단계를 구분해서 이해해야 합니다.

초보가 헷갈리는 지점 2. "백분위를 99.9로 올리면 오탐이 더 줄어드니 좋은 것 아닌가?" 오탐과 미탐(false negative, 이상을 놓치는 것)은 맞바꾸는 관계입니다. 임계치를 올리면 오탐은 줄지만 약한 이상도 통과합니다. 내리면 반대입니다. 정답은 없고, 현장이 감당 가능한 알람 건수와 놓쳐서는 안 되는 고장의 크기를 놓고 정하는 운영 판단입니다.

이상 기여도(contribution) 산출. "이상입니다"만 뜨면 운전원은 아무것도 할 수 없습니다. 어느 센서 때문인지를 함께 내야 합니다. 방법은 단순합니다. 오차를 전부 합치기 전에, 태그 차원별로 따로 쌓아 둡니다.

$$c_t = \sum_{s=0}^9 (z[s \cdot 12 + t] - \hat{z}[s \cdot 12 + t])^2 \quad t = \text{태그 번호}$$

기여율_t = $c_t / \sum c$ → 상위 3개를 알람 메시지에 붙인다

120 차원 벡터는 12 개 태그가 10 번 반복된 배열이므로, 인덱스를 12 로 나눈 나머지가 태그 번호가 됩니다. 실제 코드도 정확히 그 한 줄입니다.

```
perTag[i % tagOrder.length] += d * d; // 태그 차원별 누적
```

설계 보고서가 요구한 "이상 기여도가 높은 특정 센서 차원을 역추적한다"가 이 계산입니다.

시간을 어떻게 넣는가, 그리고 LSTM 오토인코더

지금까지 설명한 오토인코더는 한 시점의 12 개 값을 보는 구조입니다. 그런데 설비 이상에는 시간의 모양이 있습니다. 온도가 70 도인 것 자체는 정상이지만, 30 초 만에 65 도에서 70 도로 올라갔다면 비정상일 수 있습니다. 순간값만 보면 이 차이가 사라집니다.

시간을 넣는 방법은 두 갈래입니다.

방법 A — 슬라이딩 윈도우 평탄화(flatten). 최근 10 개 스캔을 모아 $12 \times 10 = 120$ 개 숫자를 한 줄로 이어 붙여 하나의 입력으로 씁니다. 이러면 일반 Dense 오토인코더가 그대로 시간 패턴을 배웁니다. "3 스텝 전의 온도와 지금의 압력 관계" 같은 것도 그냥 두 차원 사이의 관계가 되기 때문입니다. AR-100 이 택한 방법입니다. 슬라이딩(sliding)은 새 값이 하나 들어오면 가장 오래된 값을 하나 밀어내며 창이 앞으로 미끄러진다는 뜻입니다.

방법 B — LSTM 오토인코더. LSTM 은 장단기 기억(Long Short-Term Memory)의 약자로, 순서가 있는 데이터를 위해 만들어진 신경망 층입니다. 값을 하나씩 순서대로 먹으면서 내부에 "기억 상태"를 유지 하고, 무엇을 기억하고 무엇을 잊을지를 스스로 조절합니다. LSTM 오토인코더는 인코더 LSTM 이 시퀀스를 끝까지 읽어 마지막 기억 상태를 잠재 벡터로 삼고, 디코더 LSTM 이 그걸로 시퀀스 전체를 되뱉어냅니다. PDF 도 "Autoencoder 및 시계열 특화 LSTM Autoencoder 가 널리 채택된다"고 두 가지를 나란히 적고 있습니다.

비교 항목	윈도우 평탄화 + Dense AE	LSTM AE
시간 표현	고정 길이 창을 통째로 한 벡터로	순차 입력, 내부 기억 상태로
긴 의존 관계	창 길이를 넘어서면 못 봄	수백 스텝까지 추적 가능
가변 길이 입력	불가(차원 고정)	가능
파라미터 수·학습 시간	작고 빠름	크고 느림
추론 지연	행렬곱 몇 번, 1 ms 미만	스텝 수만큼 순차 계산, 더 큼
ONNX 변환	단순, 문제 거의 없음	순환 연산자 지원 확인 필요

일반적인 선택 기준은 고장이 전개되는 시간 규모입니다. 이상이 수 초~수십 초 안에 모양을 드러내면 짧은 창으로 충분하고, 수 시간에 걸쳐 서서히 번지는 열화라면 LSTM 쪽이 유리합니다.

다만 AR-100 이 창 길이를 10 으로 정한 이유는 기록에 없습니다. 저널에도 ADR 에도 근거가 남아 있지 않고, ml/train.yaml 과 flink/onnx-job/job.properties 에 값만 적혀 있습니다. 참고로 이 시스템의 고장 시나리오 기본 지속시간은 simulator/plant.yaml 기준으로 spike 8 초, dropout 15 초, noise 30 초, bearing_wear 45 초, drift 90 초입니다. 창 10 초가 이 범위 전체를 덮지는 않습니다. 그래서 이 교본은 "10 초면 충분하다"고 단정하지 않고, 대신 뒤의 실험표에서 창을 10 에서 1 로 줄여 무엇을 놓치게 되는지 직접 보게 합니다.

학습 데이터에 운전 변동이 있어야 하는 이유

이 항목은 교과서에 잘 안 나오는데, 실패하면 모델 전체가 무의미해지는 함정입니다.

오토인코더가 배우는 것은 태그들이 함께 움직이는 규칙입니다. 그런데 공정의 설정치(SP, Set Point — 운전원이 지정하는 목표값)를 고정해 두고 정상 데이터를 뽑으면, 모든 태그가 각자의 고정값 주위에서 노이즈만큼만 떨린 채 있습니다. 이때 태그 사이의 상관계수는 0 에 수렴합니다.

상관계수(correlation coefficient)는 두 값이 함께 움직이는 정도를 -1 에서 +1 사이의 숫자로 나타낸 것입니다. +1 은 완전히 같은 방향, -1 은 완전히 반대 방향, 0 은 아무 관계 없음입니다.

상관이 0 이면 신경망이 배울 구조가 없습니다. 압축해서 남길 만한 관계가 없으니, 모델이 배울 수 있는 것은 각 태그의 평균값뿐입니다. 그리고 평균에서 얼마나 벗어났는지를 재는 장치는 이미 있습니다. 바로 규칙 계층의 Z-Score 입니다. 즉 비싼 딥러닝 모델을 만들어 놓고 Z-Score 와 똑같은 일을하게 되는 결과가 됩니다. ML 계층의 존재 이유 자체가 사라집니다.

해결은 학습 데이터에 정상 범위의 운전 변동을 넣는 것입니다. 실제 플랜트는 생산 스케줄에 따라 설정치가 계속 바뀝니다. 그 변동이 있어야 "펌프 속도를 올렸더니 유량이 오르고 전류도 올랐다"는 인과의 흔적이 데이터에 남고, 모델이 그 관계를 배울 수 있습니다.

초보가 헛갈리는 지점. "그럼 변동을 크게 줄수록 좋은가?" 아닙니다. 변동을 너무 키우면 정상 범위가 넓어져서 진짜 고장이 정상 변동에 묻힙니다. 너무 작으면 상관이 안 생기고, 너무 크면 탐지력이 죽습니다. 이 균형은 이론으로 맞출 수 없어서, AR-100은 학습 직후 판별력을 스스로 측정하는 검증 단계를 파이프라인에 넣어 강제로 확인하게 했습니다.

AR-100에서는, 학습 설정은 `ml/train.yaml` 한 파일에 모여 있습니다. 코드를 고치지 않고 이 파일만 바꿔 재학습합니다.

```
# — 학습 데이터 생성 —
# 실제 운영이라면 InfluxDB 의 정상 운전 이력을 끌어오지만,
# 자립 실행을 위해 동일한 물리 모델로 정상 구간을 합성한다.
data:
  warmup_s: 1800          # 초기 과도구간 폐기
  duration_s: 21600       # 6 시간 상당 정상 운전
  dt_s: 1.0
  autopilot: true        # 설정치 변동 주입
model:
  window_steps: 10        # 슬라이딩 윈도우 길이 (Flink job.properties 와 반드시 일치)
  hidden: [64, 16]
  epochs: 60
  batch_size: 256
  learning_rate: 0.001
  val_split: 0.15
  seed: 42
threshold:
  percentile: 99.5        # 정상 데이터 재구성 오차의 상위 백분위
  safety_factor: 1.15     # 오탐 억제 여유
output:
  model: /opt/models/model.onnx
  meta: /opt/models/model_meta.json
```

맨 위 주석을 그냥 넘기면 안 됩니다. 이 학습 데이터는 실제 공장에서 수집한 6 시간치 이력이 아닙니다. 설계 보고서가 그리는 정석은 이렇습니다. "오프라인 학습 단계에서는 InfluxDB 또는 장기 데이터 레이크에 누적된 정상 운전 이력 데이터를 Python(PyTorch, TensorFlow, Scikit-learn) 기반 분산 학습 클러스터로 로드하여 Autoencoder 모델을 훈련시킨다." AR-100은 그렇게 하지 않습니다. 이 예제 가 docker compose up 한 번으로 남의 데이터 없이 돌아가야 하기 때문에, 운영 시뮬레이터와 똑같은 물리 모델을 학습기 안에서 다시 돌려 정상 구간을 합성합니다. `duration_s: 21600`은 실제로 6 시간을 기다린다는 뜻이 아니라 시뮬레이션 시간으로 6 시간(21,600 스캔)을 진행시킨다는 뜻이고, 데이터 생성과 60 에폭 학습을 합쳐 실제로는 약 90 초에 끝납니다(docs/journal/09-metrics.md).

합성이라도 학습 분포와 운영 분포가 어긋나지 않게 못을 박아 두었습니다. 학습기 이미지는 물리 모델의 복사본이 아니라 시뮬레이터 원본 파일을 그대로 가져갑니다.

```
# ml/Dockerfile
# 물리 모델을 복사본이 아니라 simulator/ 원본에서 직접 가져와
# 학습 분포와 운영 분포가 어긋나지 않게 합니다.
COPY ml/train.py ml/train.yaml ./
COPY simulator/plant.py simulator/plant.yaml ./
```

나머지 항목을 하나씩 풀면 이렇습니다. `warmup_s: 1800`은 기동 직후 30 분을 버린다는 뜻입니다. 공정이 아직 정상 상태에 들지 않은 과도 구간이 학습에 섞이면 안 됩니다. `epochs: 60`은 전체 데이터를 60 번 반복 학습, `batch_size: 256`은 한 번에 256 개씩 묶어 가중치를 한 번 갱신, `val_split: 0.15`는 15%를 검증용으로 떼어 둔다는 뜻입니다. 학습 창이 21,591 개이므로 한 에폭 에 약 72 번, 60 에폭이면 약 4,300 번 가중치가 갱신됩니다(계산값).

실제 학습 로그(docs/journal/logs/05-model-training.txt)입니다.

```
[trainer] 정상 운전 데이터 21,600 행 × 12 태그 생성 (autopilot=True)
[trainer] 학습 텐서 21,591 × 120 (윈도우 10 스텝 × 12 태그)
[trainer] epoch 1/60 train=0.580739 val=0.187217
[trainer] epoch 10/60 train=0.031756 val=0.029991
[trainer] epoch 20/60 train=0.020462 val=0.020632
[trainer] epoch 60/60 train=0.019235 val=0.019580
[trainer] ONNX 저장 완료: /opt/models/model.onnx (0.07 MB)
```

"21,600 행 생성"은 실측 수집이 아니라 시뮬레이터가 만들어 낸 행 수입입니다. 21,600 행에서 윈도우 10 을 잡으면 21,591 개의 창이 나옵니다($21600 - 10 + 1$).

정상 데이터에 운전 변동이 필요한 이유

1 권에서 설명한 autopilot 은 240 초마다 목표를 다시 정하고 45 초에 걸쳐 이동시키며, 수동 조작한 설정치는 300 초 보호합니다. 일정한 운전점만 유지하면 학습할 관계가 충분히 드러나지 않습니다. 원문에 기록된 pH-101↔CT-101 상관은 변동 주입 전 -0.007, 주입 후 -0.934 였습니다. "전" 값이 기록된 쌍은 이것 하나뿐이므로 나머지 상관값을 모두 전후 비교로 읽으면 안 됩니다. 전체 정상 상관표는 1 권 1 장에 있습니다.

원문 211 쪽에는 변동 후 TT-101↔PT-101 상관 +0.173 도 기록되어 있습니다. 헤드스페이스 압력은 온도와 액면에 함께 묶이므로, 한 태그만의 고정된 함수로 설명하지 않습니다.

이 결정은 docs/journal/08-decisions.md 에 ADR-007 · 학습 데이터에 운전 변동 주입(autopilot) 으로 기록돼 있습니다. ADR 은 아키텍처 결정 기록(Architecture Decision Record)의 약자로, "왜 이렇게 정했는가"를 남기는 문서 형식입니다. 이 ADR 의 위험 항목 원문은 이렇습니다. 괄호까지 그대로 옮깁니다.

변동을 너무 크게 주면 고장 신호가 정상 변동에 묻힌다. 실제로 1 차 시도에서 정상 오탐 49% 가 나왔다(별도 원인도 있었음). 이를 막기 위해 학습 직후 판별력 자가검증을 파이프라인에 포함시켰다.

괄호 안의 "별도 원인도 있었음"이 핵심입니다. 그 49%의 실제 원인은 변동 폭이 아니라 검증 코드 의 버그였습니다. 바로 다음 항목에서 다룹니다.

판별력 자가검증이 왜 파이프라인 안에 있어야 하는가

머신러닝 프로젝트에서 가장 흔한 착각은 "학습 손실이 낮으니 모델이 좋다"입니다. 이걸 틀린 판단입니다. 앞에서 정의했듯 여기서 손실은 복원값이 입력에서 얼마나 벗어났는지를 잴 숫자일 뿐입니다. 손실이 낮다는 것은 정상 데이터를 잘 복원한다는 뜻이지 이상을 구별한다는 뜻이 전혀 아닙니다. 모든 입력을 평균값으로 뺀 모델도 손실은 꽤 낮게 나옵니다.

모델이 쓸 만한지 판정하려면 최소 두 숫자를 함께 봐야 합니다.

정상 오탐률 — 학습에 쓰지 않은 새 정상 데이터를 넣었을 때 임계치를 넘는 비율. 낮을수록 좋습니다. 시나리오별 탐지율 — 각 고장을 주입했을 때 임계치를 넘는 비율. 담당 고장에서는 높아야 하고, 다른 방식이 담당하는 고장에서는 낮아도 정상입니다.

두 번째 항목의 단서를 놓치기 쉽습니다. ML 이 모든 고장에 반응할 필요는 없습니다. 노이즈 급증은 Z-Score 의 담당이고, ML 이 거기에 반응하지 않는 것이 설계대로 동작하는 것입니다. 검증표는 "다 잡았는가"가 아니라 "각자 제 구역만 잡았는가"를 봐야 합니다.

그래서 AR-100 의 학습 스크립트는 학습이 끝나면 항상 자가검증을 실행하고 로그를 남깁니다. 사람이 따로 실행하는 별도 절차가 아니라 파이프라인의 일부입니다.

```
if __name__ == "__main__":
    main()      # 학습 + ONNX 익스포트
    validate()  # 판별력 자가검증
```

validate() 는 시나리오마다 400 초짜리 구간을 새로 생성하고, 고장을 400 초 내내 주입한 뒤 뒤쪽 절반만 평가합니다($X = X[\text{len}(X) // 2:]$ — 주석: "고장 효과가 충분히 전개된 후반부만 평가"). 이 한 줄이 뒤에서 숫자를 해석할 때 결정적이므로 기억해 두십시오.

AR-100 에서는, 첫 모델의 자가검증 결과가 이랬습니다(docs/journal/logs/05-model-training.txt 의 [1 차]).

```
[trainer] 정상 재구성오차: 중앙 0.016226  p99.5 0.070590
[trainer] 이상 임계치 = 0.081178 (여유계수 1.15)
[trainer] — 모델 판별력 자가검증 —
[trainer] 정상      평균오차 0.06669  탐지율 49.0%   (오탐률)
[trainer] drift      평균오차 0.05256  탐지율 43.9%
[trainer] bearing_wear  평균오차 0.05714  탐지율 50.5%
[trainer] noise      평균오차 0.06053  탐지율 53.1%
[trainer] spike      평균오차 0.06207  탐지율 58.2%
[trainer] 임계치 = 0.03672
```

정상 오염률 49%입니다. 정상 운전 중에 절반 가까이 알람이 울립니다. 더 나쁜 것은 drift 탐지율 43.9%로, 정상보다 낮습니다. 이 모델은 동전 던지기만도 못합니다. 그런데 학습 손실은 $val=0.019580$ 으로 멀쩡했습니다. 손실만 봤다면 이 모델을 그대로 배포했을 것입니다.

원인은 모델도 아니고 변동 폭도 아니고, 검증 코드 한 줄이었습니다.

```
# 잘못된 — 태그마다 step() 을 호출
rows = np.array([plant.step(1.0)[t] for t in tags] for _ in range(seconds))
```

안쪽 반복이 태그 12 개를 돌면서 plant.step() 을 12 번 부릅니다. step() 은 물리 시뮬레이션의 시간을 1 초 전진시키는 함수라서, 결과가 두 가지로 망가집니다. 첫째, 한 행이 서로 다른 12 개 시각의 값으로 조합돼 상관 구조가 파괴됩니다. 둘째, 시뮬레이션 시간이 12 배속으로 흘러 고장 램프(서서히 커지는 고장의 전개)가 왜곡됩니다.

```
# 올바른 — 스캔당 정확히 한 번
for i in range(seconds):
    r = plant.step(1.0)
    rows[i] = [r[t] for t in tags]
```

수정 후 검증 결과입니다([2 차]).

```
[trainer] — 모델 판별력 자가검증 —
[trainer] 정상      평균오차 0.02053   탐지율 0.5%   (오염률)
[trainer] drift      평균오차 55.76546   탐지율 100.0%   탐지되어야 함 (단일 임계치로는 불가)
[trainer] bearing_wear 평균오차 146.13240   탐지율 100.0%
[trainer] noise      평균오차 0.03104   탐지율 3.6%   Tier-1 롤링 Z-Score 담당
[trainer] spike      평균오차 17.42750   탐지율 100.0%
[trainer] 임계치 = 0.07007
```

drift의 평균 재구성오차가 0.05256에서 55.77로 뛰었습니다. 다만 "모델은 하나도 바꾸지 않았다"고 말할 근거는 없습니다. 임계치가 0.03672에서 0.07007로 바뀌었기 때문입니다. 임계치는 validate()가 아니라 학습 본체 main()이 산출해 model_meta.json에 쓰는 값이고, validate()는 그 파일을 읽기만 합니다. 즉 두 블록 사이에 재학습이 있었다는 뜻입니다. 고쳐진 것은 검증 코드지만, 비교된 두 결과는 같은 모델 파일이 아닙니다.

주의 — 여기 나오는 임계치가 여러 개인 이유. 이 장에서 임계치 숫자가 여섯 번 나옵니다. 어느 것이 이 시스템의 값인지 한 표로 정리합니다.

값	무엇의 값인가	출처
0.081178	[1 차] 학습 본체가 산출한 값. $p99.5 = 0.070590 \times 1.15$. 시드 미고정	logs/05-model-training.txt
0.03672	[1 차] 자가검증이 읽어 쓴 메타 파일의 값	같은 로그
0.07007	[2 차] 검증 코드 수정 후 자가검증이 읽은 값	같은 로그
0.081993	[3 차] 진동 지연 모델 반영 후 학습 본체 산출값	같은 로그
0.02913	라이브 알람을 찍던 시점에 Flink 잡이 적재하고 있던 메타 파일의 값	logs/06-detection-results.txt
0.035081	현재 기준값. 시드 3종 고정 후, 2회 연속 실행에서 동일 재현 확인	docs/journal/09-metrics.md

0.081178과 0.03672가 같은 [1 차] 블록 안에 있는데 서로 다른 것이 이상해 보입니다. 코드상 한 번의 실행에서는 두 값이 같을 수밖에 없습니다. validate()가 읽는 메타 파일을 바로 앞의 main()이 방금 썼기 때문입니다. 따라서 이 로그 파일은 여러 차례 실행에서 뽑은 대표 줄을 한 블록에 모아 둔 기록이라고 읽어야 합니다. 저널이 그렇게 정리해 둔 것이지만 한 번의 실행 결과가 아닙니다.

이 장에서 숫자를 인용할 때는 어느 실행의 값인지를 매번 붙입니다. 현재 배포 기준을 묻는다면 답은 0.035081 하나입니다.

주의. 모델 검증 코드의 버그는 모델 버그처럼 보입니다. "모델이 나쁘다"고 결론 내리기 전에 데이터 파이프라인부터 의심해야 합니다. 이 사건에서 결정적 단서는 정상 오염률 49%라는 비정상적으로 높은 값이었습니다. 모델이 나쁘더라도 보통은 이 정도까지 되지 않습니다. 숫자가 "그냥 나쁜" 수준이 아니라 "말이 안 되는" 수준이면 모델이 아니라 배관을 봐야 합니다.

재현성과 시드 고정의 함정

시드(seed)는 난수 생성기의 출발점입니다. 컴퓨터의 난수는 진짜 무작위가 아니라 출발점에서 정해진 규칙으로 뻗어 나가는 수열이므로, 출발점을 같게 하면 매번 똑같은 난수열이 나옵니다. 교재나 검증에서는 이 재현성이 필수입니다. 실행할 때마다 임계치가 달라지면 "이 숫자를 확인하세요"라고 쓸 수가 없습니다.

함정은 난수 생성기가 하나가 아니라는 점입니다. 파이썬 한 프로세스 안에도 최소 셋이 따로 돕니다.

`torch.manual_seed()` — 파이토치가 쓰는 난수. 신경망 가중치 초기화와 에폭마다 순서를 섞는 `torch.randperm` 이 여기 걸립니다. `np.random.seed()` — NumPy 난수. 학습/검증 분할(`np.random.permutation`)이 여기 걸립니다. `random.seed()` — 파이썬 표준 `random` 모듈.

여기서 "드롭아웃"이라는 단어를 조심해야 합니다. 파이토치 문맥의 드롭아웃(dropout)은 학습 중에 신경망의 뉴런 일부를 무작위로 잠시 꺼서 모델이 특정 경로에만 의존하지 않게 만드는 과적합 방지 기법입니다. 그런데 이 시스템의 고정 시나리오 중에도 dropout 이 있습니다. 그쪽은 센서가 값 발행 을 아예 멈추는 통신 두절 시나리오입니다(`plant.yaml`: "TT-101 발행 중단 → Flink 결측치 보간 검증"). 이름만 같고 뜻이 전혀 다릅니다. 덧붙이면 AR-100 의 오토인코더에는 드롭아웃 층이 없습니다 (위 모델 정의에 `nn.Dropout` 이 없습니다). 그러니 이 프로젝트에서 dropout 이라는 말이 나오면 언제나 센서 발행 중단 쪽입니다.

AR-100 은 처음에 시드 셋 중 앞의 둘만 고정했습니다. 그런데 물리 시뮬레이터의 계측 노이즈가 표준 `random.gauss()` 를 쓰고 있었습니다(`simulator/plant.py` 의 `_measure()`). 결과적으로 학습 데이터 자체가 실행마다 달라졌고, 거기서 뽑는 임계치도 따라 흔들렸습니다.

실행마다 임계치가 0.070 ~ 0.082 사이에서 흔들렸습니다. 최소와 최대가 17% 가까이 차이 납니다. 모델 구조도, 설정도, 데이터 생성 시간도 전부 같은데 숫자만 달랐습니다. 세 개를 모두 고정한 뒤에야 2 회 연속 실행에서 임계치·오차·탐지율이 전부 동일함을 확인했습니다.

```
# 물리 모델의 계측 노이즈는 표준 random 모듈을 쓰므로 함께 고정해야
# 학습 결과(특히 이상 임계치)가 실행마다 재현된다.
torch.manual_seed(m["seed"])
np.random.seed(m["seed"])
random.seed(m["seed"])
```

초보가 헛갈리는 지점. "재현이 안 되면 모델이 틀린 것인가?" 아닙니다. 흔들림 자체는 정상입니다. 문제는 흔들림의 폭을 모른 채로 하나의 숫자를 정본이라고 문서에 적는 것입니다. 시드를 고정하거나, 고정하지 않겠다면 여러 번 돌려 분포로 적어야 합니다.

ONNX와 임베디드 서빙

먼저 자바 쪽 용어 넷을 풀어 돕니다. 이 항목의 모든 문장이 이 넷 위에서 있습니다.

스트림 엔진(stream engine) — 끊임없이 흘러드는 데이터를 저장하지 않고 흐르는 채로 처리하는 프로그램입니다. 이 시스템에서는 Apache Flink 가 그것입니다. JVM(Java Virtual Machine, 자바 가상 머신) — 자바로 만든 프로그램이 실제로 돌아가는 실행 한 경입니다. 자바 프로그램은 운영체제 위에서 바로 도는 게 아니라 JVM이라는 한 겹을 거쳐 돕니다. Flink 는 자바로 만들어졌으므로 Flink 가 도는 자리가 곧 JVM 프로세스입니다. TaskManager — Flink 는 지휘하는 프로세스와 실제로 계산하는 프로세스가 따로 있습니다. 지휘하는 쪽이 JobManager, 실제 데이터를 받아 계산하는 작업자가 TaskManager 입니다. 우리 모델이 적재되는 곳이 바로 이 TaskManager 의 JVM 안입니다. 슬롯(slot) — TaskManager 하나가 동시에 맡을 수 있는 작업 자리의 개수입니다. 슬롯 하나가 잡 의 한 갈래를 맡아 돌립니다. AR-100 은 `numberOfTaskSlots: 8`, `parallelism.default: 2` 로 설정돼 있습니다(`flink/conf/config.yaml`). 즉 자리는 8 개 있고 잡 하나는 기본 2 갈래로 쪼개 돌립니다.

ONNX(Open Neural Network Exchange)는 학습된 신경망을 프레임워크에 묶이지 않는 형태로 저장하는 공개 표준 바이너리 포맷입니다. 직역하면 "열린 신경망 교환 규격"입니다.

왜 필요한가부터 봅니다. PyTorch 로 학습한 모델을 그대로 저장하면 그 파일은 PyTorch 로만 열 수 있습니다. TensorFlow 로 학습했으면 TensorFlow 로만 열립니다. 학습 환경과 운영 환경이 같은 언어·같은 프레임워크라면 문제가 없는데, 스트림 처리 파이프라인에서는 거의 항상 다릅니다. 학습은 Python 생태계가 압도적으로 편하고, 실시간 추론은 스트림 엔진이 사는 JVM 안에서 해야 빠르기 때문입니다.

ONNX 는 신경망을 연산 그래프로 기술합니다. "이 입력에 이 가중치 행렬을 곱하고, ReLU 를 거치고, 다시 곱하고..."를 프레임워크 중립적인 연산자(operator) 목록으로 적어 둔 파일입니다. 그러면 ONNX Runtime 이라는 실행 엔진이 어느 언어에서든 그 그래프를 그대로 실행할 수 있습니다. ONNX Runtime 은 C++로 구현돼 있고, Python·Java·C#·JavaScript 용 바인딩을 제공합니다.

opset 이라는 개념이 같이 나옵니다. ONNX 연산자 집합의 버전 번호입니다. 새 버전일수록 표현할 수 있는 연산이 많지만, 실행 엔진이 그 버전을 알아야 열 수 있습니다. 학습 쪽에서 내보낸 opset 과 추론 쪽 런타임 버전이 어긋나면 모델이 열리지 않습니다. AR-100 은 `opset_version=17` 로 내보냅니다.

추론을 어디서 하는가 — 두 가지 방식

방식 1: RPC Callout. RPC는 원격 프로시저 호출(Remote Procedure Call)의 약자로, 다른 프로세스나 다른 서버의 함수를 네트워크 너머로 부르는 방식입니다. 모델을 별도 서버로 띄워 두고, 스트림 엔진이 데이터가 들어올 때마다 그 서버를 호출해 점수를 받아 옵니다. 그 서버를 만드는 방식이 보통 둘입니다.

REST(Representational State Transfer) API — 웹 브라우저가 쓰는 것과 같은 HTTP로 주소를 호출하고 JSON 형식의 답을 받는, 가장 흔하고 만들기 쉬운 방식입니다. 사람이 읽을 수 있는 텍스트라 디버깅이 편한 대신 크기가 크고 파싱이 느립니다. gRPC — 구글이 만든 RPC 규격입니다. HTTP/2 위에서 Protocol Buffers라는 이진 형식으로 주고받습니다. 텍스트가 아니라 뽀뽀한 바이트라 REST보다 작고 빠르며, 주고받을 항목을 규격 파일에 미리 못 박아 두므로 형식이 어긋나면 컴파일 단계에서 걸립니다. 대신 사람이 눈으로 읽을 수 없습니다.

어느 쪽이든 대가는 같습니다. 네트워크 왕복 지연이 매 건마다 붙고, 모든 스트림 태스크가 그 하나의 API 서버로 몰려 병목이 됩니다.

방식 2: 임베디드 서빙(In-Stream Embedded Serving). 모델을 스트림 엔진의 프로세스 안에 직접 적재합니다. 추론은 그냥 같은 프로세스 안의 함수 호출입니다. 네트워크 홉이 0입니다. 설계 보고서가 명확히 권고하는 쪽입니다. "실시간 온라인 추론 단계에서는 외부 REST/gRPC API 서버를 호출하는 방식(RPC Callout)을 지양해야 한다. 네트워크 홉으로 인한 지연시간 증가와 API 서버 병목 현상을 방지하기 위해, Apache Flink 태스크매니저 프로세스 내부에 C++ 기반의 고성능 ONNX Runtime 엔진을 임베디드 형태로 직접 로드한다."

그래서 학습과 추론이 갈립니다. 학습은 Python(PyTorch)에서, 추론은 Java(Flink) 안에서. 둘을 잇는 것이 ONNX 파일 하나입니다. 설계 보고서가 이것을 "모델의 오프라인 학습 주기와 온라인 추론 주기는 철저히 분리되어야 한다"로 표현했습니다. 분리한다는 것은 학습이 몇 시간 걸리든 추론 파이프라인은 멈추지 않는다는 뜻이기도 합니다.

초보가 헛갈리는 지점. "그러면 모델을 새로 학습할 때마다 Flink 잡을 재시작해야 하나?" 이 예제는 그렇습니다.

OnnxScorer.open()이 잡이 뜰 때 딱 한 번 모델 파일을 읽어 세션을 만들고, 그 뒤로는 파일을 다시 보지 않습니다. 운영 환경에서는 다음 장치들로 무중단 교체를 합니다.

모델 레지스트리(model registry) — 학습된 모델 파일을 이름·버전·성능 지표와 함께 보관하고 배포하는 저장소입니다. "지금 운영에 나가 있는 모델이 몇 번인가"를 한곳에서 답할 수 있게 해 줍니다. PDF도 "중앙 모델 레지스트리에 배포된다"고 그 자리를 잡아 두었습니다. 브로드캐스트 스트림(broadcast stream) — Flink에서 한 스트림의 원소를 모든 병렬 인스턴스에 똑같이 복사해 보내는 특수한 연결 방식입니다. 데이터는 슬롯별로 나눠 흐르지만, "새 모델이 나왔다"는 신호는 모든 슬롯이 다 받아야 하므로 이런 통로가 따로 필요합니다. 각 슬롯이 그 신호를 받아 새 세션으로 갈아끼우면 잡을 멈추지 않고 모델이 바뀝니다.



지연 비교 (개념도 · 축척 아님)

임베디드 호출 0.126 ~ 0.180 ms · 네트워크 홉 0
이 PC 5시간 실측 — 중앙값 0.178 · 95% 0.382 ms

외부 REST/gRPC 네트워크 왕복만 수 ms

원문 도해 · 학습 모델의 ONNX 내보내기와 Flink 내부 실행 (원문 217 쪽)

AR-100에서는, 학습 스크립트의 마지막 단계가 ONNX 익스포트입니다(`ml/train.py`).

```
torch.onnx.export(
    model,
    torch.zeros(1, dim),
    out["model"],
    input_names=["input"],
    output_names=["reconstruction"],
    dynamic_axes={"input": {0: "batch"}, "reconstruction": {0: "batch"}},
    opset_version=17,
)
```

`torch.zeros(1, dim)` 은 더미 입력입니다. 0 으로만 채운 1×120 텐서입니다. PyTorch 는 모델에 가짜 입력을 한 번 흘려 보며 연산 경로를 추적하는 방식으로 그래프를 뽑기 때문에, 입력 모양을 알려 줄 표본이 필요합니다.

`dynamic_axes` 는 0 번 축(배치 크기)만 가변으로 두겠다는 선언입니다. 학습 때는 256 개씩 묶어 넣지만 추론 때는 1 개씩 넣으므로, 이 선언이 없으면 배치 1 로 부를 때 형상 불일치로 실패합니다. 뒤집어 말하면 1 번 축은 120 으로 못 박혔습니다. 이 사실이 뒤의 `window.steps` 이야기에서 다시 나옵니다.

함께 나가는 `model_meta.json` 에는 모델 파일만으로는 알 수 없는 정보가 들어갑니다.

```
meta = {
    "tags": tags, # 태그 순서 — 12 개 값이 벡터의 어느 자리인지
    "window_steps": m["window_steps"],
    "input_dim": dim,
    "mean": [...], "std": [...], # 학습 시 정규화 계수
    "threshold": threshold, # 학습 시 산출한 임계치
    "train_rows": int(X.shape[0]),
}
```

태그 순서가 메타에 들어 있는 이유가 중요합니다. 추론 쪽이 태그를 다른 순서로 쫓으면 모델은 오류를 내지 않고 조용히 엉뚱한 답을 냅니다. 숫자 120 개는 그대로 들어가니까요. 순서를 파일로 못 박아 두고 추론 쪽이 그걸 읽어 쓰는 구조여야 이 사고를 막습니다.

여기에 함정이 하나 있습니다. 메타 파일에 `window_steps` 가 들어 있지만, 추론 쪽은 이 값을 읽지 않습니다. 자바 잡은 창 길이를 `job.properties` 의 `window.steps` 에서 가져옵니다(`AnomalyJob` 이 프로퍼티를 읽어 `new OnnxScorer(modelPath, metaPath, steps, ...)` 로 넘깁니다). 즉 태그 순서와 정규화 계수는 메타에서, 창 길이는 설정 파일에서 옵니다. 그래서 두 값이 어긋날 수 있고, 어긋나면 잡이 실패합니다.

AR-100 에서는. 추론 쪽은 Java 로 작성된 Flink 잡입니다(

`flink/onnx-job/src/main/java/org/uengine/iiot/OnnxScorer.java`). 잡이 뜰 때 한 번 세션을 엽니다.

```
env = OrtEnvironment.getEnvironment();
OrtSession.SessionOptions opts = new OrtSession.SessionOptions();
opts.setIntraOpNumThreads(1); // Flink 슬롯당 1 스레드 — 슬롯 간 CPU 경합 방지
session = env.createSession(modelPath, opts);
```

`setIntraOpNumThreads(1)` 은 놓치기 쉬운 설정입니다. ONNX Runtime 은 기본적으로 행렬 연산 하나를 여러 스레드로 쪼개 동시에 계산합니다. 그런데 Flink 는 이미 슬롯 단위로 병렬 처리를 하고 있습니다. 슬롯 각각이 다시 멀티스레드로 갈라지면, 물리 CPU 코어 수보다 많은 스레드가 생겨 서로 CPU 를 뺏느라 문맥 전환만 늘고 오히려 느려집니다. 슬롯당 1 스레드로 묶어 두는 것이 맞습니다.

추론은 이벤트가 올 때마다 돌지 않고 1 초 주기 타이머로 돌립니다. 이것이 이 잡에서 가장 헛갈리는 설계입니다. 코드 주석에 이유가 적혀 있습니다.

```
// 처리시간 타이머로 고정 주기 텐서를 만든다. 센서마다 도착 시각이 미세하게
// 다르므로 이벤트 단위로 추론하면 같은 시점이 중복 평가된다.
if (nextFireMs.value() == null) {
    long fire = ctx.timerService().currentProcessingTime() + inferenceIntervalMs;
    nextFireMs.update(fire);
    ctx.timerService().registerProcessingTimeTimer(fire);
}
```

같은 1 초 스캔에서 나온 12 개 태그의 메시지가 완벽히 동시에 도착하지는 않습니다. 이벤트마다 추론하면 사실상 같은 시점의 상태를 12 번 평가하게 되고, 이상이 하나 있으면 알람이 12 번 나옵니다. 그래서 `processElement` 는 값을 받아 최신 상태만 갱신해 두고, 실제 판정은 `onTimer` 가 `inference.interval.ms = 1000` 마다 한 번씩 합니다. 디바이스 (`reactor-line-01`) 단위로 키를 잡으므로 타이머도 하나, 점수도 초당 한 건입니다. 실측 발행량이 `sensor.anomaly.score` 기준 1.1 건/초로 나온 것이 이 구조 때문입니다(`09-metrics.md`).

기동 직후에는 점수 자체가 나오지 않습니다. 두 개의 관문이 있습니다.

```
if (seen < tagOrder.length) return; // ① 12 태그가 한 번씩은 관측되기 전에는 추론 안 함
int n = filled.value() == null ? 0 : filled.value();
if (n < steps) {
    window.put(n, vec);
    filled.update(n + 1);
    if (n + 1 < steps) return; // ② 창이 10 칸 찰 때까지는 추론 안 함
}
```

①을 통과한 뒤에도 창에 10 칸이 차야 하므로, 타이머가 10 번 돌 때까지, 즉 최소 10 초 동안은 점수도 알람도 전혀 나오지 않습니다. 화면에 아무것도 안 뜬다고 고장 난 것이 아닙니다. 규칙 계층이 값 하나만 보고 즉시 판정할 수 있는 것과 대조되는, ML 계층의 구조적 대가입니다.

매초 도는 추론 본체입니다.

```
long t0 = System.nanoTime();
float[] recon;
try (OnnxTensor tensor = OnnxTensor.createTensor(env, new float[][]{flat});

OrtSession.Result res = session.run(Map.of(inputName, tensor))) {
    recon = ((float[][]) res.getValue()).get(0);
}
double inferMs = (System.nanoTime() - t0) / 1_000_000.0;
```

실측 추론 지연 0.126 ~ 0.180 ms 입니다(docs/journal/09-metrics.md). 1 밀리초의 5 분의 1 수준입니다. 설계 보고서가 "1 밀리초 미만의 지연시간으로 재구성 오차를 계산하여 이상 점수를 도출한다"고 쓴 그 값이 실제로 나왔습니다. 외부 추론 서버를 호출했다면 계산 시간과 무관하게 네트워크 왕복만 으로 수 밀리초가 붙습니다.

잡의 동작은 flink/onnx-job/job.properties 에서 조정합니다.

```
kafka.topic.raw          = sensor.telemetry.raw
kafka.topic.clean        = sensor.telemetry.clean
kafka.topic.score        = sensor.anomaly.score
kafka.topic.alerts       = sensor.alerts
model.path               = /opt/models/model.onnx
model.meta               = /opt/models/model_meta.json
window.steps             = 10
inference.interval.ms    = 1000
# 재구성 오차 임계치. 비우면 model_meta.json 의 학습 시 산출값(99.5 백분위)을 쓴다.
anomaly.threshold        =
```

anomaly.threshold 를 비워 두면 메타 파일의 학습 시 산출값을 쓰고, 값을 적으면 그것이 우선합니다. 현장에서 오탐이 많을 때 재학습 없이 임계치만 올려 보는 손잡이입니다. 다만 이 값도 open() 에서 한 번만 읽으므로 바꾸려면 잡을 다시 띄워야 합니다.

AR-100에서는. 텐서를 조립하는 규칙이 두 가지 방어 장치를 갖고 있습니다.

```
double[] vec = new double[tagOrder.length];
int seen = 0;
for (int i = 0; i < tagOrder.length; i++) {
    Double v = latest.get(tagOrder[i]);
    if (v != null) { vec[i] = v; seen++; }
    else           { vec[i] = mean[i]; } // 미관측 태그는 학습 평균
}
if (seen < tagOrder.length) return; // 전 태그 관측 전에는 추론 안 함
```

첫째, 아직 한 번도 값이 온 적 없는 태그는 학습 평균으로 채웁니다. 딥러닝 입력에 NaN(Not a Number, 숫자가 아님)이 들어가면 재구성 오차가 무한대로 튀어 치명적 오탐이 됩니다. 둘째, 그렇게 채웠더라도 모든 태그가 최소 한 번은 실제로 관측되기 전에는 추론 자체를 하지 않습니다. 기동 직후 반쪽짜리 벡터로 알람을 울리는 사고를 막습니다.

여기서 결측 처리 방식이 앞 장의 clean 토픽과 일부러 다릅니다. 기록용 토픽은 선형 보간으로 물리적 타당성을 우선하고, 추론용 텐서는 직전 유효값 유지(LOCF, Last Observation Carried Forward)로 지연을 우선합니다. 선형 보간은 공백이 닫힐 때까지 기다려야 값이 확정되는데, 추론 은 기다릴 수 없기 때문입니다. 하나의 결측 정책을 전 구간에 강제하면 둘 중 하나가 망가집니다.

AR-100에서는. 탐지 결과가 어디로 가는지 파이프라인의 마지막 조각입니다. 추론은 점수를 내 는 데서 끝나지 않습니다.

설계 보고서는 이렇게 적었습니다. "이상으로 판정된 이벤트는 Kafka 의 별도 알림 토픽(sensor.alerts)으로 재발행되어 운영자 대시보드로 즉시 전파된다." AR-100 은 그대로 구현했습니다. 잡의 배선은 AnomalyJob.java 에 있습니다.

```
SingleOutputStreamOperator<AnomalyScore> scored = clean
    .keyBy(r -> r.device)
    .process(new OnnxScorer(modelPath, metaPath, steps, inferMs, threshold));
scored.sinkTo(jsonSink(bootstrap, scoreTopic, "score")); // sensor.anomaly.score
scored.getSideOutput(OnnxScorer.ALERT_TAG)
    .sinkTo(jsonSink(bootstrap, alertTopic, "ml-alert")); // sensor.alerts
```

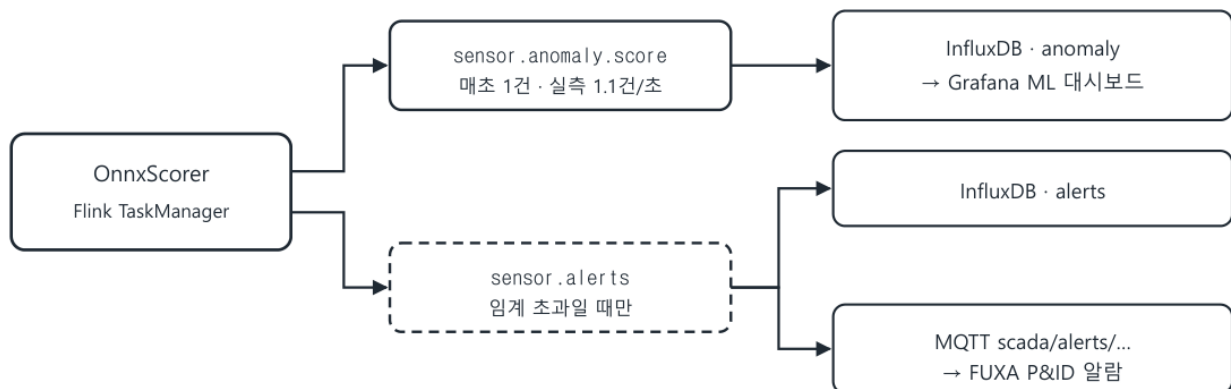
두 갈래로 갈립니다.

sensor.anomaly.score — 이상이든 아니든 매초 한 건이 나갑니다. 재구성 오차, 그때 쓴 임계치, 이상 여부, 추론 지연, 기여 상위 3 개 태그가 들어 있습니다. 점수의 정상 구간도 남겨야 "평소에 이 정도였는데 지금 올랐다"를 볼 수 있기 때문입니다. sensor.alerts — mse > threshold 일 때만 나갑니다. Flink 의 사이드 아웃풋(side output)이라는 기능으로 분류에서 결과지를 뽑아내는 구조입니다. 이 토픽은 규칙 계층의 SQL 잡들도 함께 쓰므로, 규칙 알람과 ML 알람이 한 토픽에서 합류합니다. 그래서 알람 메시지에 detector 필드 가 있고 거기에 TIER1_RULE / TIER1_ZSCORE / TIER1_CEP / TIER2_ML 이 찍힙니다.

그 뒤 두 토픽은 Telegraf 가 받아 나릅니다(telegraf/sink.conf , telegraf/alert-republish.conf).

토픽	가는 곳	거기서 무엇이 되나
sensor.anomaly.score	InfluxDB 의 anomaly 측정 값	Grafana 대시보드 "AR-100 · ML 이상 탐지"의 재구성 오차 vs 임계치 그래프, 추론 지연, 기여 센서 표
sensor.alerts	InfluxDB 의 alerts 측정 값	알람 이력 대시보드
sensor.alerts	MQTT 토픽 scada/alerts/<태그명>	FUXA 의 P&ID 화면에서 해당 설비 심볼 점멸·알람 배너

마지막 줄이 이 시스템을 닫는 고리입니다. 오토인코더가 낸 판정이 Kafka 와 MQTT 를 거쳐 운전원이 실제로 보고 있는 공정 도면 위로 되 돌아옵니다. 여기까지 와야 "탐지했다"가 의미를 갖습니다.



원문 도해 · 점수와 경보의 출력 경로 (원문 222 쪽)

drift 시나리오, 이 계층의 존재 증명

머신러닝 계층을 넣고도 "규칙으로도 잡히는 고장"만 시연하면 아무것도 증명하지 못합니다. 규칙으로는 절대 안 잡히는데 ML 로는 잡히는 고장이 있어야 이 계층의 존재 이유가 성립합니다. AR-100 의 drift 시나리오가 그 목적으로 설계됐습니다.

드리프트(drift)는 값이 튀지 않고 서서히 밀려 나가는 변화를 말합니다. 센서 캘리브레이션 틀어짐, 촉매 열화, 스케일 부착 같은 실제 고장이 이 모습입니다.

설계의 핵심은 정상 상태의 상관을 일부러 뒤집는 것입니다. AR-100 에서 pH 와 전도도는 -0.934 의 역상관입니다. pH 가 오르면 전도도가 내려갑니다. 이 고장은 두 태그를 동시에 올립니다. 그러면 각 태그의 값은 여전히 평소 운전 범위 안에 있지만, 둘의 관계는 물리적으로 있을 수 없는 모양이 됩니다.

```
# 정상 상태에서 pH↑ ⇒ CT↓ 이지만, 이 고장은 둘을 동시에 상승시켜
# 센서 간 상관 구조를 붕괴시킨다. 각 태그는 규격 내에 머무른다.
scale = 1.0 if tag == "pH-101" else 1.6
value += f.magnitude * ramp * scale
```

magnitude 는 plant.yaml 에서 0.55 이고 ramp = min(1.0, f.elapsed / 45.0) 이므로, pH 는 최대 +0.55, 전도도는 최대 +0.88(0.55×1.6)만큼 밀립니다. 45 초에 걸쳐 서서히 커지고 그 뒤로는 더 커지지 않습니다. 계단이 아니라 경사입니다. 다만 이 경사도 Z-Score 의 60 초 창보다는 빠르기 때문에, 실측에서는 Z-Score 가 반응했습니다(아래 실측 상사).

규칙 기반 탐지는 관계를 보지 않으므로 여기서 아무 반응을 하지 않습니다. pH-101 은 규격이 있는데 그 안에 머물렀고, CT-101 은 애초에 규격이 정의돼 있지 않아 규칙의 대상이 아닙니다.

AR-100에서는, drift 주입 시 실측값입니다(docs/journal/logs/05-model-training.txt).

```
정상 : pH=7.413 CT=3.384 corr=-0.934 ← 역상관
drift : pH=7.950 CT=4.246 corr=+0.992 ← 동반 상승, 구조 붕괴
      pH 최대 7.978 (USL 8.4 여유 0.42) → 단일 임계치 무반응 확인
```

"둘 다 규격 안에 있다"는 말은 정확하지 않으므로 나눠서 씁니다. 두 태그가 규격 위반 규칙에 안 걸린다는 결론은 같지만, 이유가 다릅니다.

태그	규격 (LSL / USL)	고장 중 실측	규칙이 안 울리는 이유
pH-101	6.2 / 8.4	최대 7.978	상한까지 0.42 남긴 채 끝까지 규격 안에 머무름
CT-101	없음 / 없음	3.384 → 4.246	규격이 정의돼 있지 않아 규칙의 판정 대상 자체가 아님

근거는 simulator/plant.yaml 의 태그 정의와 거기서 생성된 flink/sql/tag_limits.csv 입니다. CT-101 줄은 CT-101,ms/cm,, 으로 상·하한 자리가 모두 비어 있고, 규칙 SQL 의 조건은 WHERE (l.usl IS NOT NULL AND ...) OR (l.lsl IS NOT NULL AND ...) 이므로 이 태그는 어떤 값이 와도 걸리지 않습니다. 전도도는 운전 조건에 따라 폭넓게 변하는 값이라 현장에서도 고정 규격을 두지 않는 경우가 많습니다.

그래서 이 시나리오의 논증은 이렇게 읽어야 정확합니다. 규격이 있는 태그는 규격을 넘지 않았고, 다른 하나는 규격이라는 장치 자체가 없다. 그러므로 규격 규칙으로는 원리적으로 이 고장을 알 수 없다. 여기에 더해 CEP 패턴에 pH 와 전도도의 조합은 등록돼 있지 않습니다. 규격 규칙과 CEP 는 침묵합니다. — 다만 Z-Score 는 다릅니다. 바로 아래를 보십시오.

실측으로 이 문단을 고칩니다 — Z-Score 는 반응합니다.

2026-09-17 에 원문 작성자의 PC 에서 drift 를 직접 주입하고 같은 길이(93 초) 세 구간을 비교했습니다. 주입 전 0 건 · 주입 종료 후 0 건이던 Z-Score 알람이, 주입 중에는 pH-101 11 건 · CT-101 9 건 나왔습니다. 첫 알람은 주입 4 초 뒤입니다.

```
06:49:26 pH-101 z=4.13 (μ=7.379, σ=0.0103, 최근 5 중 3 회 위반)
06:49:27 CT-101 z=4.83 (μ=3.435, σ=0.0234, 최근 5 중 3 회 위반)
```

이유는 창 길이입니다. flink/sql/03_tier1_zscore.sql 은 μ 와 σ 를 ROWS BETWEEN 59 PRECEDING AND CURRENT ROW , 즉 60 초 이동창으로 구합니다. 그런데 드리프트의 경사는 45 초 입니다 — 창보다 빠릅니다. 그래서 μ 가 아직 주입 전 평균(7.379)에 머물러 있는 동안 값만 올라가고, 정상 운전의 σ 가 0.01 수준이라 z 가 곧바로 3.5 를 넘어섭니다. Z-Score 를 정말 눈멀게 하려면 경사가 창보다 느려야 합니다(그래야 μ 가 따라 올라갑니다).

그러면 "임계치는 못 잡는다"는 여전히 맞습니다 — 규격 규칙이 침묵한 것은 위에서 적은 이유 그대로이고, 실측에서도 pH·CT 의 규격 알람은 0 건이었습니다. 틀린 것은 여기에 Z-Score 와 CEP 까지 묶어 "규칙 세 가지 전부 침묵한다"고 넓힌 부분입니다. 시뮬레이터가 스스로 돌려주는 시나리오 설명도 "단일 임계치 무반응"까지만 적고 있습니다.

왜 레포 기록과 다른가는 확인하지 않았습니. 레포의 06-detection-results.txt 는 다른 환경·다른 실행의 기록이고, 이 문서는 원문 작성자의 PC 에서 지금 실행한 결과를 그대로 적었습니다. 둘 중 하나가 틀렸다고 단정하지 않습니다.

그 순간 ML 계층이 낸 알람입니다(docs/journal/logs/06-detection-results.txt).

```
✓ drift → TIER2_ML
재구성오차 0.03386 > 임계 0.02913
기여 상위: CT-101(52%), pH-101(30%), IT-102(8%)
추론 0.15ms
```

기여도 상위 두 개가 정확히 고장이 건드린 두 태그입니다. 운전원은 "이상입니다"가 아니라 "전도도와 pH 를 보세요"를 받습니다.

이 결과에 이르기 전의 함정 하나. 처음 실행에서는 이렇게 나왔습니다.

```
재구성오차 0.04270 > 임계 0.02913
기여 상위: VT-101(69%), CT-101(19%), IT-102(3%)
```

VT-101 은 교반기 진동입니다. drift 는 진동을 건드리지 않습니다. 원인은 직전에 실행한 bearing_wear 시험의 진동이 아직 회복 중이었던 것입니다. 물리 공정은 상태를 갖습니다. 시험 사이에 안정화 시간을 두지 않으면 앞선 시험이 뒤 시험을 오염시킵니다. 공정을 90 초 안정화한 뒤 재실행하니 정상 결과가 나왔습니다. 실제 플랜트 시운전에서든 그대로 적용되는 원칙입니다.

왜 같은 drift 가 47.15 와 0.03386 으로 다르게 나오는가

이 장에서 가장 헷갈리는 지점입니다. 같은 drift 고장인데 자가검증 로그에서는 평균 재구성오차가 55.77(2 차) 또는 46.26(3 차), 09-metrics.md 에서는 47.15 인데, 라이브 알람에서는 0.03386 입니다. 1,000 배가 넘게 차이 납니다. 모델이 바뀐 것도, 고장이 약해진 것도 아닙니다. 두 숫자가 재고 있는 대상이 다릅니다.

비교 항목	자가검증의 47.15 (또는 55.77)	라이브 알람의 0.03386
어디서 나온 값	학습 직후 validate() 가 파이썬에서 계산	운영 중인 Flink 잡이 Kafka 로 발행한 알람
측정 대상	400 초 구간 중 뒤쪽 절반의 창 전부의 평균	알람 한 건의 값
고장이 얼마나 전개됐나	램프 45 초가 끝난 포화 상태. pH +0.55, CT +0.88 이 온전히 걸림	램프 도중. 점수가 임계치를 막 넘은 첫 순간
코드 근거	<code>X = X[len(X) // 2:] (ml/train.py 의 validate)</code>	<code>next((a for a in alerts if a.get("detector") == want_detector), None) (scripts/verify.py)</code> — 받은 알람 중 첫 건을 기록
숫자의 성격	여러 표본의 평균	단일 표본

정리하면 이렇습니다. 자가검증은 "고장이 완전히 자리 잡았을 때 이 모델이 얼마나 크게 반응하는 가"를 재고, 라이브 알람은 "이 모델이 언제 처음으로 이상이라고 말했는가, 그때 값은 얼마였는가"를 기록합니다. 전자는 판별력의 크기를, 후자는 탐지의 시점을 보여 줍니다. 서로 대체할 수 없는 두 숫자입니다.

라이브 쪽 점수는 거기서 멈추지 않습니다. 잡은 1 초마다 다시 판정하므로, 램프가 진행되는 동안 매 초 새 알람이 계속 나옵니다. 저널에 남은 것은 그 연속된 알람 중 첫 건일 뿐이고, 그 이후 값이 얼마 까지 올라갔는지는 기록돼 있지 않습니다.

부차적인 차이도 하나 있습니다. 자가검증은 파이썬이 합성한 행렬을 onnxruntime 에 바로 넣지만, 라이브는 Kafka → 결측 보간 → LOCF 텐서 조립을 거친 값을 넣습니다. 경로가 같지 않습니다. 다만 1,000 배 차이를 만든 주된 요인은 위 표의 측정 시점과 평균 여부입니다.

여기서 얻을 실무 교훈. 재구성 오차는 절대 단위가 없는 숫자입니다. "0.03 이면 심각한가?"라는 질문에는 답이 없고, "그 시점의 임계치보다 큰가"만 의미가 있습니다. 그래서 알람 문구가 항상 재구성오차 0.03386 > 임계 0.02913 처럼 임계치를 함께 적습니다. 점수만 적어 두면 나중에 아무도 해석할 수 없습니다.

측정 조건 주석: 원문은 이 차이를 포화 구간의 평균과 최초 라이브 경보라는 측정 대상 차이로 설명합니다. 다만 앞 절의 로그에는 서로 다른 임계치·학습 실행이 함께 등장합니다. 첨부 자료만으로 모든 비교가 완전히 동일한 모델 버전이라고 확정하지 않고, 각 수치의 임계치와 실행 조건을 함께 남깁니다. [원문 214, 215, 226 쪽]

AR-100 에서는. 시나리오별 반응 크기를 실측 로그 그대로 옮깁니다. 앞서 본 대로 이 값들은 서로 다른 실행에서 나온 것이므로 실행별로 열을 나눕니다(docs/journal/logs/05-model-training.txt).

시나리오	[2 차] 평균오차 / 탐지율	[3 차] 평균오차 / 탐지율	담당
정상 운전	0.02053 / 0.5 %	0.01989 / 0.0 %	— (알람 없어야 정상)
drift	55.77 / 100 %	46.26 / 100 %	규격 규칙으로는 불가 · 실측에선 Z-Score 도 반응(본문 실측 상자)
bearing_wear	146.13 / 100 %	642.79 / 100 %	CEP + ML 중복 탐지
spike	17.43 / 100 %	11.36 / 100 %	규칙이 먼저 잡음

시나리오	[2 차] 평균오차 / 탐지율	[3 차] 평균오차 / 탐지율	담당
noise	0.03104 / 3.6 %	0.02796 / 0.5 %	Z-Score 전담 — ML 무반응이 정상

숫자를 나란히 놓으면 고장마다 반응 크기가 전혀 다르다는 것이 보입니다. bearing_wear 가 가장 크게 튀는데(3 차에서 642.79), 교반기 전류와 진동이라는 두 태그가 동시에 크게 벗어나고 그것이 레벨·부하와 맺고 있던 관계까지 한꺼번에 무너뜨리기 때문입니다. drift 는 그보다 한 자릿수 작지만 여전히 정상의 2,000 배가 넘습니다. 반면 noise 는 정상(0.0199)과 거의 구별되지 않습니다 (0.028). 이것이 실패가 아니라 설계입니다. 노이즈 급증은 태그 하나의 분산이 커지는 현상이라 태그 간 관계가 무너지지 않고, 그래서 Z-Score 가 잡고 ML 은 침묵합니다.

참고로 09-metrics.md 의 요약표는 시드 3 종을 고정한 또 다른 실행의 값으로 정상 0.0126 / 0.0%, drift 47.15 / 100%, 임계치 0.035081 을 싣고 있습니다. 같은 표의 bearing_wear · spike 칸에는 오차값 자리에 "100 %"가 적혀 있어 수치로 인용할 수 없고, noise 칸은 "낮음"으로만 적혀 있습니다. 그래서 위 표는 로그 원본을 썼습니다.

AR-100에서는. 계층별로 실제 나간 알람 문구입니다(docs/journal/09-metrics.md , logs/06-detection-results.txt).

그리고 정상 운전 90 초 동안 전 계층 통틀어 알람 0 건입니다. 측정 방법도 기록돼 있습니다. sensor.alerts 토픽의 오프셋을 재고, 90 초 기다린 뒤 다시 재서 차이를 본 것입니다.

```
$ a=$(off); sleep 90; b=$(off); echo "$((b-a))건"
0건
```

이 0 건을 만든 것은 오토인코더가 아니라 Z-Score 의 지속성 요건입니다. 같은 로그에 이렇게 적혀 있습니다.

```
(참고: Z-Score 지속성 조건 도입 전에는 120 초간 8 건 오탐)
[TIER1_ZSCORE] TT-101 = 71.87803 :: TT-101 z=3.09 (μ=72.319, σ=0.1431)
[TIER1_ZSCORE] TT-101 = 71.6813 :: TT-101 z=3.02 (μ=72.267, σ=0.1941)
```

autopilot 이 설정치를 45 초에 걸쳐 밀 때마다 이동평균이 뒤쳐져 z 가 3 을 살짝 넘었습니다. 임계를 3.5 로 올리고 "최근 5 샘플 중 3 회 이상"을 걸어서 이 순간 위반을 걸러 낸 것이 ADR-006 이고, 그 결과가 0 건입니다. ML 계층을 넣었다고 오탐이 0 이 된 것이 아닙니다. 규칙 쪽을 고쳐서 0 이 됐습니다.

이상탐지에서 "잡는다"만큼 "안 올린다"가 중요합니다. 오탐이 잦으면 운전원이 알람을 끕니다. 그러면 진짜 알람도 함께 꺼집니다.

AR-100에서는. 추론이 Java 인 것은 설계 취향이 아니라 실사 결과였습니다(docs/journal/08-decisions.md 의 ADR-002). 원래 계획은 PyFlink 였습니다. PyFlink 는 Flink 의 Python API 로, 학습과 추론을 같은 언어로 묶을 수 있어 편합니다.

막힌 지점을 이해하려면 세 단어가 필요합니다. CPU 아키텍처는 CPU 가 알아듣는 기계어 명령의 규격입니다. 인텔·AMD 계열이 x86_64 (amd64 라고도 씁니다), 애플 실리콘·라즈베리파이 등 ARM 계열이 aarch64 (arm64)입니다. 둘은 기계어가 아예 달라서 한쪽용으로 컴파일된 프로그램이 다른 쪽에서 그냥 돌지 않습니다. 휠(wheel)은 파이썬 패키지를 미리 컴파일해 둔 배포 파일이고, 아키텍처별로 따로 만들어 올립니다. QEMU 에뮬레이션은 다른 아키텍처의 기계어를 소프트웨어로 한 명령씩 통역해 실행하는 방식입니다. 돌기는 도는데 느립니다.

구현 환경이 Apple Silicon(linux/arm64)이었고, PyPI 에 apache-flink 의 linux-aarch64 휠이 존재하지 않았습니다. 있는 것은 macosx_arm64 와 manylinux_x86_64 뿐이었습니다. 소스에서 직접 빌드 하는 길이 있지만 Flink 규모에서는 비현실적입니다.

대안으로 간 Java + com.microsoft.onnxruntime 은 결과적으로 설계 보고서 원문에 더 정확히 부합했습니다. 이 라이브러리는 JNI(Java Native Interface, 자바가 C/C++로 만든 네이티브 코드를 직접 부르는 규격)로 C++ 네이티브를 감싸고 있어서, 설계 보고서가 말한 "C++ 기반 ONNX Runtime 엔진을 임베디드 형태로 직접 로드"가 문자 그대로 성립합니다. 네이티브 포함 여부는 JAR 을 열어 확인했습니다.

```
$ unzip -l onnxruntime-1.20.0.jar | grep linux-aarch64
ai/onnxruntime/native/linux-aarch64/libonnxruntime.so      13,648,160
ai/onnxruntime/native/linux-aarch64/libonnxruntime4j_jni.so 133,256
```

ADR-002 가 기각한 대안도 함께 기록돼 있습니다.

기각한 대안	기각 사유
PyFlink	arm64 휠 부재. 소스 빌드는 비현실적
별도 Python 추론 사이드카 (REST/gRPC)	PDF p.10 이 명시적으로 "RPC Callout 지양"을 권고. 네트워크 홈 추가
amd64 에뮬레이션(QEMU)	성능 저하가 심해 추론 지연 측정이 무의미해짐

대가도 정직하게 적혀 있습니다. Jackson 라이브러리 버전 충돌로 NoSuchMethodError 를 겪었고, Maven 최초 빌드에 약 3 분이 걸립니다. 이 한 줄에 자바 용어가 몰려 있으니 풀어 둡니다.

NoSuchMethodError — "컴파일할 때는 있던 메서드가 실행할 때 보니 없다"는 자바 오류입니다. 문법 오류가 아니라 빌드 시점의 라이브러리와 실행 시점의 라이브러리가 다르다는 신호입니다. 셰이드 JAR(shaded JAR) — 내 코드와 내가 쓰는 라이브러리를 전부 하나의 JAR 파일로 합쳐 만든 것입니다. 실행할 때 의존 라이브러리를 따로 챙기지 않아도 되지만, 합치는 과정에서 같은 라이브러리의 다른 버전이 섞여 들어갈 수 있습니다. 전이 의존(transitive dependency) — 내가 직접 적지 않았는데, 내가 쓰는 라이브러리가 끌고 들어오는 라이브러리입니다. A 를 쓰겠다고 적었더니 A 가 B 를 끌고 오고 B 가 C 를 끌고 오는 식입니다.

BOM(Bill of Materials, 자재 명세서) — 서로 관련된 라이브러리들의 버전을 한곳에 못 박아 두는 목록 파일입니다. Maven 에 BOM 을 걸면, 전이 의존으로 어떤 버전이 달려 오든 BOM 이 적어 둔 버전으로 통일됩니다.

실제 증상은 이랬습니다. 셰이드 JAR 안에서 jackson-annotations 와 jackson-databind 는 2.17.2 인데 jackson-core 만 전이 의존으로 2.15.2 가 선택돼 있었습니다. 서로 다른 버전의 조각들이 한 JAR 안에 섞여 있었으니 실행 중에 없는 메서드를 부른 것입니다. jackson-bom 2.17.2 를 걸어 셋을 같은 버전으로 맞춰 해결했습니다. 처음에는 "외부와 충돌하는 것"으로 보고 패키지 이름을 바꾸는 relocate 를 시도했는데, 같은 오류가 바뀐 이름으로 그대로 재발해서 충돌이 JAR 내부에 있음을 알게 된 과정도 기록돼 있습니다.

같은 자리의 다른 도구들 — 이상탐지 알고리즘 단순 임계치 / Z-Score — 규칙 계층에 이미 있습니다. 태그 하나만 붙입니다. 값이 튀는 이상에는 가장 빠르고 확실하며 설명도 쉽습니다. 관계 이상은 원리적으로 못 붙입니다. ML 이 이걸 대체하는 것이 아니라 못 보는 자리를 메웁니다. PCA(주성분 분석, Principal Component Analysis) — 오토인코더와 목적이 거의 같습니다. 데이터를 저 차원으로 압축했다가 복원해 그 차이를 보는 방식입니다. 차이는 선형만 표현할 수 있다는 점입니다. 압축 과 복원이 전부 곱하고 더하기로만 이뤄지므로, 꺾이는 관계나 문턱이 있는 관계는 표현하지 못합니다. 오토인코더는 ReLU 덕분에 그것을 배웁니다. 대신 PCA 는 학습이 몇 초면 끝나고 어느 방향이 주된 변동인지 눈으로 볼 수 있어 해석이 쉽습니다. 먼저 PCA 로 재구성 오차를 재 보고 부족할 때 오토인코더로 올라가는 순서가 실무의 정석입니다. 공정이 온화한 선형 영역에서 운전되면 PCA 만으로 충분한 경우가 많습니다. Isolation Forest(고립 숲) — 설계 보고서가 Autoencoder·LSTM 과 나란히 언급한 알고리즘입니다. 데이터를 무작위로 자르고 또 잘라 나가면서 "몇 번 만에 혼자 떨어져 나오는가"로 이상 점수를 매깁니다. 다수와 다른 점일수록 적은 횟수로 고립되기 때문입니다. 학습이 매우 빠르고 조절할 값이 적으며 신경망이 필요 없습니다. 대신 시간 순서를 보지 않고, 어느 센서 때문인지 역추적하기가 오토인코더보다 까다롭습니다. One-Class SVM(단일 클래스 서포트 벡터 머신) — SVM(Support Vector Machine)은 데이터를 가르는 경계면을 찾는 고전 알고리즘입니다. 보통은 두 부류를 가르는 데 쓰지만, One-Class 는 정상 데이터 한 부류만 주고 그것을 가능한 한 빠듯하게 감싸는 경계면을 찾습니다. 경계 바깥이면 이상입니다. 이론이 탄탄하고 데이터가 적을 때 강하지만, 데이터가 많아지면 계산량이 데이터 수의 제곱에 가깝게 커져 스트리밍에는 잘 안 맞습니다. LSTM 오토인코더 — 앞의 표에서 비교했습니다. 시간 의존이 길 때 유리하고, 대신 무겁습니다. 지도 학습 분류기 — 랜덤 포레스트(Random Forest)는 조금씩 다르게 만든 결정나무 수백 그루의 다수결로 답을 내는 방식이고, XGBoost 는 나무를 한 그루씩 차례로 쌓되 앞 나무가 틀린 부분을 다음 나무가 메우도록 학습시키는 방식(부스팅)의 대표 구현체입니다. 둘 다 표 형태 데이터에서 가장 정확한 측에 들지만, 공통의 전제가 있습니다. 고장 라벨이 충분히 쌓여 있어야 하고, 학습 때 본 고장만 알아봅니다. 설비 이상탐지의 출발점이 될 수는 없고, 오토인코더가 이상 구간을 표시해 주고 사람이 원인을 확정해 라벨이 쌓인 다음 단계에 올라가는 도구입니다.

같은 자리의 다른 도구들 — 모델 서빙 방식 임베디드 ONNX Runtime (이 예제의 선택) — 스트림 엔진 프로세스 안에서 직접 실행. 네트워크 홈 0, 실측 0.126~0.180 ms. 대신 모델 교체가 잡 수명과 엮이고, 언어 경계를 넘기 위해 ONNX 변환이 반드시 필요합니다. TensorFlow Serving / TorchServe / KServe / Seldon Core — 모델을 독립 서버로 띄우는 정석 방식입니다. 버전 관리에 더해 세 가지 운영 장치가 다 갖춰져 있습니다. 카나리(canary) 배포는 새 버전을 전체가 아니라 요청의 일부(예: 5%)에만 먼저 태워 보고 문제가 없으면 비율을 늘려 가는 방식으로, 탄광의 카나리아에서 온 이름입니다. A/B 테스트는 두 버전에 요청을 나눠 주고 어느 쪽 결과가 더 나은지 통계로 비교하는 것입니다. 오토스케일(autoscale)은 요청이 몰리면 서버 인스턴스를 자동으로 늘리고 한가해지면 줄이는 기능입니다. 대가는 매 건마다 네트워크 왕복과 단일 병목입니다. 설계 보고서가 "RPC Callout 을 지양해야 한다"고 못 박은 대상이 정확히 이

구성입니다. 초당 수십 건이면 문제없지만, 초당 수만 건의 센서 스트림을 태그 단위로 돌리는 자리에는 맞지 않습니다. TorchScript — PyTorch 모델을 직렬화하는 PyTorch 자체 포맷입니다. PyTorch 안에서는 ONNX 보다 변환 이 매끄럽지만 Python/C++ 런타임에 묶입니다. JVM 에서 부르려면 결국 별도 브리지가 필요해, 언어를 건너야 하는 이 구조에서는 ONNX 가 더 맞습니다. PMML(Predictive Model Markup Language) — XML 기반의 오래된 모델 교환 표준입니다. 로지스틱 회귀, 의사결정나무 같은 전통 모델은 잘 표현하지만 딥러닝 신경망 표현력이 부족합니다. ONNX 가 사실상 이 자리를 대체했습니다. 스트림 엔진 내장 ML 블록 — Apache StreamPipes 처럼 이상탐지 알고리즘을 그래픽 블록으로 조립하는 노코드 플랫폼이 있습니다. 설계 보고서의 평가는 이렇습니다. "센서 데이터 어댑터, 윈도우 변환 필터, 이상 탐지 알고리즘 등을 웹 파이프라인 에디터 상에서 그래픽 블록 조립 형태로 구성할 수 있어 현장 도메인 엔지니어의 접근성이 매우 높다. 그러나 고성능 분산 분기 연산이나 복잡한 딥러닝 텐서 처리에는 제약이 따른다." 블록으로 조립할 수 있는 것은 미리 준비된 알고리즘뿐이고, 120 차원 텐서를 직접 조립해 넣는 이 예제 같은 작업은 그 틀 밖입니다.

반대로 설계 보고서는 이 프로젝트가 택한 Flink-Kafka 조합의 스트림 분석·ML 항목을 "Flink 기반 분산 상태 연산 및 ONNX 실시간 서빙"으로, 분산 확장성을 "극대화(Kafka 파티션 및 Flink 슬롯 선형 확장)"로 평가했습니다. Kafka 파티션(partition)은 하나의 토픽을 여러 갈래로 쪼갠 단위입니다. 갈래가 6 개면 컨슈머 6 개가 나눠 읽을 수 있어 처리량이 거의 비례해서 늘어납니다. AR-100 은 sensor.telemetry.raw 를 파티션 6 개 로 잡았습니다(태그 12 개를 2 개씩 나눠 담을 수 있는 크기). "선형 확장"이란 파티션과 슬롯을 늘린 만큼 처리량이 늘어난다는 뜻이고, 이것이 가능한 이유는 태그별로 처리가 독립적이라 쪼갤 수 있기 때문입니다.

주의. 머신러닝 계층을 넣었다고 규칙 계층을 걷어내면 안 됩니다. 이유가 넷입니다. 첫째, 속도. 과압 같은 안전 직결 이상은 규칙이 값 하나만 보고 즉시 잡습니다. ML 은 창이 잘 때까지 기다립니다 (AR-100 은 기동 후 최소 10 초). 둘째, 설명 가능성. "PT-101 = 7.493 barg / 규격 6.0 초과"는 누구나 즉시 이해하고 조치할 수 있지만 "재구성오차 0.03386 > 임계 0.02913"은 해석이 필요합니다. 셋째, 신뢰의 근거. 규칙은 코드를 읽으면 왜 올랐는지 100% 설명됩니다. 모델은 그렇지 않습니다. 실제 사고 조사에서 근거로 서는 것은 규칙 쪽입니다. 넷째, 오탐 억제는 규칙 쪽 일이기도 합니다. 이 시스템의 "정상 90 초 알람 0 건"을 만든 것은 오토인코더가 아니라 Z-Score 의 지속성 요건이었습니다. 두 방식은 합쳐서 하나의 탐지 체계입니다.

이 장을 손으로 확인하는 방법

교재를 읽는 것만으로 끝내지 않으려면 다음 네 가지를 직접 돌려 보면 됩니다. 어느 것도 자바·파이썬 코드를 고칠 필요가 없고 설정 파일만 바꿉니다. 다만 바꾼 뒤 적용하는 절차가 실험마다 다릅니다. 모델은 학습기가 다시 만들어야 하고, 잡은 새 파일을 다시 읽어야 하기 때문입니다.

실험	바꿀 것	적용 절차	확인할 것
운전 변동의 효과	ml/train.yaml 의 autopilot: true → false	재학습 + 잡 재기동	학습 로그의 상관계수가 0 근처로 내려가는가. 자가검증의 drift 탐지율이 왜 떨어지는가
임계치 트레이드 오프	ml/train.yaml 의 threshold.percentile 을 99.0 / 99.5 / 99.9	재학습 + 잡 재기동	자가검증의 정상 오탐률과 drift 탐지율이 반대 방향으로 움직이는 곡선
시간 창 역할	ml/train.yaml 의 window_steps 와 flink/onnx-job/job.properties 의 window.steps 를 둘 다 10 → 1	재학습 + Flink 이미지 재빌드 + 잡 재기동	어떤 종류의 이상을 놓치게 되는가. 순간값만으로는 못 보는 것이 무엇인가
결측 정책의 차이	job.properties 의 interpolation.mode 를 locf 로	Flink 이미지 재빌드 + 잡 재기동 → make fault-dropout	clean 토픽 값 패턴이 균등 증가에서 계단형으로 바뀌는지

재학습 명령은 make train 이고, 그 실체는 학습기 컨테이너를 한 번 돌리는 것입니다.

```
make train
# = docker compose run --rm model-trainer
make jobs
# = docker compose run --rm flink-job-submitter
```

주의 — make train 만으로는 새 모델이 반영되지 않습니다. 이유가 둘입니다.

첫째, OnnxScorer.open() 이 모델 파일을 잡이 뜰 때 한 번만 읽습니다. 공유 볼륨의 model.onnx 가 새것으로 바뀌어도, 이미 들고 있는 잡은 메모리에 올려 둔 옛 모델을 계속 씁니다.

둘째, 잡 제출 스크립트가 멍등하게 짜여 있습니다. flink/submit-jobs.sh 에 다음 줄이 있습니다.

```
running=$(curl -sf "http://${JM}/jobs/overview" | grep -o '"state":"RUNNING"' | wc -l)
if [ "${running:-0}" -gt 0 ]; then
    echo "✓ 이미 ${running}개 잡이 실행 중 — 제출을 건너뛰니다."
    exit 0
fi
```

즉 RUNNING 상태인 잡이 하나라도 있으면 제출 자체를 건너뛰니다. 기동 시 자동 제출이 두 번 일어나지 않게 하려고 넣은 장치인데, 재학습 상황에서는 이것 때문에 make train && make jobs 를 그대로 쳐도 아무 일이 일어나지 않습니다.

그래서 재학습 후에는 잡을 내린 뒤 다시 제출해야 합니다.

```
make train
# 잡을 전부 내린다 (이 구성은 JobManager 재기동 시 잡 목록이 남지 않음)
docker compose restart flink-jobmanager flink-taskmanager
make jobs # 4개 잡 재제출 — SQL 3개 + ONNX 1개
```

잡을 개별로 내리려면 Flink UI(잡 선택 → Cancel) 또는 flink cancel <JobID> 를 쓰면 되는데, SQL 잡 3개가 남아 있으면 위 조건에 걸려 제출이 건너뛰어지므로 4개 전부 내려야 합니다.

이 재기동 절차는 submit-jobs.sh · OnnxScorer.java · flink/conf/config.yaml 의 내용에서 도출한 것입니다. 프로젝트의 RUNBOOK.md 는 이 자리에 make train && make jobs 만 적어 두었고, 잡을 내리는 단계가 빠져 있습니다. 이 교본을 쓰면서 실제로 실행해 확인한 기록은 없으므로, 처음 돌릴 때는 make jobs 의 출력에 "제출을 건너뛰니다"가 뜨지 않는지 반드시 눈으로 확인하십시오.

세 번째 실험에서 window.steps 를 한쪽만 바꾸면 어떻게 되지도 정확히 알아 둘 만합니다. 흔히 "모델이 안 열린다"거나 "조용히 틀린 값이 나온다"고 오해하는데 둘 다 아닙니다. 모델 세션을 만드는 createSession() 은 설정 파일의 window.steps 와 아무 상관이 없으므로 정상적으로 성공합니다. 잡도 일단 뜹니다. 문제는 첫 추론에서 드러납니다. ONNX 익스포트 시 dynamic_axes 가 0번 축(배치)만 가변으로 열어 두었으므로 1번 축은 120으로 고정돼 있고, 창을 1로 줄이면 12차원 텐서가 들어가 session.run() 이 형상 불일치 예외를 던지며 즉시 실패합니다. 조용히 틀린 값을 내지 않고 소리를 내며 멈춥니다. 이것은 다행한 설계입니다. 조용히 틀리는 쪽이 훨씬 위험하기 때문입니다.

그러므로 ml/train.yaml 의 window_steps 와 flink/onnx-job/job.properties 의 window.steps 는 항상 같은 값이어야 합니다. 앞서 본 것처럼 추론 쪽은 메타 파일의 window_steps 를 읽지 않고 설정 파일의 값을 쓰기 때문에, 두 파일을 사람이 맞춰 주는 수밖에 없습니다. 이건 함정이 아니라 설계상 반드시 지켜야 하는 계약이고, 두 파일 모두에 "반드시 일치"라는 주석이 달려 있는 이유입니다.

이 권에서 통합한 반복 설명

반복 주제	편집본의 정본 위치
중복 제거·전달 보장	4장에 낮은 정상 데이터 손실과 싱크 재출력의 차이 통합
시간·상태·재시작	5장에 정본 설명, CEP·ONNX에서는 해당 기능의 차이만 연결
보간·품질	5장에 실측/추정·linear/LOCF·STALE의 구현 차이 통합
drift 해석	7장에 초기 의도와 Z-Score도 반응한 후속 실측을 함께 유지

분량 감소는 반복 문장·표·설정의 통합과 연속 페이지 배치에서 만들었습니다. 원문에 없던 기능을 보충하거나, 서로 다른 측정을 임의로 하나의 정답으로 맞추지는 않았습니다. 원문 전체와 이 편집본은 설치·운영 시 함께 확인할 수 있는 별개의 자료입니다.